VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING

REPORT
CAPSTONE PROJECT

REINFORCEMENT LEARNING-BASED SYMBOLIC REASONING FOR COMPLEX AND LOGICAL QUESTION ANSWERING ON HYBRID KNOWLEDGE

Major: COMPUTER SCIENCE

THESIS COMMITTEE: 2CC
SUPERVISOR(s): QUAN THANH THO, PhD
BUI CONG TUAN, MEng
REVIEWER: PHAN TRAN MINH KHUE, PhD
---o0o---

STUDENT 1: THAI QUANG PHAT - 2252606

HO CHI MINH CITY, 06/2026

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING

REPORT

CAPSTONE PROJECT

SEMESTER 252 ACADEMIC YEAR 2025-2026

REINFORCEMENT LEARNING-BASED SYMBOLIC REASONING FOR COMPLEX AND LOGICAL QUESTION ANSWERING ON HYBRID KNOWLEDGE

MAJOR: COMPUTER SCIENCE

COUNCIL:	COMPUTER SCIENCE - 2CC
SUPERVISOR(s):	Quan Thanh Tho, PhD
	Bui Cong Tuan, MSc
CHAIRMAN:	Tran Van Hoai, PhD
SECRETARY:	Nguyen Minh Tam, MSc
MEMBER:	Phan Tran Minh Khue, PhD
REVIEWER:	Phan Tran Minh Khue, PhD

——o0o——

STUDENT 1: Thai Quang Phat - 2252606

HO CHI MINH CITY, June 2026

ĐẠI HỌC QUỐC GIA TP.HCM
TRƯỜNG ĐẠI HỌC BÁCH KHOA

CỘNG HÒA XÃ HỘI CHỦ NGHĨA VIỆT NAM
Độc lập - Tự do - Hạnh phúc

KHOA: KH&KT Máy tính
BỘ MÔN: **Khoa học Máy tính**

NHIỆM VỤ ĐỒ ÁN TỐT NGHIỆP

HỌ VÀ TÊN: **THÁI QUANG PHÁT** MSSV: **2252606**
NGÀNH: **Khoa học Máy tính** LỚP: **CC22KHM1**

1. Đầu đề đồ án:
Nghiên cứu phương pháp học tăng cường cho bài toán hỏi đáp phức tạp và suy luận logic trên nguồn dữ liệu lai

2. Nhiệm vụ (yêu cầu về nội dung và số liệu ban đầu):
Conduct a comprehensive review of multihop question answering (QA) research, focusing on Knowledge Graphs (KGs), Graph Neural Networks (GNNs), Reinforcement Learning (RL), hierarchical reasoning, and symbolic reasoning approaches. Existing benchmark datasets will be analyzed and categorized while establishing the theoretical foundations required for the study.
Develop and refine symbolic-enhanced multihop QA frameworks by constructing and preprocessing knowledge graphs, implementing graph representation learning models, and integrating reinforcement learning techniques to improve reasoning path retrieval, generation, and interpretability.
Conduct extensive experiments on publicly available benchmark datasets to evaluate the effectiveness of the proposed approaches, compare their performance with existing methods, analyze their strengths and limitations, and summarize the findings through reports, presentations, and recommendations for future research directions.

3. Ngày giao nhiệm vụ đồ án: 12/01/2026

4. Ngày hoàn thành nhiệm vụ: 08/05/2026

5. Họ tên giảng viên hướng dẫn **Phần hướng dẫn:**

1) PGS. TS. Quản Thành Thơ

Nội dung và yêu cầu ĐATN đã được thông qua Bộ môn.

Ngày ..... tháng ..... năm .....

TRƯỞNG BỘ MÔN
(Ký và ghi rõ họ tên)

GIẢNG VIÊN HƯỚNG DẪN CHÍNH
(Ký và ghi rõ họ tên)

Quản Thành Thơ

PHẦN DÀNH CHO KHOA, BỘ MÔN:

Người duyệt (chấm sơ bộ): _____
Đơn vị: _____
Ngày bảo vệ: _____
Điểm tổng kết: _____
Nơi lưu trữ luận án: _____

TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KH & KT MÁY TÍNH

CỘNG HÒA XÃ HỘI CHỦ NGHĨA VIỆT NAM
Độc lập - Tự do - Hạnh phúc

TP. HCM, ngày 18 tháng 6 năm 2026

PHIẾU CHẤM BẢO VỆ ĐỒ ÁN/ LUẬN VĂN TỐT NGHIỆP

(Dành cho người hướng dẫn)

1. Họ và tên SV: **Thái Quang Phát**
MSSV: 2252606 Ngành (chuyên ngành): Khoa học Máy tính

2. Đề tài: Nghiên cứu phương pháp học tăng cường cho bài toán hỏi đáp phức tạp và suy luận logic trên nguồn dữ liệu lai

3. Họ tên người hướng dẫn: Quản Thành Thơ

4. Tổng quát về bản thuyết minh:
Số trang: 79 Số chương: 7
Số bảng số liệu: 9 Số hình vẽ: 38
Số tài liệu tham khảo: 43 Phần mềm tính toán: 0
Hiện vật (sản phẩm): 0

5. Tổng quát về các bản vẽ:
- Số bản vẽ:0 Bản A1: 0 Bản A2: 0 Khổ khác: 0
- Số bản vẽ vẽ tay: Số bản vẽ trên máy tính:

6. Những ưu điểm chính của LVTN:
Conducts a thorough review of multi-hop question answering, knowledge graph reasoning, graph neural networks, reinforcement learning, and symbolic reasoning, providing a solid theoretical foundation for the proposed research.
Proposes two complementary frameworks, NeuroPath and SymR, that address key challenges in multi-hop reasoning, including long-horizon decision making, sparse rewards, and reasoning interpretability.
Introduces a symbolic hierarchical reinforcement learning approach that leverages AMR-based subgoal decomposition, enabling more stable, explainable, and logically coherent reasoning paths.
Performs extensive experiments on multiple benchmark datasets and compares the proposed methods against representative neural, reinforcement learning, and LLM-based baselines, demonstrating strong empirical effectiveness.
Published papers at ICCIES 2025 and CITA 2026 conference from the thesis work.

7. Những thiếu sót chính của LVTN:
The proposed framework relies heavily on the quality of the constructed knowledge graph; errors in entity and relation extraction may negatively affect reasoning performance.
The current reward design and manager policy remain relatively simple, leaving room for more adaptive planning and stronger logical constraints.
Experiments are limited to benchmark datasets with pre-constructed knowledge graphs, and the scalability of the framework on dynamic or real-world large-scale knowledge graphs has not been fully investigated.

8. Đề nghị: Được bảo vệ ☑ Bổ sung thêm để bảo vệ □ Không được bảo vệ □

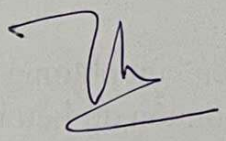
9. 3 câu hỏi SV phải trả lời trước Hội đồng:

Khôn

10. Đánh giá chung (bằng chữ: giỏi, khá, TB): Giỏi Điểm : 10 /10

NGƯỜI VIẾT PHIẾU CHẤM

(ký và ghi rõ họ tên)



Quản Thành Thơ

TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KH & KT MÁY TÍNH

CỘNG HÒA XÃ HỘI CHỦ NGHĨA VIỆT NAM
Độc lập - Tự do - Hạnh phúc

TP. HCM, ngày 16 tháng 6 năm 2026

PHIẾU CHẤM BẢO VỆ ĐỒ ÁN/ LUẬN VĂN TỐT NGHIỆP

(Dành cho người phản biện)

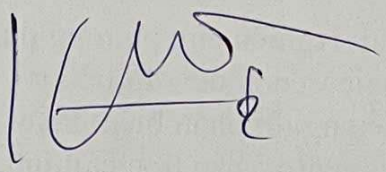
1. Họ và tên SV: **Thái Quang Phát**
MSSV: 2252606 Ngành (chuyên ngành): Khoa học Máy tính

2. Đề tài: Nghiên cứu phương pháp học tăng cường cho bài toán hỏi đáp phức tạp và suy luận logic trên nguồn dữ liệu lai
3. Họ tên người phản biện: Phan Trần Minh Khuê
4. Tổng quát về bản thuyết minh:
Số trang: 79 Số chương: 7
Số bảng số liệu: 9 Số hình vẽ: 38
Số tài liệu tham khảo: 43 Phần mềm tính toán:
Hiện vật (sản phẩm):
5. Tổng quát về các bản vẽ:
- Số bản vẽ: Bản A1: Bản A2: Khổ khác:
- Số bản vẽ vẽ tay: Số bản vẽ trên máy tính:
6. Những ưu điểm chính của LVTN:
The thesis studies Multi-hop Question Answering, in which the system must reason through multiple steps on a Knowledge Graph to find the correct answer. Two main models are proposed: NeuroPath, a neural policy based on a Relational Graph Attention Network with a learned stopping mechanism; and SymR (Symbolic Reinforcement), a Hierarchical RL framework integrated with Abstract Meaning Representation to guide reasoning — evaluated on HotpotQA, 2WikiMultihopQA, and MuSiQue, achieving Exact Match scores of 73.63, 80.91, and 85.83 respectively.
-The thesis addresses one of the most technically challenging problems in NLP and does so with novel contributions. NeuroPath's Relational Graph Attention Network combined with a learned stopping mechanism, and SymR's Hierarchical Reinforcement Learning framework with AMR-guided subgoals, are both architecturally distinct and clearly motivated ideas. The mathematical formulations are presented accurately and completely throughout.
- The experimental results are convincing, surpassing all tested baselines, and qualitative reasoning path analysis adds interpretive depth beyond standard metrics. Most importantly, both models have been accepted at international conferences — NeuroPath in Springer Nature 2026 and SymR at CITA 2026 — providing independent peer-reviewed validation of the work's quality, which is an exceptional achievement for an undergraduate thesis.
7. Những thiếu sót chính của LVTN:
- Section 6.4 on hyperparameter settings contains essentially no content, stating only that values were 'selected empirically' — no learning rate, batch size, hidden dimension, or loss function weights are reported.
- No ablation study exists for SymR's main components.
- NeuroPath training data is generated by the Gemini commercial API but quality and error rate of these reasoning paths is never evaluated.
- The Related Work entirely omits Reinforcement Learning on Knowledge Graphs — the most directly comparable research direction.
8. Đề nghị: Được bảo vệ ☑ Bổ sung thêm để bảo vệ □ Không được bảo vệ □

9. 3 câu hỏi SV phải trả lời trước Hội đồng:
- If the Abstract Meaning Representation subgoal in SymR is removed and only a random subgoal is used, how much do the Exact Match results change?
- How can we replicate the experiment if, in the future, gemini-2.0-flash and gemini-2.5-flash are no longer available to generate training data and reasoning paths?

10. Đánh giá chung (bằng chữ: giỏi, khá, TB): Giỏi Điểm : 9.1 /10

NGƯỜI VIẾT PHIẾU CHẤM
(ký và ghi rõ họ tên)



Phan Trần Minh Khuê

Instructor's Signature

_____ Date: _____

Assoc. Prof. Quan Thanh Tho, Ph.D. (Instructor)
Associate Professor
Faculty of Computer Science and Engineering

Declaration of Authenticity

The author, Thai Quang Phat, declare that this Capstone Project was composed and implemented entirely by the author under the guidance and supervision of Assoc. Prof. Quan Thanh Tho and Mr. Bui Cong Tuan at the Faculty of Computer Science and Engineering, Vietnam National University - Ho Chi Minh City University of Technology.

In the process of researching and implementing this Capstone Project, the author has referenced multiple previous studies from other people and themselves. All of the referenced work have been fully and clearly stated in the References part. This Capstone Project has been published as conference papers which are presented at Appendix C.

Ho Chi Minh City, June 2026
Thai Quang Phat

Acknowledgement

First and foremost, the author would like to express his sincere gratitude to Assoc. Prof. Quan Thanh Tho and Mr. Bui Cong Tuan for dedicating their valuable time to guide, orient, and encourage him throughout the outline and implementation phases of this Capstone Project.

He is also thankful to the lecturers at the Faculty of Computer Science and Engineering, Vietnam National University - Ho Chi Minh City University of Technology, for providing him with essential foundational knowledge. This background has served as a strong springboard, enabling him to complete this thesis and contribute to the Vietnamese Computer Science community.

In addition, the author would like to acknowledge his own efforts during the thesis-writing process. Although taking the first step was challenging, he remained determined. By creating a practical plan and committing wholeheartedly to it, he was able to stay on track and complete the project on time.

Last but not least, he extends heartfelt thanks to his family for their unwavering support throughout his university journey. His parents' diligence has been his greatest source of motivation, encouraging him to work hard and persevere.

Abstract

Recent advances in large language models (LLMs) have led to remarkable performance across a wide range of natural language processing tasks, including question answering, text generation, and reasoning-intensive benchmarks. Despite these successes, reasoning-oriented tasks - particularly multi-hop question answering (QA) - remain challenging. LLM-based approaches often suffer from hallucinated reasoning steps, limited interpretability, and a lack of explicit verification mechanisms. Moreover, their reliance on large-scale models and extensive inference-time computation results in high computational costs, making them inefficient and difficult to deploy in resource-constrained or reliability-critical settings. These limitations highlight the need for alternative or complementary reasoning frameworks that can provide structured, verifiable, and computationally efficient multi-hop inference.

Multi-hop question answering requires an agent to connect entities and relations across multiple reasoning steps, rather than relying solely on surface-level semantic interpretation. Although large language models (LLMs) demonstrate strong abilities in natural language understanding, their reasoning traces often remain ungrounded and difficult to verify. In contrast, traditional graph-based methods enforce explicit structure but lack the flexibility to generalize across diverse question types and domains.

In this thesis, I propose a Neural Policy (**NeuroPath**), and a Symbolic Hierarchical Reinforcement Learning (**SymR**) designed to address these limitations. The **SymR** approach decomposes complex questions into layered decision-making processes that better reflect the hierarchical nature of multi-hop reasoning. Both methods incorporate principled mathematical formulations, and the thesis provides rigorous background knowledge on neural networks and reinforcement learning, including theoretical analyses.

To evaluate these models, I conduct extensive experiments and compare them against traditional and current baselines in multi-hop QA. The results on benchmark datasets demonstrate that both proposed approaches deliver improvements in accuracy, reasoning coherence, and interpretability. These findings highlight the promise of combining neural modeling, structured decision-making, and SymR to advance explainable and effective multi-hop inference.

Contents

1 Introduction **1**

1.1 Motivation 1

1.2 Goals 2

1.3 Scope 2

1.4 Thesis Structure 3

2 Related Works **5**

2.1 Reinforcement Learning With Knowledge Graph 5

2.2 Reasoning Chain Generation 7

2.2.1 Instruction Fine-Tuning and Scaling of Reasoning 7

2.2.2 Self-Improving Rationale Generation 8

2.2.3 Interleaved Retrieval and Reasoning 8

2.2.4 Generative Path Modeling 9

2.2.5 Integrating Structured Knowledge and Symbolic Logic 9

2.3 Question Decomposition 10

2.3.1 Prompt-Based Decomposition 10

2.3.2 Parallel Decomposition and Error Mitigation 11

2.3.3 Learning Decomposition from Unlabeled Data 11

2.3.4 Structured and Graph-Based Decomposition 12

3 Deep Learning Theoretical Background **14**

3.1 Neural Networks: Recurrent Architectures 14

3.1.1 Fundamental Concepts 14

3.1.2 Feedforward Neural Networks 14

3.1.3 Language Models and Sequential Data 14

3.1.4 Recurrent Architectures 15

3.1.5 Gated Recurrent Units 16

3.1.6 Long Short-Term Memory Networks 17

3.2 Transformers 18

3.2.1 Attention 18

3.2.2 Encoder-Decoder Architecture . . . 21
3.3 Graph Neural Networks . . . 22
3.3.1 Graph . . . 23
3.3.2 An Encoder-Decoder Perspective . . . 23
3.3.3 Neural Message Passing . . . 23
3.3.4 Generalized Neighborhood Aggregation . . . 24
3.4 Knowledge Graphs . . . 25
3.4.1 Definition . . . 25
3.4.2 Example . . . 26
3.4.3 Applications . . . 26
3.5 Abstract Meaning Representation . . . 27
3.5.1 Definition . . . 27
3.5.2 Example . . . 28
3.5.3 AMR Parsing . . . 28

4 Reinforcement Learning Theoretical Background 29
4.1 Introduction to Reinforcement Learning . . . 29
4.1.1 Markov Decision Process . . . 29
4.1.2 Policy, Trajectories, and Return . . . 30
4.2 State Values, Optimality, and the Bellman Equations . . . 30
4.2.1 State and Action Values . . . 30
4.2.2 Bellman Equation . . . 30
4.2.3 Optimal State Values and the Bellman Optimality Equation . . . 31
4.2.4 Value Iteration, Policy Iteration, and Truncated Policy Iteration . . . 31
4.3 Value Estimation Methods . . . 32
4.3.1 Monte Carlo Methods . . . 32
4.3.2 Temporal-Difference Methods . . . 32
4.3.3 Comparison of MC and TD Methods . . . 33
4.4 Hierarchical Reinforcement Learning . . . 34
4.4.1 Temporal Abstraction . . . 34
4.4.2 Hierarchical Policies . . . 34
4.4.3 Subgoals and Options . . . 35
4.4.4 Hierarchical Markov Decision Process . . . 35
4.4.5 Advantages of Hierarchical Reinforcement Learning . . . 35

5 Proposed Solutions 36
5.1 Problem Definition . . . 36
5.2 NeuroPath: Neural Policy for Knowledge Graph Reasoning . . . 36

5.2.1 Framework Overview . . . 36
5.2.2 Relational Graph Attention Network . . . 37
5.2.3 Path Retrieval on the Knowledge Graph . . . 38
5.2.4 Stopping Module . . . 39
5.2.5 Training Objectives . . . 40
5.3 SymR: Symbolic Reinforcement for Multi-Hop Knowledge Graph Reasoning . 41
5.3.1 Framework Overview . . . 41
5.3.2 AMR Graph Encoding . . . 43
5.3.3 Hierarchical Policy Architecture . . . 44
5.3.4 Hierarchical Reward Structure . . . 46
5.3.5 Training and Optimization . . . 47

6 Experiment Settings & Results **49**
6.1 Datasets & Preprocessing . . . 49
6.2 Baselines . . . 50
6.3 Training Configurations . . . 51
6.4 Hyperparameter Implementation Settings . . . 51
6.5 Question Answering Results . . . 51
6.6 Reasoning Paths Analysis . . . 52
6.6.1 Model Output Examples . . . 53
6.6.2 Special Cases of Reasoning Path Deviations . . . 53
6.6.3 Qualitative Comparison Between NeuroPath and SymR . . . 54
6.6.4 Success Rate and Convergence Analysis . . . 55
6.6.5 Chain Generation Results . . . 56
6.6.6 Analysis and Discussion . . . 57

7 Conclusion **58**
7.1 Summary . . . 58
7.2 Possible Improvements . . . 59
7.3 Future Plans . . . 59

References **60**

A Workload Description **65**

B Timeline Of Workload **66**

C Publications **67**

List of Tables

4.1 Comparison of TD and MC methods. 33

6.1 Training and testing splits of the datasets used in our experiments. 50
6.2 HotpotQA results. 51
6.3 2WikiMultihopQA results. 52
6.4 MuSiQue results. 52
6.5 Representative examples of model-generated reasoning paths and answers for entity-based, short commonsense, and long commonsense multi-hop questions. 53
6.6 Comparison between target and predicted reasoning paths, illustrating exact matches, over-extended paths, and compressed paths produced by the model. . 54
6.7 Comparison between SymR and NeuroPath across three multi-hop questions. SymR consistently produces correct answers; NeuroPath fails due to premature stopping or incorrect relation selection. 55
6.8 Navigation accuracy (%) across three datasets. 57

List of Figures

2.1 Architecture of M-Walk. (a) The state vector is fed into π_θ and Q_θ. (b) The GRU-RNN encoder maps the state to its representation h_t, where $h_{A,t-1}$ and $h_{a_{t-1},t-1}$ come from the previous step $t-1$. 5

2.2 Training framework of Lin *et al.* [20]. At step t, the agent samples an outgoing link using the perturbed REINFORCE policy $\tilde{\pi}_\theta(a_t|s_t)$, obtained by applying a binary mask $\mathbf{m}$ to $\pi_\theta(a_t|s_t)$. The agent receives reward 1 if it stops at an observed answer to $(e_s, r_q, ?)$; otherwise, it receives a shaped reward $f(e_s, r_q, e_T)$ from a pre-trained RS network with fixed parameters. 6

2.3 RL-MHR, a revised RL framework for multi-hop KG reasoning, introduces two key components: the stop signal and the worth-trying signal. 6

2.4 The relation selected by the high-level policy is passed to the low-level policy, whose predicted tail entity is combined with the relation and encoded by an LSTM to capture historical information. 6

2.5 DRKG pipeline: an LLM generates reasoning plans from natural language questions, guiding hop-constrained KG traversal to produce faithful and interpretable multi-hop answers. 7

2.6 Flan-T5 fine-tunes language models on approximately 1.8K instruction-based tasks and evaluates their generalization on unseen tasks under zero-shot, few-shot, and chain-of-thought settings. 7

2.7 Overview of STaR and a STaR-generated rationale on CommonsenseQA. Questions and ground-truth answers are provided in the dataset, while rationales are generated by STaR. 8

2.8 IRCoT alternates between chain-of-thought generation and retrieval, where CoT guides retrieval and retrieved documents support subsequent reasoning steps until termination. 8

2.9 Overview of the PEI framework for multi-hop QA, consisting of the type prompter, knowledge prompter, and unified prompter. 9

2.10 Overview of PATHFID, where question-passage pairs are encoded in parallel and concatenated into a unified representation for reasoning path and answer generation. 9

2.11 Overview of KG-o1 framework for enhancing multi-hop question answering in LLMs. 10

2.12 Least-to-most prompting solves math word problems by first decomposing them into subproblems and then sequentially solving each subproblem. 11

2.13 Chain-of-Thought prompting includes reasoning steps for each labeled example, while Decomposed Prompting specifies procedures to break tasks into subtasks handled by different sub-task modules such as standard prompting, further decomposition, or symbolic functions. 11
2.14 Pipeline of GenDec, where multi-hop questions are first decomposed into subquestions, followed by sub-question–enhanced paragraph retrieval and final answer extraction using the retrieved context. 12
2.15 An overview of the proposed DECOMPT5 pipeline. The final decomposition is an actual output from the pipeline. 12
2.16 Overview of D&R Distillation, where a teacher model generates subquestions and intermediate answers to form training data, which is used to train a Decomposer and a Responser that interactively solve multi-hop questions using retrieval. 12
2.17 Overview of the QDAMR framework, which transforms a multi-hop question into an AMR graph, decomposes it into sub-questions, answers intermediate queries using a QA system, and substitutes results to obtain the final answer. . . 13
2.18 Overview of BeamAggR, which decomposes complex questions into trees, performs multi-source reasoning with probabilistic normalization, and aggregates beam-based reasoning trajectories to select the most promising answer path. . . 13

3.1 A Recurrent Neural Network (RNN). Three time-steps are shown. 15
3.2 A deep bi-directional RNN with three RNN layers. 16
3.3 The detailed internals of a GRU. 17
3.4 The detailed internals of an LSTM. 18
3.5 Visualisation of the dot-product self-attention mechanism, with $d_k = d_v = 4$. . . 19
3.6 Multi-head self-attention with $h = 2$ heads and $d_k = d_v = 2$. 20
3.7 The transformer layer. Multi-head attention (with residual connection and LayerNorm) is followed by a position-wise feedforward network (with residual connection and LayerNorm). 20
3.8 Cross-attention. Queries are derived from the decoder embeddings; keys and values from the encoder embeddings. 22
3.9 Encoder-decoder architecture. The encoder processes the source sequence; the decoder generates the target sequence using masked self-attention and cross-attention over the encoder output. 22
3.10 A graph and its corresponding adjacency matrix. 23
3.11 Visualisation of basic message passing for node 1. 24
3.12 Graph Neural Network. A simple example of a knowledge graph is a small graph describing relationships among people, organizations, and locations. . . . 26

4.1 The standard RL framework. At time step t, the agent observes state S_t, takes action A_t, and receives reward R_t. 29

4.2 Illustration of the hierarchical reinforcement learning framework. A high-level *Manager* observes the environment state S_t and issues sub-goals to a low-level *Worker*. The Worker executes primitive actions A_t to interact with the environment. In response, the environment transitions to the next state S_{t+1} and returns rewards R_t and R_{t+1}, which are used to guide learning at different levels of the hierarchy. 34

5.1 The retrieval mechanism of our model. The input question is encoded by a pretrained encoder to produce the initial representation q^0. At each hop t, the embedded knowledge graph is processed together with the current query representation q^t by the policy block, which outputs the next candidate node j^* and updates the query state to q^{t+1}. This iterative procedure repeats until no unvisited neighbors remain or the stopping module signals termination. 37

5.2 **Left.** The attention mechanism $a^\top [W_k h_i \| W_q r_{ij} \| W_v h_j]$ employed by our model, parameterized by $a \in \mathbb{R}^{3F}$ and followed by a LeakyReLU activation. **Right.** Multi-head attention with $K = 3$ heads applied by node 1 to its neighborhood. Different arrow styles denote independent attention computations; features from all heads are concatenated to obtain h'_1. 38

5.3 An overview of the retrieval model. At hop t, the model is at node e_i with query representation q^t. A probability distribution $\pi_\theta(a_t = j)$ is computed over unvisited neighbors; the selected node's embedding is integrated with q^t via an LSTM update to produce q^{t+1} for the next hop. 39

5.4 High-level architecture of SymR for multi-hop question answering on knowledge graphs. The question is parsed into an AMR graph and encoded by an AMR GCN to provide symbolic semantic guidance, while the knowledge graph is encoded using a relational graph attention network. A hierarchical reinforcement learning framework integrates both representations: the manager selects AMR-grounded symbolic subgoals, and the worker navigates the knowledge graph to execute them across multiple hops. 42

5.5 Architecture of SymR. The manager selects AMR-based semantic subgoals, while the worker navigates the knowledge graph using RGAT-based entity representations. A hierarchical reward combines extrinsic, intrinsic, and subgoal-oriented signals. 43

6.1 Success rate over training epochs (top) and convergence rate (down) across three datasets. 55

Chapter 1

Introduction

In chapter 1, the overview, objectives, and goals of the research project are illustrated. The outline of the report is also presented.

1.1 Motivation

Multi-hop question answering (QA) requires a system to derive an answer by traversing multiple relational steps across a knowledge source. Unlike single-hop QA, where a direct fact may suffice, multi-hop QA demands reasoning over a chain of interconnected entities and relations. This process is inherently challenging due to the combinatorial search space, long reasoning dependencies, and the necessity for interpretable and verifiable inference.

Recent advancements in large language models (LLMs) have resulted in remarkable improvements in natural language understanding. However, despite their capabilities, LLMs often exhibit limitations in multi-hop scenarios: their reasoning traces may be ungrounded, prone to hallucination, or inconsistent across runs. Chain-of-Thought prompting and question decomposition methods partially address these issues, but largely operate through ungrounded text generation and static prompting strategies, limiting their reliability when applied to structured reasoning over knowledge graphs.

Knowledge graphs (KGs) offer an appealing structured environment for multi-hop reasoning, yet designing reasoning algorithms that are both efficient and interpretable remains a non-trivial challenge. Reinforcement learning (RL) provides a principled framework for sequential decision-making over graph structures, but training effective policies on large KGs is difficult without proper hierarchical decomposition. Existing hierarchical RL methods typically condition decisions on dense latent question embeddings, providing weak alignment with the compositional structure of natural language questions and often resulting in spurious reasoning paths.

These challenges collectively motivate the need to investigate hybrid approaches that combine neural representation learning, structured semantic guidance, and hierarchical reinforcement learning. Specifically, incorporating symbolic semantic representations - such as those provided by Abstract Meaning Representation (AMR) graphs - can bridge the gap between natural language question structure and knowledge graph traversal, enabling more interpretable and grounded reasoning chains. This thesis addresses these challenges by proposing two novel reasoning frameworks and assessing their effectiveness against established baselines.

1.2 Goals

The primary objective of this thesis is to develop, analyze, and evaluate models that perform interpretable and effective multi-hop reasoning over knowledge graphs. To achieve this, the thesis is structured around the following goals:

- **Investigate the theoretical foundations** of deep learning and reinforcement learning relevant to multi-hop reasoning, including neural architectures, graph neural networks, value functions, and hierarchical policies.
- **Develop a Symbolic Hierarchical Reinforcement Learning approach** (SymR) that integrates AMR-derived semantic subgoals into a two-level decision process, where a high-level manager selects linguistically grounded subgoals and a low-level worker navigates the knowledge graph to satisfy them, enabling more structured and consistent reasoning behaviors.
- **Construct a complete experimental pipeline** for training, validating, and testing the proposed models across benchmark datasets, including implementation of baselines and evaluation metrics.
- **Compare the proposed approach with previous and competitive baselines**, including LLM-based, retrieval-augmented generation, and LLM+KG hybrid methods, highlighting strengths, weaknesses, and observed reasoning patterns derived from experiment results.
- **Provide analytical insights** into reasoning consistency, interpretability, and the limitations of neural and RL-based reasoning on KGs.

Through these goals, the thesis aims to contribute both practical algorithms and theoretical understanding to the field of multi-hop reasoning.

1.3 Scope

The scope of this thesis is centered around developing and analyzing multi-hop reasoning methods over knowledge graphs using neural, symbolic, and reinforcement learning techniques. The main boundaries of the research include:

- **Focus on KG-based multi-hop QA**: The primary setting involves reasoning over relational graph structures constructed from context paragraphs. While LLMs are invoked at the final stage to verbalize predicted reasoning paths into natural language answers, they are not used as reasoning components; all multi-hop inference is performed explicitly over the knowledge graph.
- **Investigation of main approach**: Symbolic Hierarchical Reinforcement Learning approach (SymR). SymR integrates AMR-based semantic subgoal guidance with hierarchical RL, distinguishing it from purely neural or latent-embedding-based HRL methods.
- **Theoretical background coverage**: The thesis provides foundational knowledge in deep learning (neural networks, graph neural networks, relational graph attention networks) and reinforcement learning (value functions, policy optimization, proximal policy optimization, generalized advantage estimation), limited to concepts necessary for supporting the proposed architectures.

- **Experimental evaluation**: Experiments are conducted on three standard multi-hop QA benchmarks - HotpotQA, 2WikiMultihopQA, and MuSiQue - following standard data preprocessing, baseline comparisons, training configurations, and hyperparameter settings described in the thesis.
- **Exclusion of dynamic or incomplete KG settings**: Although the thesis recognizes the practical importance of reasoning over dynamic or incomplete knowledge graphs, the proposed models operate on static, pre-constructed graphs and extensions to more general graph settings are left as future work.

This scope ensures a focused exploration of KG reasoning while leaving room for future extensions involving end-to-end symbolic-neural integration, multilingual applications, and open-domain settings.

1.4 Thesis Structure

This thesis is organized into eight chapters, each addressing a specific component of the research:

- **Chapter 1: Introduction.** Presents the motivation, goals, scope, and structure of the thesis.
- **Chapter 2: Related Works.** Provides a comprehensive review of existing literature on multi-hop QA using large language models, graph-based reasoning techniques, graph prompting, neural-symbolic reasoning, graph neural networks, and reinforcement learning on graphs.
- **Chapter 3: Deep Learning Theoretical Background.** Covers the fundamentals of neural networks, including activation functions, architectures, optimization techniques, transformers, and graph neural networks. This chapter also introduces knowledge graphs with definitions, detailed examples, and real-world applications.
- **Chapter 4: Reinforcement Learning Theoretical Background.** Introduces core RL concepts: states, actions, transitions, rewards, policies, state values, Bellman equations, optimality principles, value iteration, policy iteration, and value estimation methods such as Monte Carlo and temporal-difference learning.
- **Chapter 5: Proposed Solutions.** This chapter presents the proposed reasoning framework and introduces two complementary approaches for multi-hop question answering on knowledge graphs: *Neural Policy* and *Symbolic Hierarchical Reinforcement Learning*, an enhanced framework that employs a two-level decision structure, consisting of a high-level subgoal selection policy and a low-level goal-conditioned navigation policy, together with intrinsic and extrinsic reward designs, hindsight experience relabeling, and advanced training strategies, with symbolic representation injected.
- **Chapter 6: Experiment.** Presents the datasets, preprocessing pipeline, baseline methods, training configurations, and hyperparameter settings used for evaluating the models.
- **Chapter 7: Results.** Reports quantitative results on multi-hop QA tasks and provides qualitative analyses of reasoning paths.

- **Chapter 8: Conclusion.** Summarizes key findings, discusses potential improvements, and outlines future research directions including symbolic reasoning integration and application to the Vietnamese domain.

Together, these chapters build a coherent narrative from foundational theory to model design, experimental evaluation, and future prospects for advancing multi-hop reasoning over knowledge graphs.

Chapter 2

Related Works

This chapter presents an overview of state-of-the-art methods for multi-hop question answering, examining multiple methodological perspectives and their respective strengths and limitations.

2.1 Reinforcement Learning With Knowledge Graph

Reinforcement learning-based frameworks such as MINERVA [6] and M-Walk [27] (Figure 2.1) formulate multi-hop reasoning as a policy-learning problem, where an agent sequentially selects graph edges to reach the correct answer. M-Walk enhances this paradigm by integrating neural policies with Monte Carlo Tree Search (MCTS) to address sparse rewards and exploration challenges, encoding traversal history via an RNN and jointly modeling action probabilities and Q-values.

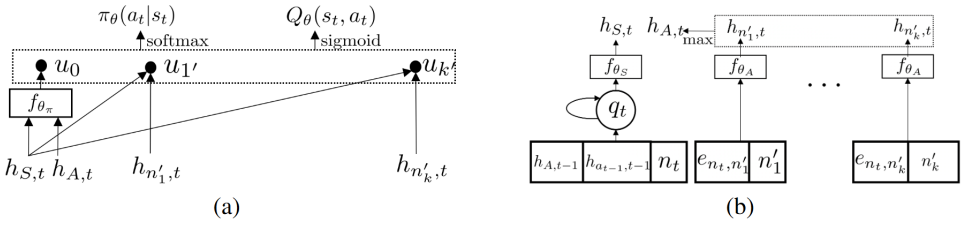


Figure 2.1: Architecture of M-Walk. (a) The state vector is fed into π_θ and Q_θ. (b) The GRU-RNN encoder maps the state to its representation h_t, where $h_{A,t-1}$ and $h_{a_{t-1},t-1}$ come from the previous step $t-1$.

Lin *et al.* [20] extend this direction with reward-based exploration strategies that do not rely on gold reasoning paths, enabling more scalable reasoning where annotated supervision is limited (Figure 2.2). However, reliance on static, predefined KGs limits adaptability to evolving domains and restricts coverage under open-world knowledge.

Addressing the stopping problem in multi-hop reasoning, Liao *et al.* [19] proposed RL-MHR (Figure 2.3), which augments the RL reward with an explicit stopping signal, improving both answer performance and computational efficiency.

To handle long-context and multi-relational questions, [42] propose a hierarchical RL framework that decomposes reasoning into high-level relation detection and low-level entity selection, reducing the search space while improving reasoning focus (Figure 2.4).

More recently, DRKG [3] leverages LLMs as reasoning agents trained with RL (Figure 2.5). It

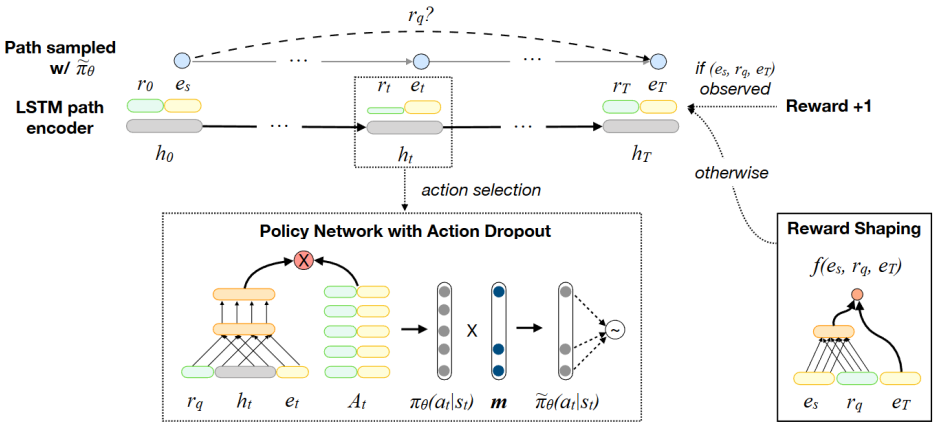


Figure 2.2: Training framework of Lin *et al.* [20]. At step t, the agent samples an outgoing link using the perturbed REINFORCE policy $\tilde{\pi}_\theta(a_t|s_t)$, obtained by applying a binary mask $\mathbf{m}$ to $\pi_\theta(a_t|s_t)$. The agent receives reward 1 if it stops at an observed answer to $(e_s, r_q, ?)$; otherwise, it receives a shaped reward $f(e_s, r_q, e_T)$ from a pre-trained RS network with fixed parameters.

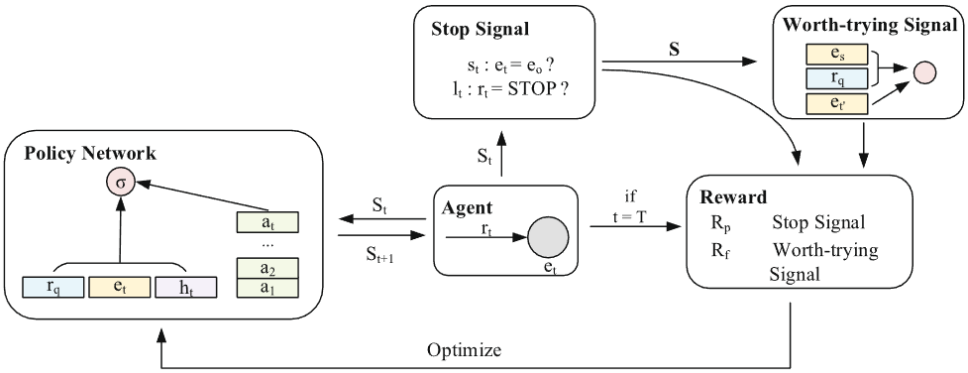


Figure 2.3: RL-MHR, a revised RL framework for multi-hop KG reasoning, introduces two key components: the stop signal and the worth-trying signal.

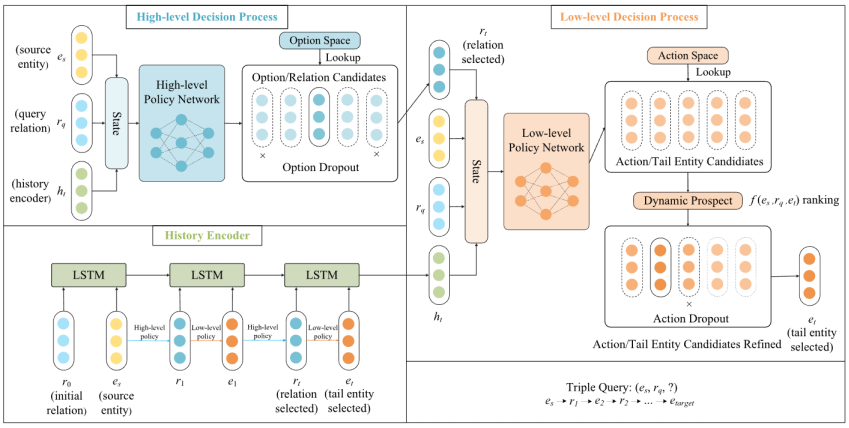


Figure 2.4: The relation selected by the high-level policy is passed to the low-level policy, whose predicted tail entity is combined with the relation and encoded by an LSTM to capture historical information.

decomposes questions into explicit multi-hop plans and performs hop-constrained KG traversal, combining supervised fine-tuning with RL rewards to reduce hallucination and improve reasoning transparency.

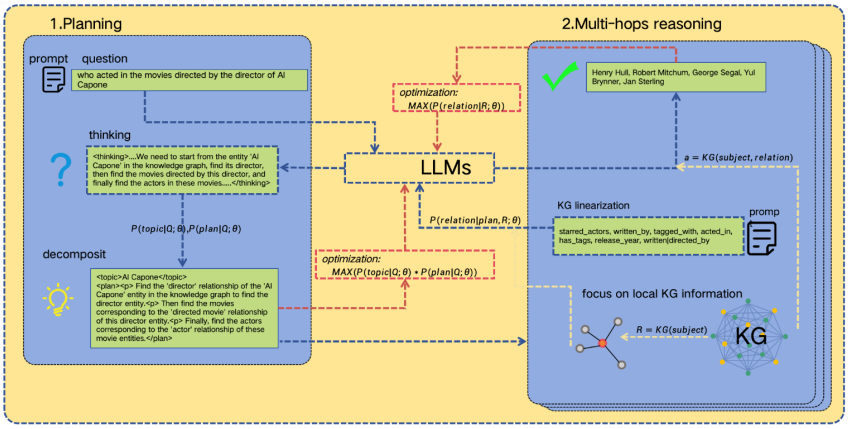


Figure 2.5: DRKG pipeline: an LLM generates reasoning plans from natural language questions, guiding hop-constrained KG traversal to produce faithful and interpretable multi-hop answers.

2.2 Reasoning Chain Generation

The emergence of LLMs and Chain-of-Thought (CoT) prompting has substantially advanced multi-step reasoning. Rather than mapping questions directly to answers, CoT-based methods produce intermediate reasoning steps before arriving at a final response, making this paradigm foundational to modern multi-hop QA.

2.2.1 Instruction Fine-Tuning and Scaling of Reasoning

Early CoT investigations established that training data quality and diversity are decisive for generalizing to complex reasoning tasks. Instruction fine-tuning research [5] shows that scaling task diversity and incorporating CoT-formatted examples are critical to prevent reasoning degradation on novel problems. Notably, smaller architectures such as Flan-T5, when fine-tuned on rich and diverse corpora, can match or surpass much larger models, suggesting that training structure matters as much as parameter count.

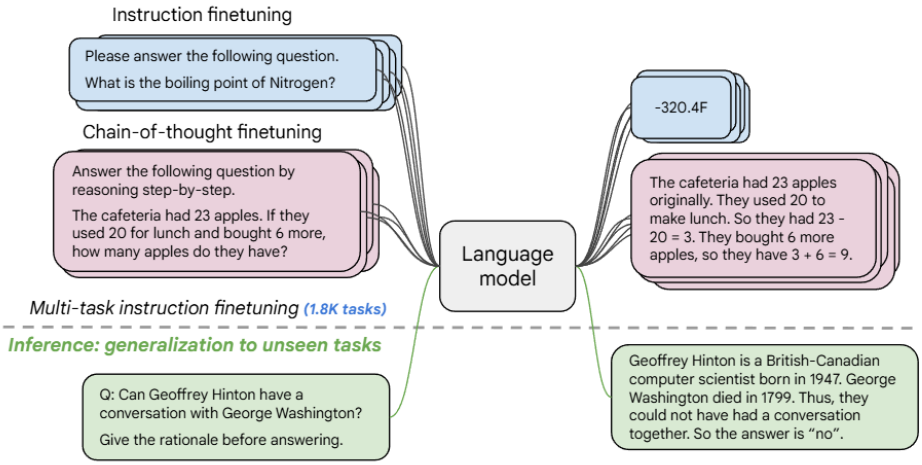


Figure 2.6: Flan-T5 fine-tunes language models on approximately 1.8K instruction-based tasks and evaluates their generalization on unseen tasks under zero-shot, few-shot, and chain-of-thought settings.

2.2.2 Self-Improving Rationale Generation

Supervised CoT training is limited by its reliance on costly manually annotated reasoning chains. STaR [38] addresses this with a bootstrapping mechanism where a model generates rationales for correctly answered questions and incorporates them into subsequent training rounds (Figure 2.7). Over multiple iterations, the model produces increasingly coherent reasoning chains without external annotation, though it still requires correct final answers as a supervisory signal.

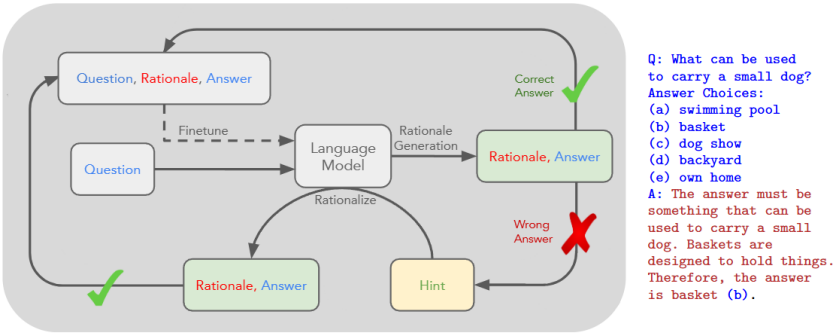


Figure 2.7: Overview of STaR and a STaR-generated rationale on CommonsenseQA. Questions and ground-truth answers are provided in the dataset, while rationales are generated by STaR.

2.2.3 Interleaved Retrieval and Reasoning

Purely generative CoT approaches are constrained by static parametric knowledge that may be incomplete or outdated. IRCoT [29] interleaves CoT generation with dynamic retrieval, using each reasoning step as a query to retrieve supporting documents, which in turn inform the next step (Figure 2.8). This bidirectional dependency significantly reduces hallucination by grounding each step in retrieved evidence.

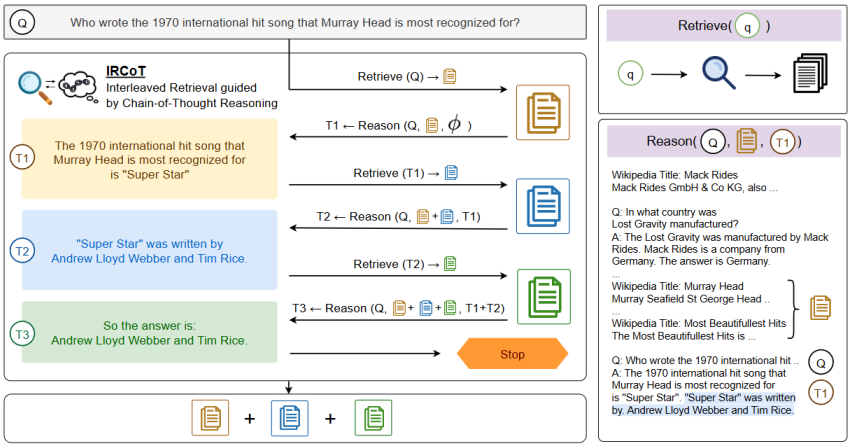


Figure 2.8: IRCoT alternates between chain-of-thought generation and retrieval, where CoT guides retrieval and retrieved documents support subsequent reasoning steps until termination.

PEI [13] complements this by bridging explicit retrieved knowledge and implicit parametric knowledge via soft prompts (Figure 2.9), demonstrating notable robustness to noisy or irrelevant retrieved passages.

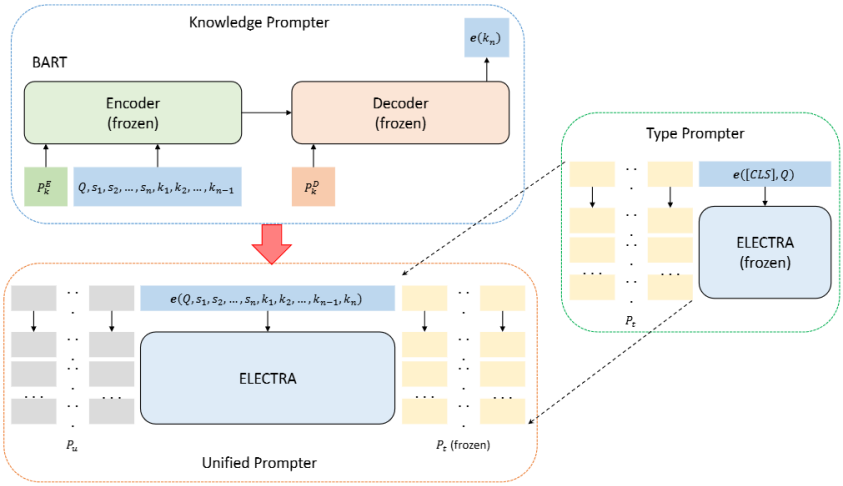


Figure 2.9: Overview of the PEI framework for multi-hop QA, consisting of the type prompter, knowledge prompter, and unified prompter.

2.2.4 Generative Path Modeling

PATHFID [37] frames multi-hop reasoning as a single sequence-to-sequence prediction, representing the full reasoning path from retrieved passage through intermediate evidence to final answer as a linear sequence (Figure 2.10). This end-to-end formulation enhances interpretability and allows intermediate conclusions to be inspected, achieving competitive empirical performance despite simplifying the graph-structured nature of multi-hop reasoning.

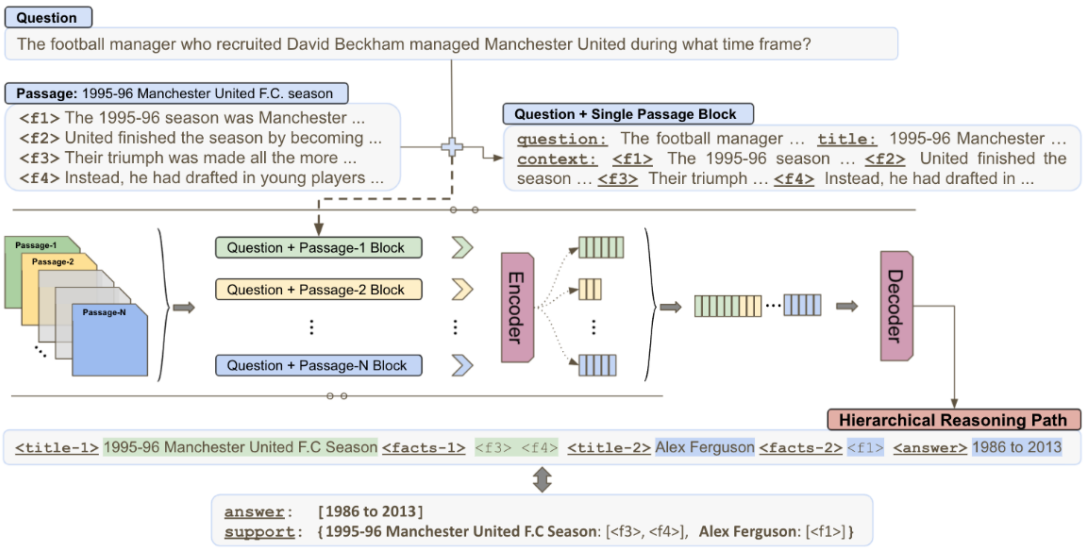


Figure 2.10: Overview of PATHFID, where question-passage pairs are encoded in parallel and concatenated into a unified representation for reasoning path and answer generation.

2.2.5 Integrating Structured Knowledge and Symbolic Logic

KG-o1 [33] internalizes KG reasoning paths directly into LLM inference via a "Slow-Thinking" mechanism combined with Adaptive Direct Preference Optimization (DPO) (Figure 2.11). Rather than querying the KG externally at inference time, it encodes relational reasoning patterns into model parameters through preference-based fine-tuning, representing a significant step toward tighter neural-symbolic integration.

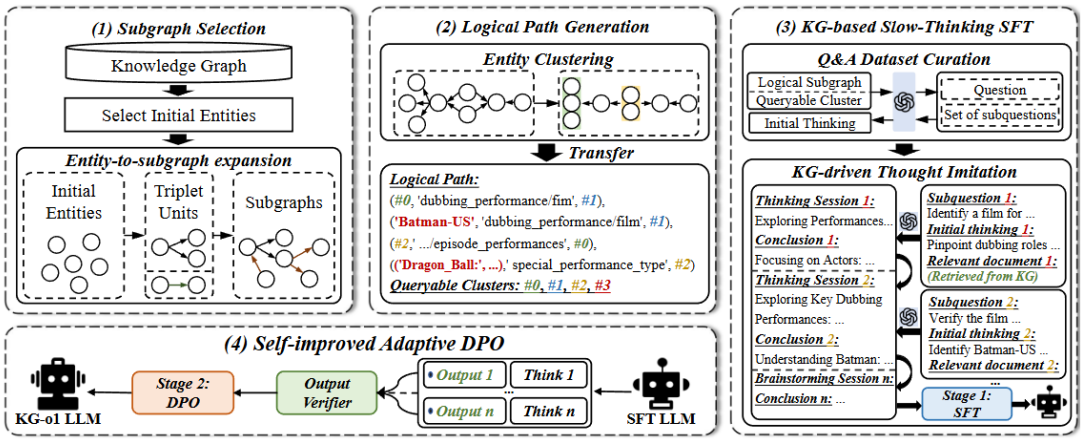


Figure 2.11: Overview of KG-o1 framework for enhancing multi-hop question answering in LLMs.

The gap between neural generation and formal reasoning is further highlighted by FOLIO [10], which evaluates LLMs on First-Order Logic inference tasks, exposing persistent difficulties in respecting strict entailment relationships. This tension between neural fluency and symbolic rigor is a central motivation for SymR, which seeks to harness both paradigms within a hierarchical reinforcement learning framework.

2.3 Question Decomposition

Question Decomposition (QD) offers an alternative to end-to-end chain-of-thought generation for multi-hop question answering. Rather than producing a monolithic reasoning chain, QD adopts a divide-and-conquer strategy: a complex query is broken into simpler, independently answerable sub-questions whose answers are aggregated to address the original query. This approach suits questions with inherently compositional structure, where the answer depends on several distinct factual sub-queries resolvable in sequence or in parallel.

2.3.1 Prompt-Based Decomposition

The most direct approach leverages the in-context learning capabilities of LLMs through carefully designed prompts. Least-to-Most Prompting [41] instructs an LLM to first identify constituent sub-problems—proceeding from simpler to more complex components—substantially improving performance on tasks requiring length generalization. This exposes a key limitation of standard CoT prompting: when a novel query requires a significantly longer reasoning chain than those demonstrated in-context, performance degrades markedly. Decomposition into shorter sub-tasks mitigates this failure.

Decomposed Prompting [15] routes each sub-problem to a specialized handler—a prompted LLM or symbolic module—rather than requiring a single model to handle all reasoning types. This modular design allows each sub-task to be addressed by the most appropriate solver, with the final answer assembled from individual handler outputs.

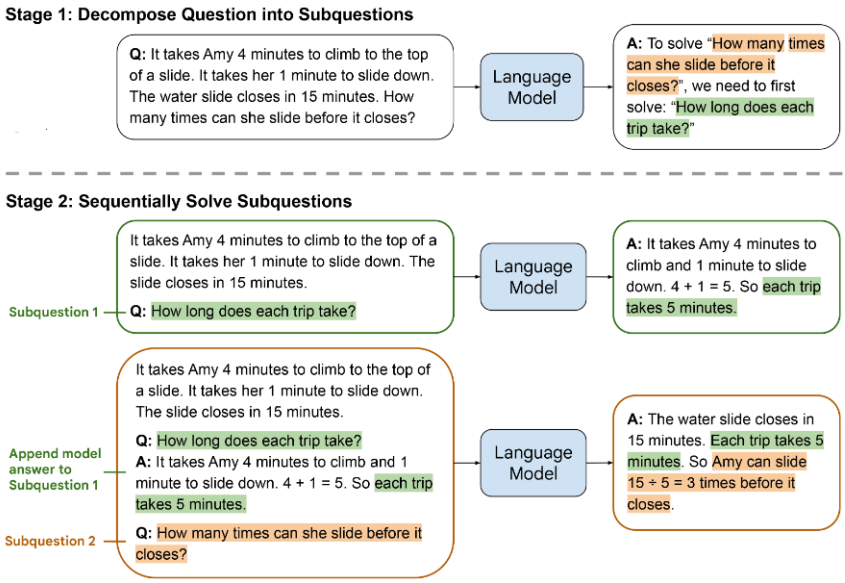


Figure 2.12: Least-to-most prompting solves math word problems by first decomposing them into subproblems and then sequentially solving each subproblem.

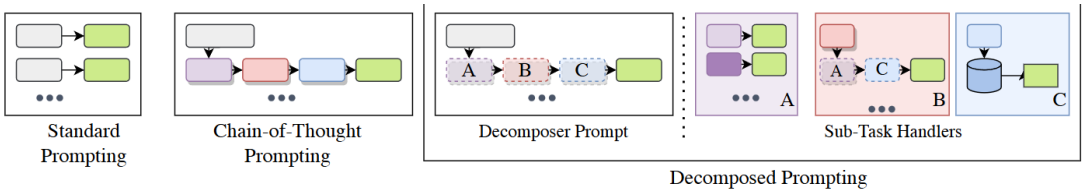


Figure 2.13: Chain-of-Thought prompting includes reasoning steps for each labeled example, while Decomposed Prompting specifies procedures to break tasks into subtasks handled by different sub-task modules such as standard prompting, further decomposition, or symbolic functions.

2.3.2 Parallel Decomposition and Error Mitigation

A well-documented failure mode of sequential decomposition is error propagation: an incorrectly answered early sub-question cascades through subsequent steps, compounding into an incorrect final answer. GenDec [34] addresses this by generating independent, parallel sub-questions rather than sequentially dependent ones. Because sub-questions do not condition on prior answers, errors in one do not corrupt others. The final answer is obtained by aggregating independently generated sub-question answers via a learned mechanism or majority voting, trading the expressive power of sequential conditioning for robustness to individual sub-task errors.

2.3.3 Learning Decomposition from Unlabeled Data

The requirement for annotated decomposition examples presents a practical bottleneck for supervised QD systems. DECOMPT5 [40] addresses this via distant supervision, leveraging the structural parallelism of news articles to train a decomposition model without explicit sub-question annotations. By treating co-reported events as implicit evidence of compositional query structure, it induces decomposition patterns from naturally occurring text.

D&R Distillation [18] takes a complementary approach by separating decomposition and answer generation into a *Decomposer* and a *Responser*, each optimized independently. Knowl-

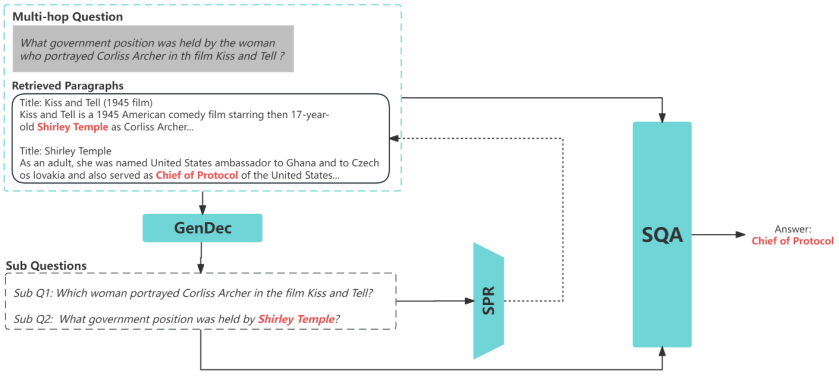


Figure 2.14: Pipeline of GenDec, where multi-hop questions are first decomposed into sub-questions, followed by sub-question–enhanced paragraph retrieval and final answer extraction using the retrieved context.

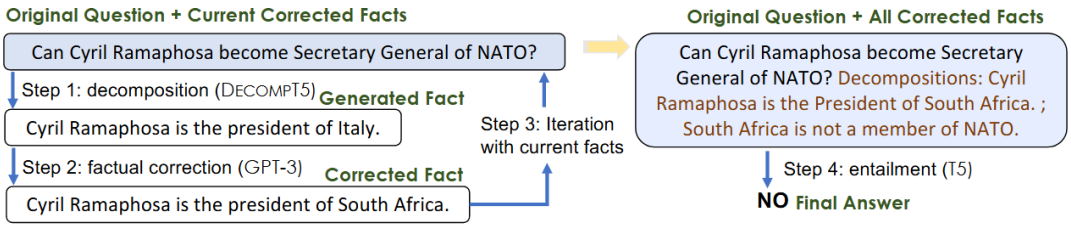


Figure 2.15: An overview of the proposed DECOMPT5 pipeline. The final decomposition is an actual output from the pipeline.

edge distilled from larger teacher models into each component separately extends complex QA capabilities to Small Language Models (SLMs) unsuited for end-to-end multi-hop reasoning.

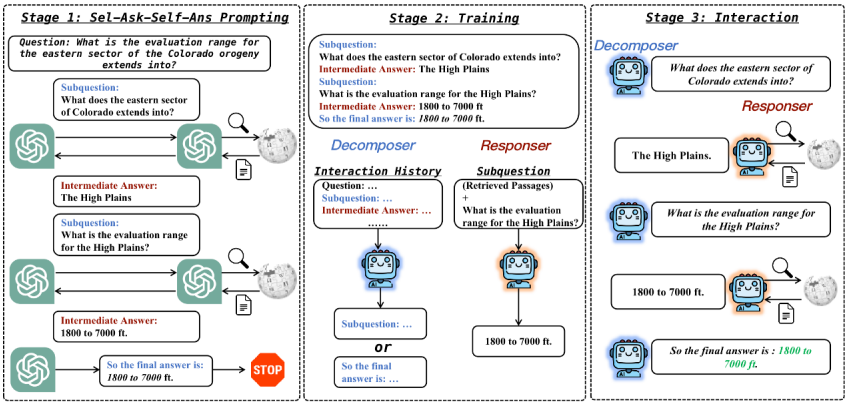


Figure 2.16: Overview of D&R Distillation, where a teacher model generates subquestions and intermediate answers to form training data, which is used to train a Decomposer and a Responser that interactively solve multi-hop questions using retrieval.

2.3.4 Structured and Graph-Based Decomposition

The most structurally sophisticated QD approaches move beyond flat sub-question lists toward hierarchically organized reasoning structures grounded in explicit semantic or logical representations.

AMR-Based QD [7] parses a complex question into its Abstract Meaning Representation (AMR)

graph, identifying semantic variables and relational paths to produce sub-questions corresponding to coherent query segments. This yields a decomposition interpretable in terms of the question's semantic structure rather than surface-level heuristics.

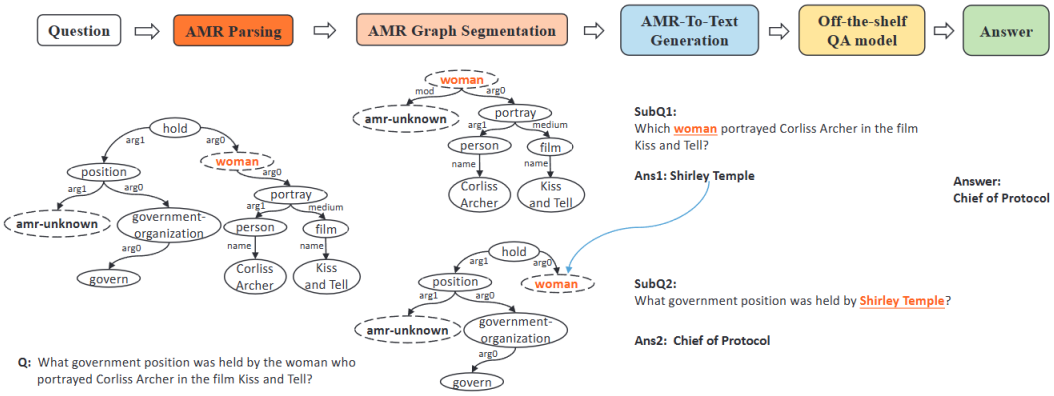


Figure 2.17: Overview of the QDAMR framework, which transforms a multi-hop question into an AMR graph, decomposes it into sub-questions, answers intermediate queries using a QA system, and substitutes results to obtain the final answer.

TreeQA [39] constructs hierarchical logic trees organized under the MECE principle (Mutually Exclusive and Collectively Exhaustive), ensuring full coverage without redundancy. It combines dense and sparse retrieval with an iterative self-correction mechanism that prunes inconsistent reasoning branches, improving robustness to retrieval noise.

BeamAggR [4] frames decomposition as a structured search over Question Decomposition Meaning Representation (QDMR) trees. Rather than greedily committing to a single decomposition path, it uses beam search to maintain multiple candidate trees simultaneously, aggregating evidence via bottom-up traversal and deferring path selection until sufficient evidence is gathered.

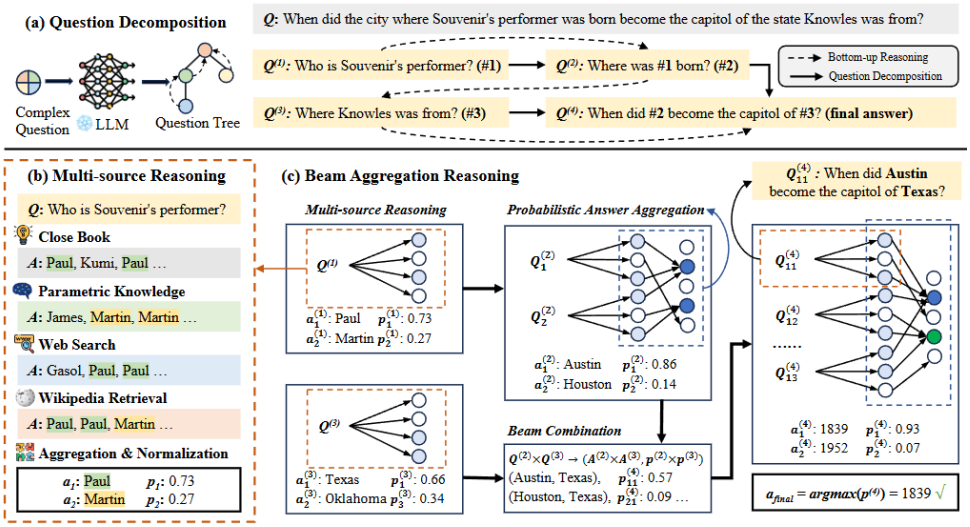


Figure 2.18: Overview of BeamAggR, which decomposes complex questions into trees, performs multi-source reasoning with probabilistic normalization, and aggregates beam-based reasoning trajectories to select the most promising answer path.

Chapter 3

Deep Learning Theoretical Background

This chapter provides a concise overview of core deep learning principles, establishing the theoretical foundations required for the analysis and methodologies presented in the remainder of this thesis.

3.1 Neural Networks: Recurrent Architectures

3.1.1 Fundamental Concepts

A **neuron** $\mathcal{N} : \mathbb{R}^n \to \mathbb{R}$ is the fundamental computational unit of a neural network, defined by

$$\mathcal{N}(\boldsymbol{x}) = \sigma(\boldsymbol{w}^\top \boldsymbol{x} + b),$$

where $\boldsymbol{w} \in \mathbb{R}^n$ is the weight vector, $b \in \mathbb{R}$ the bias, and $\sigma : \mathbb{R} \to \mathbb{R}$ the activation function. For fixed n and σ, the space of all neurons is parameterized by $(w_1, \ldots, w_n, b) \in \mathbb{R}^{n+1}$.

3.1.2 Feedforward Neural Networks

Definition 3.1.1 (Feedforward Neural Network)**.** *A **feedforward neural network** of depth $L \in \mathbb{N}$ is a function $\Phi : \mathbb{R}^{d_0} \to \mathbb{R}^{d_{L+1}}$ defined by the composition*

$$\boldsymbol{x}^{(0)} = \boldsymbol{x}, \tag{3.1a}$$

$$\boldsymbol{x}^{(\ell)} = \sigma\Big(\boldsymbol{w}^{(\ell-1)}\boldsymbol{x}^{(\ell-1)} + \boldsymbol{b}^{(\ell-1)}\Big), \quad \ell = 1, \ldots, L, \tag{3.1b}$$

$$\boldsymbol{x}^{(L+1)} = \boldsymbol{w}^{(L)}\boldsymbol{x}^{(L)} + \boldsymbol{b}^{(L)}, \tag{3.1c}$$

*where $\boldsymbol{w}^{(\ell)} \in \mathbb{R}^{d_{\ell+1} \times d_\ell}$ are weight matrices, $\boldsymbol{b}^{(\ell)} \in \mathbb{R}^{d_{\ell+1}}$ are bias vectors, and σ is applied componentwise in the hidden layers only. Networks with $L = 1$ are termed **shallow**; those with $L > 1$ are termed **deep**.*

3.1.3 Language Models and Sequential Data

A **language model** assigns a probability to a sequence of words $\{w_1, \ldots, w_m\}$. Since conditioning on the full history is intractable, the joint probability is approximated over a window of n preceding words:

$$P(w_1, \ldots, w_m) \approx \prod_{i=1}^{m} P(w_i \mid w_{i-n}, \ldots, w_{i-1}). \tag{3.2}$$

Classical n-gram models estimate these probabilities from corpus counts, e.g.

$$p(w_2 \mid w_1) = \frac{\text{count}(w_1, w_2)}{\text{count}(w_1)}, \qquad p(w_3 \mid w_1, w_2) = \frac{\text{count}(w_1, w_2, w_3)}{\text{count}(w_1, w_2)}. \tag{3.3}$$

Such models suffer from two fundamental limitations: *sparsity*, when n-gram combinations are absent from the training corpus, and poor *scalability*, as model size grows with n. These limitations motivate the use of neural architectures that learn distributed word representations.

3.1.4 Recurrent Architectures

Feedforward networks process fixed-size inputs and cannot naturally handle variable-length sequences. Recurrent neural networks address this by maintaining a hidden state that evolves as the sequence is processed, conditioning each output on all previous inputs.

Recurrent Neural Networks

Definition 3.1.2 (Recurrent Neural Network)**.** *A **recurrent neural network** processes a sequence $\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(T)} \in \mathbb{R}^{d_x}$ via the recurrence*

$$\boldsymbol{h}^{(t)} = \sigma\left(\boldsymbol{w}_h \boldsymbol{h}^{(t-1)} + \boldsymbol{w}_x \boldsymbol{x}^{(t)} + \boldsymbol{b}_h\right), \tag{3.4a}$$

$$\mathbf{y}^{(t)} = \text{softmax}\left(\boldsymbol{w}_y \boldsymbol{h}^{(t)}\right), \tag{3.4b}$$

where $\boldsymbol{h}^{(t)} \in \mathbb{R}^{d_h}$ is the hidden state (initialized to $\boldsymbol{h}^{(0)} = \mathbf{0}$), $\mathbf{y}^{(t)} \in \mathbb{R}^{d_y}$ is the output distribution over the vocabulary, and $\boldsymbol{w}_h$, $\boldsymbol{w}_x$, $\boldsymbol{w}_y$, $\boldsymbol{b}_h$ are shared parameters across all time steps.

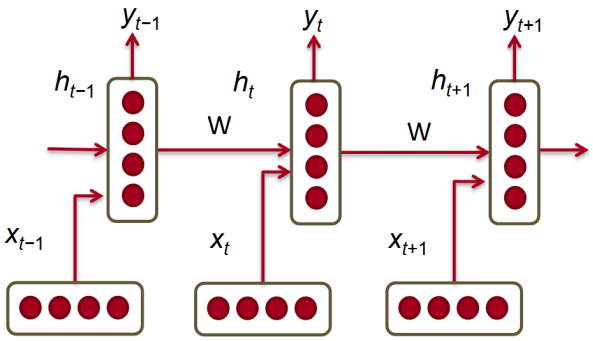


Figure 3.1: A Recurrent Neural Network (RNN). Three time-steps are shown.

Parameter sharing across time steps ensures the parameter count is independent of sequence length T. The standard training objective is the cross-entropy loss,

$$J = -\frac{1}{T} \sum_{t=1}^{T} \sum_{j=1}^{|V|} y_{t,j} \log \hat{y}_{t,j}, \tag{3.5}$$

and model quality is reported via **perplexity** $= 2^J$, where lower values indicate greater predictive confidence.

Bidirectional and Deep RNNs

Standard RNNs condition predictions on past context only. Bidirectional RNNs augment this by processing the sequence in both directions,

$$\overrightarrow{\boldsymbol{h}}^{(t)} = f\left(\overrightarrow{W}\,\boldsymbol{x}^{(t)} + \overrightarrow{V}\,\overrightarrow{\boldsymbol{h}}^{(t-1)} + \overrightarrow{\boldsymbol{b}}\right), \tag{3.6}$$

$$\overleftarrow{\boldsymbol{h}}^{(t)} = f\left(\overleftarrow{W}\,\boldsymbol{x}^{(t)} + \overleftarrow{V}\,\overleftarrow{\boldsymbol{h}}^{(t+1)} + \overleftarrow{\boldsymbol{b}}\right), \tag{3.7}$$

and combining both hidden states at the output: $\hat{y}^{(t)} = g\left(U\left[\overrightarrow{\boldsymbol{h}}^{(t)}; \overleftarrow{\boldsymbol{h}}^{(t)}\right] + \boldsymbol{c}\right)$. RNN layers may further be stacked into deep architectures, where the output of layer $i-1$ serves as input to layer i, with the final output produced by the topmost layer. Deeper architectures generally improve predictive accuracy at the cost of increased computational complexity.

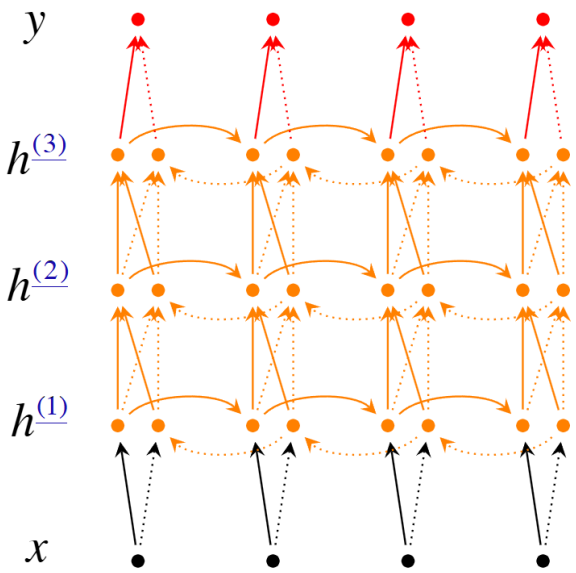


Figure 3.2: A deep bi-directional RNN with three RNN layers.

3.1.5 Gated Recurrent Units

Gated Recurrent Units (GRUs) augment the standard RNN with learnable gate mechanisms that regulate information flow across time steps, alleviating the difficulty of capturing long-range dependencies.

Definition 3.1.3 (Gated Recurrent Unit)**.** *A **GRU** computes the hidden state $\boldsymbol{h}^{(t)}$ from $\boldsymbol{x}^{(t)}$ and $\boldsymbol{h}^{(t-1)}$ through*

$$\boldsymbol{z}^{(t)} = \sigma_{\text{sig}}\left(W^{(z)}\boldsymbol{x}^{(t)} + U^{(z)}\boldsymbol{h}^{(t-1)}\right), \tag{3.8a}$$

$$\boldsymbol{r}^{(t)} = \sigma_{\text{sig}}\left(W^{(r)}\boldsymbol{x}^{(t)} + U^{(r)}\boldsymbol{h}^{(t-1)}\right), \tag{3.8b}$$

$$\widetilde{\boldsymbol{h}}^{(t)} = \tanh\left(\boldsymbol{r}^{(t)} \odot U\boldsymbol{h}^{(t-1)} + W\boldsymbol{x}^{(t)}\right), \tag{3.8c}$$

$$\boldsymbol{h}^{(t)} = \left(1 - \boldsymbol{z}^{(t)}\right) \odot \widetilde{\boldsymbol{h}}^{(t)} + \boldsymbol{z}^{(t)} \odot \boldsymbol{h}^{(t-1)}, \tag{3.8d}$$

where $\odot$ denotes the Hadamard product.

The **reset gate** $\boldsymbol{r}^{(t)}$ controls the contribution of the previous state to the candidate $\widetilde{\boldsymbol{h}}^{(t)}$, effectively discarding irrelevant history when $\boldsymbol{r}^{(t)} \approx \mathbf{0}$. The **update gate** $\boldsymbol{z}^{(t)}$ then interpolates between the candidate and the previous state, carrying past information forward when $\boldsymbol{z}^{(t)} \approx \mathbf{1}$.

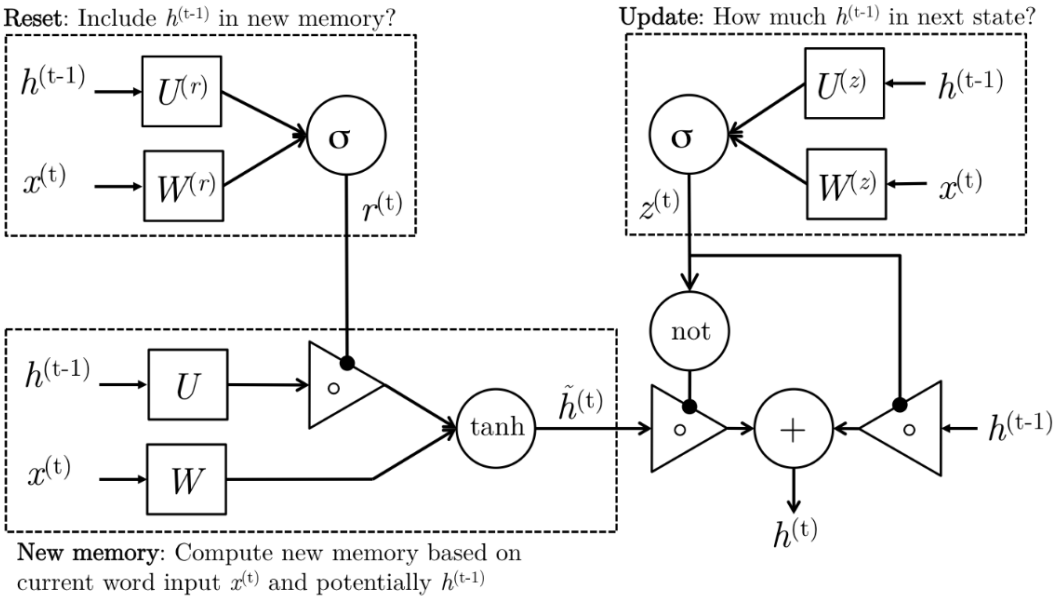


Figure 3.3: The detailed internals of a GRU.

3.1.6 Long Short-Term Memory Networks

Long Short-Term Memory (LSTM) networks [12] address the vanishing gradient problem through an explicit separation of short- and long-term memory via a dedicated *cell state*.

Definition 3.1.4 (Long Short-Term Memory Network)**.** *An **LSTM** maintains a hidden state* $\boldsymbol{h}^{(t)} \in \mathbb{R}^{d_h}$ *and a cell state* $\boldsymbol{c}^{(t)} \in \mathbb{R}^{d_h}$*, updated at each time step by*

$$\boldsymbol{i}^{(t)} = \sigma_{\text{sig}}\left(W^{(i)}\boldsymbol{x}^{(t)} + U^{(i)}\boldsymbol{h}^{(t-1)}\right), \tag{3.9a}$$

$$\boldsymbol{f}^{(t)} = \sigma_{\text{sig}}\left(W^{(f)}\boldsymbol{x}^{(t)} + U^{(f)}\boldsymbol{h}^{(t-1)}\right), \tag{3.9b}$$

$$\boldsymbol{o}^{(t)} = \sigma_{\text{sig}}\left(W^{(o)}\boldsymbol{x}^{(t)} + U^{(o)}\boldsymbol{h}^{(t-1)}\right), \tag{3.9c}$$

$$\widetilde{\boldsymbol{c}}^{(t)} = \tanh\left(W^{(c)}\boldsymbol{x}^{(t)} + U^{(c)}\boldsymbol{h}^{(t-1)}\right), \tag{3.9d}$$

$$\boldsymbol{c}^{(t)} = \boldsymbol{f}^{(t)} \odot \boldsymbol{c}^{(t-1)} + \boldsymbol{i}^{(t)} \odot \widetilde{\boldsymbol{c}}^{(t)}, \tag{3.9e}$$

$$\boldsymbol{h}^{(t)} = \boldsymbol{o}^{(t)} \odot \tanh\left(\boldsymbol{c}^{(t)}\right). \tag{3.9f}$$

The **forget gate** $\boldsymbol{f}^{(t)}$ determines what fraction of the previous cell state is retained, while the **input gate** $\boldsymbol{i}^{(t)}$ controls how much of the new candidate $\widetilde{\boldsymbol{c}}^{(t)}$ is written to memory. The predominantly *additive* update in (3.9e) is the principal mechanism by which LSTMs mitigate vanishing gradients, as gradient signals can propagate through the cell state with minimal attenuation. The **output gate** $\boldsymbol{o}^{(t)}$ then selectively exposes the cell state in the hidden state $\boldsymbol{h}^{(t)}$, maintaining a separation between long-term storage and short-term output.

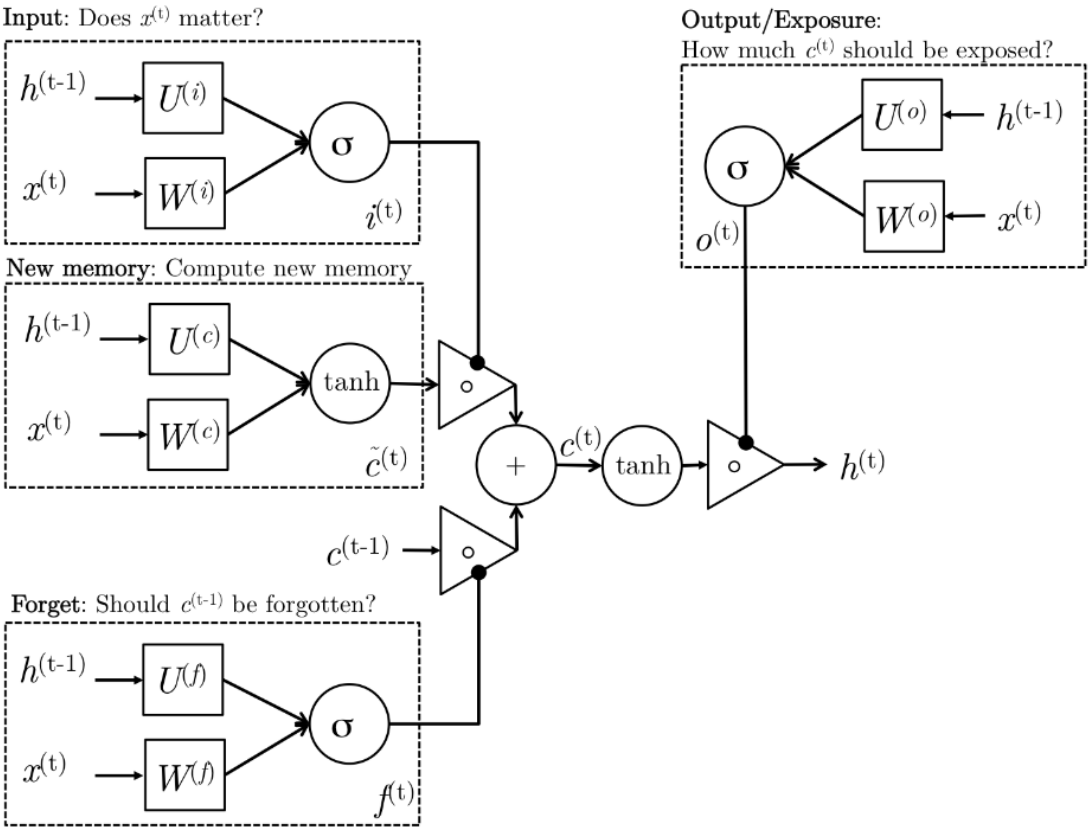


Figure 3.4: The detailed internals of an LSTM.

3.2 Transformers

Transformers are a class of neural network architectures that have become fundamental in modern deep learning, particularly for sequence-to-sequence tasks. Unlike RNNs, transformers process entire sequences in parallel through an *attention* mechanism, which allows the model to weigh the relevance of all input positions when producing each output.

3.2.1 Attention

Dot-Product Self-Attention

Let $\boldsymbol{X} \in \mathbb{R}^{n \times d}$ denote an input sequence of length n with feature dimension d. The self-attention mechanism projects $\boldsymbol{X}$ into three matrices via learned linear transformations,

$$\boldsymbol{Q} = \boldsymbol{X}\boldsymbol{W}_Q \in \mathbb{R}^{n \times d_k}, \quad \boldsymbol{K} = \boldsymbol{X}\boldsymbol{W}_K \in \mathbb{R}^{n \times d_k}, \quad \boldsymbol{V} = \boldsymbol{X}\boldsymbol{W}_V \in \mathbb{R}^{n \times d_v}, \tag{3.10}$$

termed the *query*, *key*, and *value* matrices respectively, where $\boldsymbol{W}_Q, \boldsymbol{W}_K \in \mathbb{R}^{d \times d_k}$ and $\boldsymbol{W}_V \in \mathbb{R}^{d \times d_v}$ are learnable weights. Pairwise similarities between queries and keys are computed, scaled, and normalized to produce attention weights:

$$\text{Attention}(\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}) = \text{softmax}\left(\frac{\boldsymbol{Q}\boldsymbol{K}^\top}{\sqrt{d_k}}\right)\boldsymbol{V} \in \mathbb{R}^{n \times d_v}. \tag{3.11}$$

The output at each position is thus a weighted combination of the value vectors, where the weights encode pairwise relevance. The scaling by $1/\sqrt{d_k}$ prevents the dot products from growing large in magnitude as d_k increases, stabilizing the softmax gradients during training.

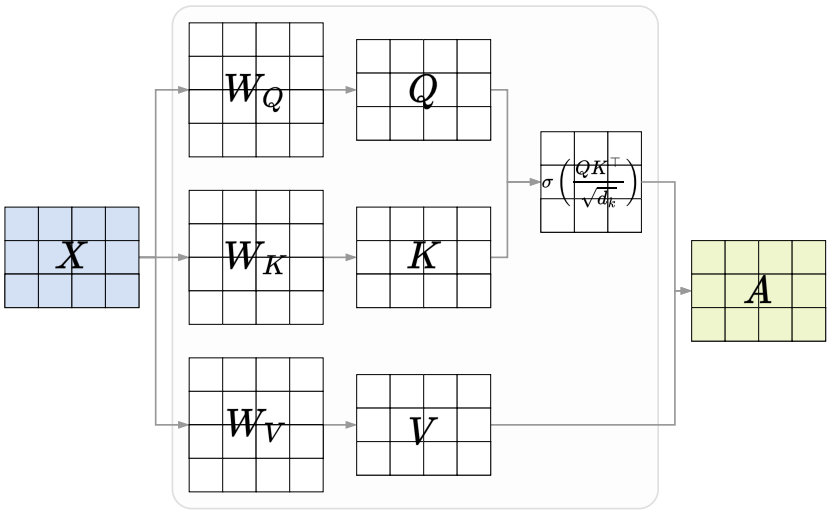


Figure 3.5: Visualisation of the dot-product self-attention mechanism, with $d_k = d_v = 4$.

Multi-Head Attention

Multi-head attention computes h attention operations in parallel, each with independent projections, enabling the model to jointly attend to information from different representation subspaces. For head $i = 1, \ldots, h$,

$$\text{head}_i = \text{Attention}(\boldsymbol{X}\boldsymbol{W}_Q^{(i)}, \boldsymbol{X}\boldsymbol{W}_K^{(i)}, \boldsymbol{X}\boldsymbol{W}_V^{(i)}) \in \mathbb{R}^{n \times d_v}. \tag{3.12}$$

The outputs are concatenated and linearly projected:

$$\text{MultiHead}(\boldsymbol{X}) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\boldsymbol{W}_O, \tag{3.13}$$

where $\boldsymbol{W}_O \in \mathbb{R}^{hd_v \times d}$. Setting $d_k = d_v = d/h$ ensures the concatenated output matches the input dimension d.

Transformer Layer

A transformer layer combines multi-head attention with a position-wise feedforward network, residual connections, and layer normalization. Given input $\boldsymbol{X} \in \mathbb{R}^{n \times d}$, the layer computes:

$$\boldsymbol{X}^{(1)} = \text{LayerNorm}(\boldsymbol{X} + \text{MultiHead}(\boldsymbol{X})), \tag{3.14}$$

$$\boldsymbol{X}^{(2)} = \text{LayerNorm}\Big(\boldsymbol{X}^{(1)} + \text{FFN}(\boldsymbol{X}^{(1)})\Big), \tag{3.15}$$

where layer normalization is defined for each position as

$$\text{LayerNorm}(\boldsymbol{x}) = \frac{\boldsymbol{x} - \mu}{\sqrt{\sigma^2 + \varepsilon}} \odot \boldsymbol{\gamma} + \boldsymbol{\beta}, \tag{3.16}$$

with μ, σ^2 the per-position mean and variance, $\varepsilon > 0$ a stability constant, and $\boldsymbol{\gamma}, \boldsymbol{\beta} \in \mathbb{R}^d$ learnable parameters. The position-wise feedforward network is

$$\text{FFN}(\boldsymbol{x}) = \boldsymbol{W}_2\, \sigma(\boldsymbol{W}_1 \boldsymbol{x} + \boldsymbol{b}_1) + \boldsymbol{b}_2, \tag{3.17}$$

with $\boldsymbol{W}_1 \in \mathbb{R}^{d_{ff} \times d}$, $\boldsymbol{W}_2 \in \mathbb{R}^{d \times d_{ff}}$, and intermediate dimension $d_{ff} = 4d$ in standard practice. The residual connections facilitate gradient flow in deep networks, while layer normalization stabilizes training.

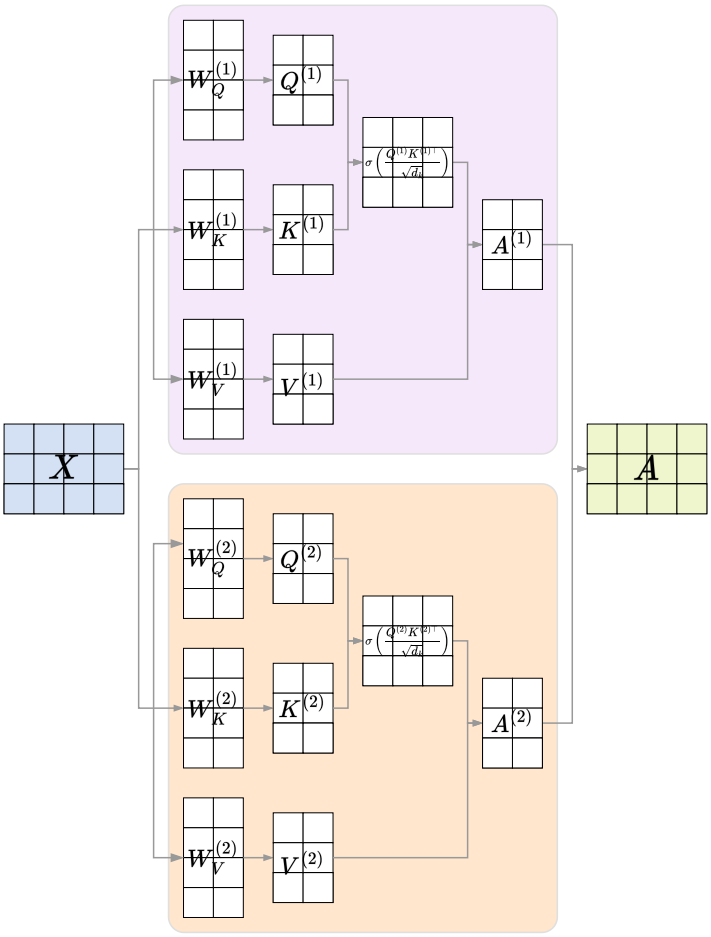


Figure 3.6: Multi-head self-attention with $h = 2$ heads and $d_k = d_v = 2$.

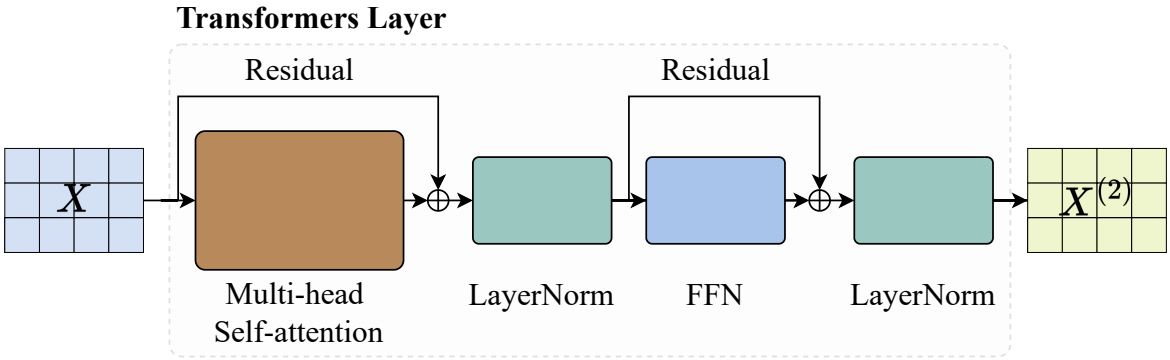


Figure 3.7: The transformer layer. Multi-head attention (with residual connection and LayerNorm) is followed by a position-wise feedforward network (with residual connection and LayerNorm).

Remark 3.2.1. *Since the attention mechanism is permutation-invariant,* positional encodings *are added to the input embeddings to inject sequence order. The standard sinusoidal encoding at position i and dimension j is*

$$PE_{i,2j} = \sin\left(\frac{i}{10000^{2j/d}}\right), \qquad PE_{i,2j+1} = \cos\left(\frac{i}{10000^{2j/d}}\right).$$

3.2.2 Encoder-Decoder Architecture

Encoder

The encoder is a stack of N identical transformer layers. Given input embeddings with positional encodings $\boldsymbol{X}^{(0)} \in \mathbb{R}^{n \times d}$, the encoder computes

$$\boldsymbol{X}^{(\ell)} = \text{TransformerLayer}(\boldsymbol{X}^{(\ell-1)}), \quad \ell = 1, \ldots, N, \tag{3.18}$$

producing contextualized representations $\boldsymbol{X}^{(N)} \in \mathbb{R}^{n \times d}$ in which each position incorporates information from the full input sequence.

Decoder

Each of the N decoder layers comprises three sub-layers. First, **masked multi-head self-attention** prevents positions from attending to future tokens by setting

$$\tilde{S}_{ij} = \begin{cases} \boldsymbol{q}_i^\top \boldsymbol{k}_j / \sqrt{d_k} & j \le i, \\ -\infty & j > i, \end{cases} \tag{3.19}$$

before the softmax, so that $A_{ij} = 0$ for all $j > i$. Second, **cross-attention** allows the decoder to attend to the encoder output:

$$\text{CrossAttention}(\boldsymbol{Y}, \boldsymbol{X}^{(N)}) = \text{softmax}\left(\frac{\boldsymbol{Y}\boldsymbol{W}_Q (\boldsymbol{X}^{(N)}\boldsymbol{W}_K)^\top}{\sqrt{d_k}} \right) \boldsymbol{X}^{(N)}\boldsymbol{W}_V, \tag{3.20}$$

where queries originate from the decoder hidden state $\boldsymbol{Y}$ and keys and values from the encoder output. Third, a **position-wise feedforward network** identical to that in the encoder is applied. Each sub-layer is wrapped with a residual connection and layer normalization:

$$\boldsymbol{Y}^{(1)} = \text{LayerNorm}(\boldsymbol{Y} + \text{MaskedMultiHead}(\boldsymbol{Y})), \tag{3.21}$$

$$\boldsymbol{Y}^{(2)} = \text{LayerNorm}(\boldsymbol{Y}^{(1)} + \text{CrossAttention}(\boldsymbol{Y}^{(1)}, \boldsymbol{X}^{(N)})), \tag{3.22}$$

$$\boldsymbol{Y}^{(3)} = \text{LayerNorm}(\boldsymbol{Y}^{(2)} + \text{FFN}(\boldsymbol{Y}^{(2)})). \tag{3.23}$$

Output Generation

The decoder generates output tokens autoregressively. After N decoder layers, the final hidden states are projected to vocabulary logits and normalized:

$$p(y_i \mid y_{<i}, \boldsymbol{X}) = \text{softmax}\Big(\boldsymbol{Y}^{(N)}\boldsymbol{W}_{out} + \boldsymbol{b}_{out} \Big)_i, \tag{3.24}$$

where $\boldsymbol{W}_{out} \in \mathbb{R}^{d \times V}$ and V is the vocabulary size. The model is trained to minimize the cross-entropy loss over the target sequence,

$$\mathscr{L}(\boldsymbol{w}) = -\frac{1}{m} \sum_{i=1}^{m} \log p(y_i^* \mid y_{<i}^*, \boldsymbol{X}), \tag{3.25}$$

while at inference time tokens are generated one at a time, each new token being fed back into the decoder.

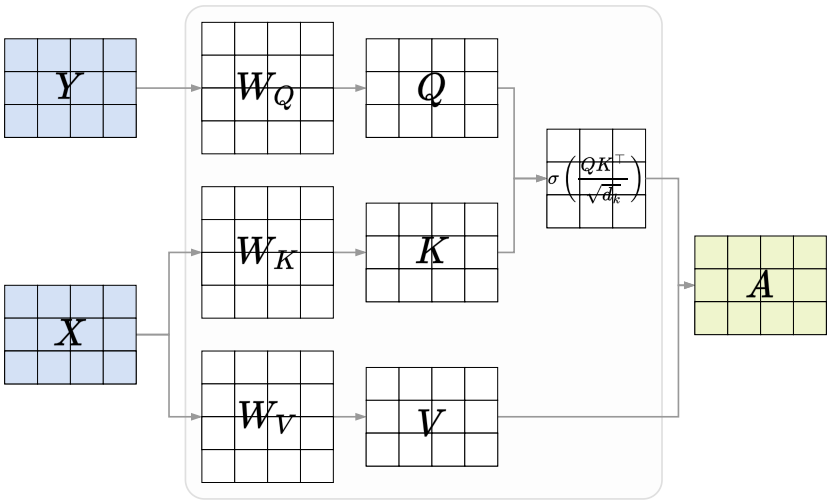


Figure 3.8: Cross-attention. Queries are derived from the decoder embeddings; keys and values from the encoder embeddings.

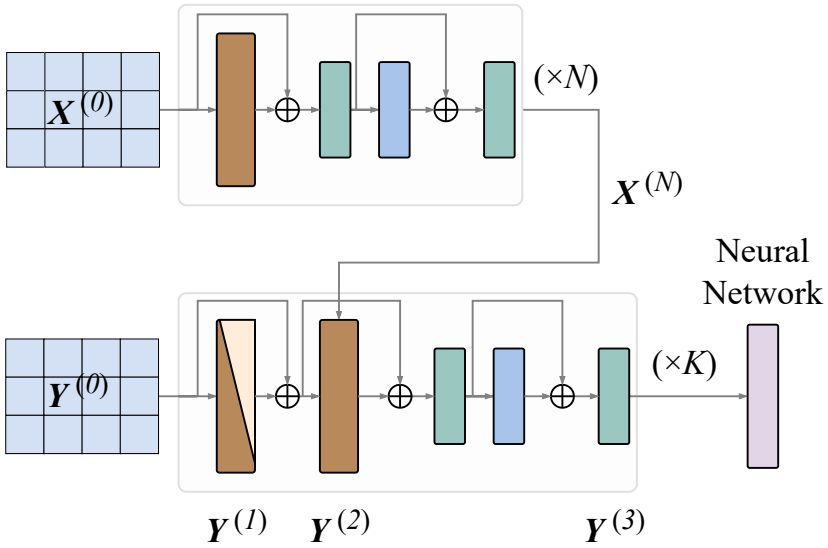


Figure 3.9: Encoder-decoder architecture. The encoder processes the source sequence; the decoder generates the target sequence using masked self-attention and cross-attention over the encoder output.

Remark 3.2.2. *Compared to RNNs, transformers offer two principal advantages: (i) all sequence positions are processed in parallel, and (ii) direct attention connections between all position pairs enable more effective modeling of long-range dependencies. The per-layer complexity of self-attention is $O(n^2 d)$ versus $O(nd^2)$ for recurrent layers, which is favorable when $n \ll d$. However, the $O(n^2)$ memory footprint becomes prohibitive for very long sequences, motivating the development of efficient attention variants.*

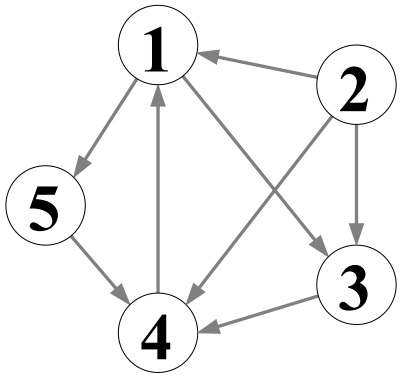
3.3 Graph Neural Networks

This section provides background on graph neural networks as a framework for computing node embeddings. Training and parameter update details are omitted as they are not relevant to the problem at hand.

3.3.1 Graph

Definition 3.3.1 (Graph)**.** *A **graph** $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ consists of a finite node set $\mathcal{V}$ and an edge set $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$. The graph is **simple** if it contains no loops or multiple edges, and **undirected** if $(u,v) \in \mathcal{E} \Leftrightarrow (v,u) \in \mathcal{E}$. The **neighborhood** of a node $v \in \mathcal{V}$ is $\mathcal{N}(v) = \{u \in \mathcal{V} \mid (u,v) \in \mathcal{E}\}$.*

For a simple undirected graph with $\mathcal{V} = \{v_1, \ldots, v_N\}$, the **adjacency matrix** $\mathbf{A} \in \mathbb{R}^{N \times N}$ has entries $a_{ij} = \mathbf{1}[(v_i, v_j) \in \mathcal{E}]$, and the **degree matrix D** is the diagonal matrix with $d_{ii} = \sum_j a_{ij} = |\mathcal{N}(v_i)|$. Adding self-loops yields $\hat{\mathbf{A}} = \mathbf{A} + \mathbf{I}_N$ with corresponding degree matrix $\hat{\mathbf{D}}$ satisfying $\hat{d}_{ii} = d_{ii} + 1$.



0	0	1	0	1
1	0	1	1	0
0	0	0	1	0
1	0	0	0	0
0	0	0	1	0

Figure 3.10: A graph and its corresponding adjacency matrix.

3.3.2 An Encoder-Decoder Perspective

GNNs can be understood through an encoder-decoder framework. The **encoder** maps each node $v \in \mathcal{V}$ to a vector embedding $\boldsymbol{z}_v \in \mathbb{R}^d$,

$$\text{ENC} : \mathcal{V} \mapsto \mathbb{R}^d, \tag{3.26}$$

while the **decoder** reconstructs graph statistics from the embeddings. Standard decoders are pairwise,

$$\text{DEC} : \mathbb{R}^d \times \mathbb{R}^d \mapsto \mathbb{R}^+, \tag{3.27}$$

predicting, for instance, the likelihood of an edge between two nodes from their embeddings.

3.3.3 Neural Message Passing

The defining feature of a GNN is neural message passing, in which each node iteratively aggregates information from its neighborhood to update its embedding. At iteration k, the hidden state $\boldsymbol{h}_u^{(k)}$ of node u is updated as

$$\boldsymbol{m}_{\mathcal{N}(u)}^{(k)} = \text{AGGREGATE}^{(k)}\left(\left\{\boldsymbol{h}_v^{(k-1)} \mid v \in \mathcal{N}(u)\right\}\right), \tag{3.28a}$$

$$\boldsymbol{h}_u^{(k)} = \text{UPDATE}^{(k)}\left(\boldsymbol{h}_u^{(k-1)}, \boldsymbol{m}_{\mathcal{N}(u)}^{(k)}\right), \tag{3.28b}$$

where AGGREGATE and UPDATE are differentiable functions. Embeddings are initialized from node features, $\boldsymbol{h}_u^{(0)} = \boldsymbol{x}_u$, and the final embedding is taken from the last layer: $\boldsymbol{z}_u = \boldsymbol{h}_u^{(K)}$. Since AGGREGATE acts on a set, GNNs defined in this way are permutation equivariant by design.

The Basic GNN

The simplest instantiation defines aggregation as a sum and update as a linear transformation followed by a nonlinearity:

$$\boldsymbol{h}_u^{(k)} = \sigma\left(\mathbf{W}_{\text{self}}^{(k)}\boldsymbol{h}_u^{(k-1)} + \mathbf{W}_{\text{neigh}}^{(k)}\sum_{v\in\mathcal{N}(u)}\boldsymbol{h}_v^{(k-1)} + \boldsymbol{b}^{(k)}\right), \tag{3.29}$$

where $\mathbf{W}_{\text{self}}^{(k)}, \mathbf{W}_{\text{neigh}}^{(k)} \in \mathbb{R}^{d^{(k)}\times d^{(k-1)}}$ are trainable weight matrices and σ is an elementwise nonlinearity. Adding self-loops and tying $\mathbf{W}_{\text{self}} = \mathbf{W}_{\text{neigh}} = \mathbf{W}$ yields the compact graph-level update

$$\mathbf{H}^{(k)} = \sigma\left(\hat{\mathbf{A}}\,\mathbf{H}^{(k-1)}\mathbf{W}^{(k)}\right). \tag{3.30}$$

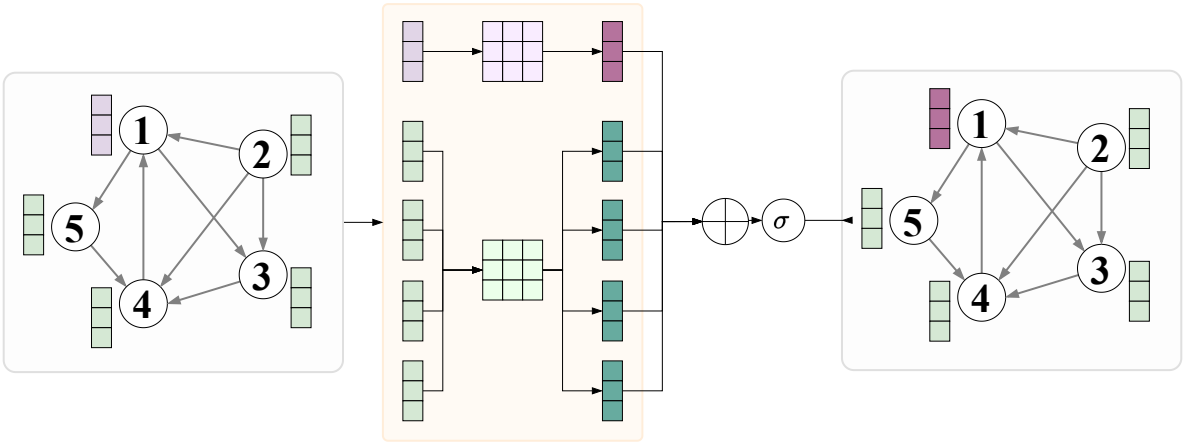


Figure 3.11: Visualisation of basic message passing for node 1.

3.3.4 Generalized Neighborhood Aggregation

Neighborhood Normalization

When node degrees vary widely, the unnormalized sum in (3.29) can cause numerical instability. A simple remedy is mean aggregation,

$$\boldsymbol{m}_{\mathcal{N}(u)} = \frac{\sum_{v\in\mathcal{N}(u)}\boldsymbol{h}_v}{|\mathcal{N}(u)|}, \tag{3.31}$$

while the more effective symmetric normalization,

$$\boldsymbol{m}_{\mathcal{N}(u)} = \sum_{v\in\mathcal{N}(u)}\frac{\boldsymbol{h}_v}{\sqrt{|\mathcal{N}(u)|\,|\mathcal{N}(v)|}}, \tag{3.32}$$

has become one of the most widely adopted GNN baselines.

Set Aggregators

A principled approach to aggregation draws on the theory of permutation-invariant functions. Any permutation-invariant mapping from a set of embeddings to a single embedding can be

approximated arbitrarily well by

$$\boldsymbol{m}_{\mathcal{N}(u)} = \mathrm{MLP}_\theta\left(\sum_{v\in\mathcal{N}(u)} \mathrm{MLP}_\theta(\boldsymbol{h}_v)\right). \tag{3.33}$$

An alternative, Janossy pooling, averages the output of a permutation-sensitive function ρ_ϕ over a set of permutations Π:

$$\boldsymbol{m}_{\mathcal{N}(u)} = \mathrm{MLP}_\theta\left(\frac{1}{|\Pi|}\sum_{\pi_i\in\Pi} \rho_\phi\left(\boldsymbol{h}_{v_1},\ldots,\boldsymbol{h}_{v_{|\mathcal{N}(u)|}}\right)_{\pi_i}\right), \tag{3.34}$$

where ρ_ϕ is typically implemented as an LSTM.

Neighborhood Attention

Graph Attention Networks (GATs) assign a learned importance weight $\alpha_{u,v}$ to each neighbor during aggregation,

$$\boldsymbol{m}_{\mathcal{N}(u)} = \sum_{v\in\mathcal{N}(u)} \alpha_{u,v}\,\boldsymbol{h}_v, \tag{3.35}$$

where the original GAT computes coefficients via

$$\alpha_{u,v} = \frac{\exp\big(\boldsymbol{a}^\top[\mathbf{W}\boldsymbol{h}_u \,\|\, \mathbf{W}\boldsymbol{h}_v]\big)}{\sum_{v'\in\mathcal{N}(u)}\exp\big(\boldsymbol{a}^\top[\mathbf{W}\boldsymbol{h}_u \,\|\, \mathbf{W}\boldsymbol{h}_{v'}]\big)}, \tag{3.36}$$

with $\mathbf{W}\in\mathbb{R}^{d'\times d}$ and $\boldsymbol{a}\in\mathbb{R}^{2d'}$ learnable parameters. Alternative scoring functions include the bilinear form $\alpha_{u,v}\propto\exp(\boldsymbol{h}_u^\top\mathbf{W}\boldsymbol{h}_v)$ and MLP-based scoring $\alpha_{u,v}\propto\exp(\mathrm{MLP}(\boldsymbol{h}_u\,\|\,\boldsymbol{h}_v))$.

3.4 Knowledge Graphs

In the field of knowledge representation and reasoning, a *Knowledge Graph* is a structured knowledge base that represents information using a graph-structured data model. In a knowledge graph, data is encoded as a network of interconnected entities - such as objects, events, situations, or abstract concepts - along with the semantic relationships that link them. This representation enables both human - readable organization of knowledge and machine-operable reasoning over complex, interconnected data.

Knowledge graphs traditionally power large-scale search engines and intelligent services, including major systems such as Google Search, Bing, and Yahoo, as well as question-answering assistants like Siri, Alexa, and WolframAlpha. More recently, with advances in graph neural networks and representation learning, knowledge graphs have gained significant traction in scientific research, enterprise analytics, and intelligent decision support systems.

3.4.1 Definition

Formally, a knowledge graph is a directed, labeled graph

$$\mathcal{G} = (\mathcal{V},\mathcal{E},\mathcal{R}),$$

where:

- $\mathscr{V}$ is a set of **entities** or **nodes**, representing real-world objects, people, locations, biological components, or abstract concepts.
- $\mathscr{R}$ is a set of **relation types** that define permissible relationships.
- $\mathscr{E} \subseteq \mathscr{V} \times \mathscr{R} \times \mathscr{V}$ is a set of **edges**, each describing a directed relationship between two entities.

Each edge (h, r, t) is typically called a *triple*, where h is the head entity, r is the relation, and t is the tail entity.

Knowledge graphs differ from traditional databases by emphasizing *semantic structure*: they encode not only data records but also the meaning and context of how entities relate to each other. As such, they can be viewed as a form of semantic database comparable - though conceptually distinct - to Relational Database Management Systems (RDBMS), NoSQL stores, or Object-Oriented Database Management Systems (OODBMS).

3.4.2 Example

A simple example of a knowledge graph is a small graph describing relationships among people, organizations, and locations (Figure 3.12): (Alan Turing, *workedAt*, University of Manchester), (University of Manchester, *locatedIn*, Manchester), (Alan Turing, *bornIn*, London). Here, "Alan Turing" and "University of Manchester" are entities, while relations such as *workedAt* and *bornIn* capture the semantic structure connecting them. Larger knowledge graphs contain millions or billions of such triples.

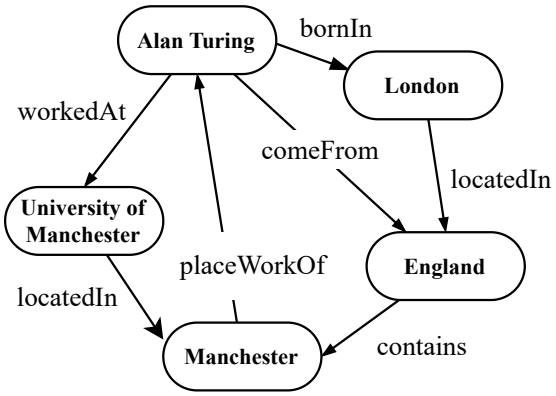


Figure 3.12: Graph Neural Network. A simple example of a knowledge graph is a small graph describing relationships among people, organizations, and locations.

3.4.3 Applications

Knowledge graphs support a broad range of real-world applications due to their ability to represent complex, hierarchical, and interconnected information:

- **Search engines.** Knowledge graphs improve search accuracy and relevance by linking concepts and enabling semantic understanding of user queries.
- **Recommendation systems.** By modeling rich relationships between items, users, and attributes, knowledge graphs enable context-aware and cross - domain recommendations (e.g., "users who liked this movie may also enjoy this book").

- **Fraud detection.** Analyzing relational patterns in transactional, financial, or behavioral graphs helps detect anomalous or suspicious activities that are not apparent from isolated data points.
- **Predictive maintenance.** Knowledge graphs can encode dependencies among components in industrial systems, enabling prediction of failure points based on relational patterns and historical interactions.
- **Question answering and virtual assistants.** Assistants such as Siri and Alexa rely on knowledge graphs to provide structured, context-aware answers based on interconnected facts.
- **Scientific discovery.** In biomedical, chemical, and materials science domains, knowledge graphs integrate heterogeneous datasets to enable hypothesis generation, link prediction, and knowledge-driven reasoning.

In summary, a knowledge graph provides a flexible, semantically rich framework for representing real-world information as a graph of entities and relationships. Its ability to encode meaning, structure, and context makes it a powerful foundation for modern AI systems, data analytics, and intelligent decision-making.

3.5 Abstract Meaning Representation

Abstract Meaning Representation (AMR) is a semantic formalism designed to capture the meaning of natural language sentences as directed, rooted graphs. Originally proposed by [1], AMR encodes "who did what to whom" in a sentence by abstracting away from surface syntactic variation, thereby providing a canonical, language-neutral representation of sentence-level semantics. AMR has since become a widely adopted benchmark in natural language processing (NLP), underpinning tasks such as machine translation, summarization, question answering, and information extraction.

3.5.1 Definition

Formally, an AMR graph is a directed, acyclic graph (DAG)

$$\mathscr{A} = (\mathscr{V}, \mathscr{E}, \ell),$$

where:

- $\mathscr{V}$ is a set of **nodes**, each representing an abstract concept — a PropBank frame [24], a named entity, or a general concept derived from the sentence's meaning.
- $\mathscr{E} \subseteq \mathscr{V} \times \mathscr{L} \times \mathscr{V}$ is a set of directed **edges**, each labeled with a semantic relation from a fixed inventory $\mathscr{L}$.
- $\ell : \mathscr{V} \to \Sigma$ is a **concept labeling function** that assigns a symbolic concept label to each node.

The graph is rooted at a single node representing the main predicate or focus of the sentence. Unlike dependency trees, AMR graphs are not anchored to individual word tokens; instead, they operate at the level of abstract meaning, enabling a single graph to represent semantically equivalent sentences regardless of their syntactic realization.

3.5.2 Example

Consider the sentence *"The boy wants the girl to believe him."* Its AMR representation abstracts over syntactic differences between this and semantically equivalent paraphrases, encoding the predicate–argument structure as a graph:

```
(w / want-01
:ARG0 (b / boy)
:ARG1 (b2 / believe-01
:ARG0 (g / girl)
:ARG1 b))
```

Here, `want-01` and `believe-01` are PropBank framesets, while `:ARG0` and `:ARG1` denote core semantic roles. The co-reference between the *boy* as the wanter and the object of belief is captured by re-using the node variable `b`, a capability that naturally handles phenomena such as control, raising, and anaphora [1].

3.5.3 AMR Parsing

The task of converting a natural language sentence into its AMR graph is known as *AMR parsing*. Early approaches relied on rule-based and statistical methods [9], but modern AMR parsers are predominantly neural. Sequence-to-sequence architectures [16] and, more recently, pre-trained transformer-based models [8] have achieved strong performance on standard benchmarks such as LDC2020T02.

In summary, Abstract Meaning Representation provides a linguistically principled, graph-structured formalism for capturing sentence-level semantics. Its ability to abstract over surface syntactic variation while preserving predicate–argument structure, negation, and coreference makes it a powerful intermediate representation for a broad range of natural language understanding tasks.

Chapter 4

Reinforcement Learning Theoretical Background

This chapter presents the foundational theory of reinforcement learning, serving as a basis for the analysis and methodologies discussed in the remainder of this thesis.

4.1 Introduction to Reinforcement Learning

Reinforcement learning (RL) studies how an autonomous agent learns to make decisions through sequential interaction with an environment. At each time step, the agent observes a state, selects an action, transitions to a new state, and receives a scalar reward. The objective is to learn a policy that maximizes the expected long-term return.

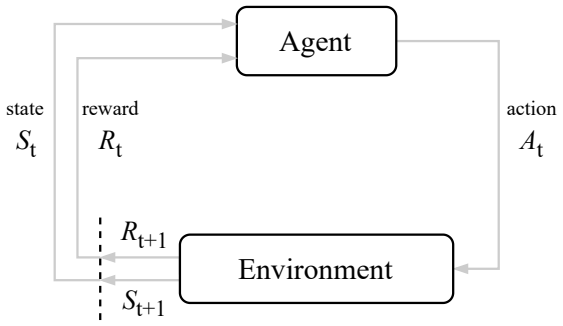


Figure 4.1: The standard RL framework. At time step t, the agent observes state S_t, takes action A_t, and receives reward R_t.

4.1.1 Markov Decision Process

The standard formalism for RL is the **Markov Decision Process** (MDP), defined by the tuple $(\mathscr{S}, \mathscr{A}, P, R)$. Here $\mathscr{S}$ is the **state space**, whose elements $s_t \in \mathscr{S}$ encode all information about the environment at time t; $\mathscr{A}$ is the **action space**, with $\mathscr{A}(s_t) \subseteq \mathscr{A}$ denoting the feasible actions in state s_t; $P(s_{t+1} \mid s_t, a_t)$ is the **transition distribution** specifying the probability of reaching s_{t+1} after taking action a_t in state s_t; and $R : \mathscr{S} \times \mathscr{A} \times \mathscr{S} \to \mathbb{R}$ is the **reward function**, with $r_t = R(s_t, a_t, s_{t+1})$ providing a scalar evaluation of each transition. The transition distribution

satisfies the **Markov property**,

$$P(s_{t+1} \mid s_t, a_t, s_{t-1}, a_{t-1}, \dots) = P(s_{t+1} \mid s_t, a_t),$$

so that future evolution depends only on the current state-action pair.

4.1.2 Policy, Trajectories, and Return

A **policy** π specifies the agent's behavior. A stochastic policy is a conditional distribution $\pi(a \mid s) = P(a_t = a \mid s_t = s)$; a deterministic policy is a function $\mu : \mathscr{S} \to \mathscr{A}$ with $a_t = \mu(s_t)$.

A **trajectory** $\tau = (s_0, a_0, r_1, s_1, a_1, r_2, \dots, s_T)$ is a sequence of states, actions, and rewards generated through interaction with the environment, where T may be finite (episodic) or infinite (continuing). The **return** from time step t is the discounted cumulative reward,

$$G_t = \sum_{k=0}^{T-t} \gamma^k r_{t+k+1},$$

where $\gamma \in [0,1)$ is the discount factor governing the weight assigned to future rewards. The objective of RL is to find a policy π that maximizes the expected return $\mathbb{E}_{\tau\sim\pi}[G_0]$. In episodic settings, each trajectory begins from an initial state $s_0 \sim p_0$ and terminates upon reaching a terminal state or after a fixed horizon T.

4.2 State Values, Optimality, and the Bellman Equations

4.2.1 State and Action Values

Definition 4.2.1 (State Value and Action Value)**.** *Given a policy π, the **state value** of $s \in \mathscr{S}$ and the **action value** of a pair $(s,a) \in \mathscr{S} \times \mathscr{A}$ are respectively defined as*

$$v_\pi(s) = \mathbb{E}\left[G_t \mid S_t = s\right], \tag{4.1}$$

$$q_\pi(s,a) = \mathbb{E}\left[G_t \mid S_t = s, A_t = a\right], \tag{4.2}$$

where $G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$ is the discounted return.

4.2.2 Bellman Equation

The return satisfies the recursion $G_t = R_{t+1} + \gamma G_{t+1}$, which yields a recursive characterization of the state value. Expanding via the law of total expectation and the Markov property gives the **Bellman equation**:

Definition 4.2.2 (Bellman Equation)**.**

$$v_\pi(s) = \underbrace{\sum_{a\in\mathscr{A}} \pi(a \mid s) \sum_{r\in\mathscr{R}} p(r \mid s,a)\, r}_{\text{mean immediate reward}} + \gamma \underbrace{\sum_{a\in\mathscr{A}} \pi(a \mid s) \sum_{s'\in\mathscr{S}} p(s' \mid s,a)\, v_\pi(s')}_{\text{mean future reward}}, \quad \forall s \in \mathscr{S}. \tag{4.3}$$

Introducing the shorthand $r_\pi(s) = \sum_a \pi(a \mid s) \sum_r p(r \mid s,a)\, r$ and $p_\pi(s' \mid s) = \sum_a \pi(a \mid s)\, p(s' \mid s,a)$, the Bellman equation admits the compact **matrix-vector form**:

$$v_\pi = r_\pi + \gamma P_\pi v_\pi, \tag{4.4}$$

where $v_\pi, r_\pi \in \mathbb{R}^{|\mathscr{S}|}$ and $[P_\pi]_{ij} = p_\pi(s_j \mid s_i)$. This linear system has the closed-form solution $v_\pi = (I - \gamma P_\pi)^{-1} r_\pi$, and can also be solved iteratively via $v_{k+1} = r_\pi + \gamma P_\pi v_k$, which converges to v_π for any initialization v_0.

4.2.3 Optimal State Values and the Bellman Optimality Equation

Definition 4.2.3 (Optimal Policy and Optimal State Value)**.** *A policy π^* is **optimal** if $v_{\pi^*}(s) \geq v_\pi(s)$ for all $s \in \mathcal{S}$ and all policies π. The corresponding values $v^* = v_{\pi^*}$ are the **optimal state values**.*

The **Bellman Optimality Equation** (BOE) characterizes v^*:

$$v(s) = \max_{\pi(s)\in\Pi(s)} \sum_{a\in\mathcal{A}} \pi(a \mid s)\, q(s,a), \quad q(s,a) \coloneqq \sum_r p(r \mid s,a)\, r + \gamma \sum_{s'} p(s' \mid s,a)\, v(s'), \tag{4.5}$$

or in matrix-vector form, $v = f(v) \coloneqq \max_{\pi\in\Pi}(r_\pi + \gamma P_\pi v)$. Since $\sum_a \pi(a \mid s) = 1$, the maximum in (4.5) is achieved by the deterministic greedy policy $\pi^*(a \mid s) = \mathbf{1}[a = \arg\max_{a'} q(s,a')]$.

Theorem 4.2.4 (Existence, Uniqueness, and Algorithm)**.** *The function $f(v)$ is a contraction mapping under $\|\cdot\|_\infty$ with constant $\gamma \in (0,1)$. Consequently, the BOE has a unique solution v^*, which can be computed by the iteration*

$$v_{k+1} = f(v_k) = \max_{\pi\in\Pi}(r_\pi + \gamma P_\pi v_k), \quad k = 0,1,2,\ldots, \tag{4.6}$$

converging exponentially fast from any initialization v_0. The optimal policy is then recovered as $\pi^* = \arg\max_\pi(r_\pi + \gamma P_\pi v^*)$, and satisfies $v^* = v_{\pi^*} \geq v_\pi$ for all π.

Remark 4.2.5. *Although v^* is unique, there may exist multiple optimal policies. However, Theorem 4.2.4 guarantees that a deterministic greedy policy is always optimal. Furthermore, optimal policies are invariant to affine reward transformations $r \mapsto \alpha r + \beta$ with $\alpha > 0$: the optimal state values transform as $v^* \mapsto \alpha v^* + \frac{\beta}{1-\gamma}\mathbf{1}$, leaving the greedy action ordering unchanged.*

4.2.4 Value Iteration, Policy Iteration, and Truncated Policy Iteration

When the model P and R are known, the BOE can be solved by dynamic programming. The three fundamental algorithms differ in how thoroughly they evaluate each policy before improving it.

Value iteration directly implements the BOE contraction, alternating between a one-step greedy policy update and a one-step value update:

$$\pi_{k+1} = \arg\max_\pi(r_\pi + \gamma P_\pi v_k), \qquad v_{k+1} = r_{\pi_{k+1}} + \gamma P_{\pi_{k+1}} v_k. \tag{4.7}$$

The intermediate values v_k are not true state values, but converge to v^* as $k \to \infty$.

Policy iteration fully evaluates each policy before improving it. Each iteration comprises:

1. **Policy evaluation (PE):** Solve $v_{\pi_k} = r_{\pi_k} + \gamma P_{\pi_k} v_{\pi_k}$ for v_{π_k}, either in closed form or by running the inner iteration $v^{(j+1)} = r_{\pi_k} + \gamma P_{\pi_k} v^{(j)}$ to convergence.
2. **Policy improvement (PI):** Set $\pi_{k+1} = \arg\max_\pi(r_\pi + \gamma P_\pi v_{\pi_k})$.

Since each improvement step satisfies $v_{\pi_{k+1}} \geq v_{\pi_k}$ (elementwise) and the sequence is bounded above by v^*, the state values converge monotonically to v^*.

Truncated policy iteration unifies the two: it runs the inner policy evaluation iteration for a finite number of steps j_{trunc} before improving the policy. Value iteration corresponds to $j_{\text{trunc}} = 1$, while policy iteration corresponds to $j_{\text{trunc}} = \infty$.

The iteration structures can be visualized as:

$$\text{Policy iteration:} \quad \pi_0 \xrightarrow{PE} v_{\pi_0} \xrightarrow{PI} \pi_1 \xrightarrow{PE} v_{\pi_1} \xrightarrow{PI} \cdots \tag{4.8}$$

$$\text{Value iteration:} \quad v_0 \xrightarrow{PU} \pi_1 \xrightarrow{VU} v_1 \xrightarrow{PU} \pi_2 \xrightarrow{VU} \cdots \tag{4.9}$$

4.3 Value Estimation Methods

In model-free RL, state and action values must be estimated from experience rather than computed from the dynamics. This section introduces Monte Carlo (MC) methods, Temporal-Difference (TD) methods, and their variants.

4.3.1 Monte Carlo Methods

MC methods estimate values by averaging sample returns over complete episodes, relying on the law of large numbers: for i.i.d. samples $\{x_i\}_{i=1}^n$ of a random variable X, $\frac{1}{n}\sum_i x_i \to \mathbb{E}[X]$ with variance shrinking as $1/n$.

Since $q_\pi(s,a) = \mathbb{E}[G_t \mid S_t = s, A_t = a]$, the action value can be estimated by averaging returns from episodes starting at (s,a) under π:

$$q_\pi(s,a) \approx \frac{1}{n}\sum_{i=1}^{n} g_\pi^{(i)}(s,a). \tag{4.10}$$

This replaces model-based policy evaluation in policy iteration, yielding a fully model-free algorithm. Two visit strategies are commonly used: **first-visit**, which uses only the first occurrence of each (s,a) per episode, and **every-visit**, which uses all occurrences and is more sample-efficient.

To ensure adequate exploration, an **ε-greedy policy** is used:

$$\pi(a \mid s) = \begin{cases} 1 - \dfrac{\varepsilon(|\mathscr{A}(s)|-1)}{|\mathscr{A}(s)|}, & a = \arg\max_{a'} q(s,a'), \\ \dfrac{\varepsilon}{|\mathscr{A}(s)|}, & \text{otherwise}, \end{cases} \tag{4.11}$$

which selects the greedy action with high probability while maintaining a minimum exploration probability $\varepsilon/|\mathscr{A}(s)|$ for all actions.

4.3.2 Temporal-Difference Methods

TD methods combine MC sampling with dynamic programming bootstrapping: instead of waiting for the complete return, they update estimates using the current value estimate at the next state.

TD Learning of State Values

Given experience (s_t, r_{t+1}, s_{t+1}) generated under π, the TD update is

$$v_{t+1}(s_t) = v_t(s_t) - \alpha_t(s_t)\underbrace{\left[v_t(s_t) - (r_{t+1} + \gamma v_t(s_{t+1}))\right]}_{\text{TD error } \delta_t}, \tag{4.12}$$

where $\bar{v}_t = r_{t+1} + \gamma v_t(s_{t+1})$ is the **TD target** and $\delta_t = v_t(s_t) - \bar{v}_t$ is the **TD error**. This is a stochastic approximation of the Bellman equation $v_\pi(s) = \mathbb{E}[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s]$, and $v_t(s) \to v_\pi(s)$ almost surely provided $\sum_t \alpha_t(s) = \infty$ and $\sum_t \alpha_t^2(s) < \infty$.

Sarsa: On-Policy TD Control

The **Sarsa** algorithm extends TD to action values using the tuple $(s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1})$:

$$q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha_t(s_t, a_t)\left[q_t(s_t, a_t) - (r_{t+1} + \gamma q_t(s_{t+1}, a_{t+1}))\right]. \tag{4.13}$$

Sarsa is **on-policy**: the action a_{t+1} is drawn from the same policy being evaluated, so the behavior and target policies coincide. Under the same step-size conditions as TD, $q_t(s,a) \to q_\pi(s,a)$ almost surely.

n-Step Sarsa

The n-step return $G_t^{(n)} = R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n q_\pi(S_{t+n}, A_{t+n})$ unifies Sarsa ($n = 1$) and Monte Carlo ($n = \infty$). The n-step Sarsa update uses $G_t^{(n)}$ as the target, interpolating between the low-variance but biased estimates of TD and the unbiased but high-variance estimates of MC.

Q-Learning: Off-Policy TD Control

Q-learning is an **off-policy** algorithm that estimates optimal action values directly, regardless of the behavior policy:

$$q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha_t(s_t, a_t)\left[q_t(s_t, a_t) - \left(r_{t+1} + \gamma \max_{a'} q_t(s_{t+1}, a')\right)\right]. \tag{4.14}$$

The target $r_{t+1} + \gamma \max_{a'} q_t(s_{t+1}, a')$ is a stochastic approximation of the Bellman optimality equation for action values, $q^*(s,a) = \mathbb{E}[R_{t+1} + \gamma \max_{a'} q^*(S_{t+1}, a') \mid S_t = s, A_t = a]$, which is equivalent to the BOE derived in Section 4.2. Because the update uses $\max_{a'}$ rather than the action drawn from the behavior policy, Q-learning can learn the optimal policy while following any exploratory behavior policy.

4.3.3 Comparison of MC and TD Methods

TD Methods	**MC Methods**
Online updates after each step	Requires complete episodes
Handles episodic and continuing tasks	Episodic tasks only
Bootstraps from current estimates (biased)	Directly uses sampled returns (unbiased)
Lower variance	Higher variance

Table 4.1: Comparison of TD and MC methods.

4.4 Hierarchical Reinforcement Learning

Hierarchical Reinforcement Learning (HRL) extends the classical reinforcement learning framework by introducing temporal and structural abstraction into the decision-making process. Instead of learning a single policy that operates at a fixed time scale, HRL decomposes a complex task into multiple levels of control, where higher-level policies make abstract decisions and lower-level policies execute concrete actions. This decomposition is particularly effective for long-horizon tasks with sparse rewards and large action spaces.

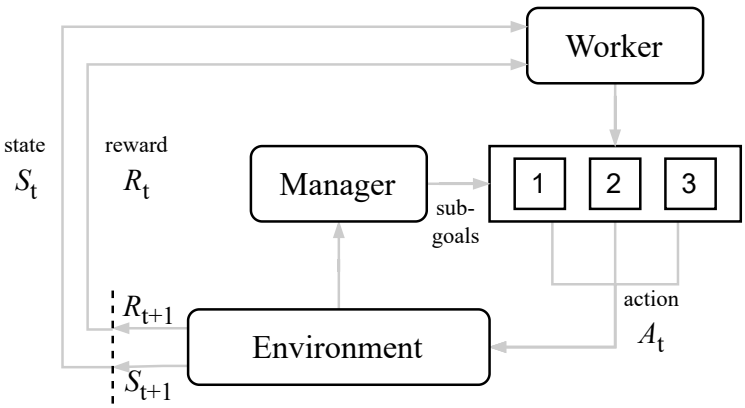


Figure 4.2: Illustration of the hierarchical reinforcement learning framework. A high-level *Manager* observes the environment state S_t and issues sub-goals to a low-level *Worker*. The Worker executes primitive actions A_t to interact with the environment. In response, the environment transitions to the next state S_{t+1} and returns rewards R_t and R_{t+1}, which are used to guide learning at different levels of the hierarchy.

4.4.1 Temporal Abstraction

Definition 4.4.1 (Temporal abstraction)**.** Temporal abstraction *refers to the ability of an agent to reason and act over extended time intervals by grouping multiple primitive actions into higher-level decisions. Formally, a temporally abstract action may correspond to a sequence of primitive actions executed over several time steps.*

Temporal abstraction enables the agent to operate at multiple time scales, allowing high-level decisions to guide behavior over longer horizons while low-level policies handle fine-grained control.

4.4.2 Hierarchical Policies

Definition 4.4.2 (Hierarchical policy)**.** *A* hierarchical policy *consists of multiple interacting policies organized across different levels of abstraction. Typically, a high-level policy selects abstract decisions or subgoals, while one or more low-level policies determine the primitive actions required to achieve these subgoals.*

Let π_h denote a high-level policy and π_ℓ denote a low-level policy. At selected time steps, the high-level policy produces an abstract command z_t based on the current state:

$$z_t \sim \pi_h(\cdot \mid s_t).$$

Conditioned on z_t, the low-level policy selects primitive actions:

$$a_t \sim \pi_\ell(\cdot \mid s_t, z_t).$$

This hierarchical structure allows the agent to decompose complex decision-making into manageable subproblems.

4.4.3 Subgoals and Options

Definition 4.4.3 (Subgoal)**.** *A* subgoal *is an intermediate objective that specifies a desirable state, state property, or latent target that guides the agent's behavior over a finite time horizon.*

Subgoals may be defined explicitly in the state space or implicitly in a latent representation space. Once a subgoal is selected, the agent executes actions until the subgoal is achieved or a termination condition is met.

Definition 4.4.4 (Option)**.** *An* option *is a temporally extended course of action defined by a triple* $(\mathscr{I}, \pi, \beta)$*, where* $\mathscr{I} \subseteq \mathscr{S}$ *is the initiation set,* π *is an intra-option policy, and* $\beta : \mathscr{S} \to [0,1]$ *is a termination function.*

The options framework provides a formal foundation for HRL by modeling abstract actions that span multiple time steps.

4.4.4 Hierarchical Markov Decision Process

Definition 4.4.5 (Hierarchical Markov Decision Process)**.** *A* Hierarchical Markov Decision Process *(HMDP) is an extension of the standard MDP in which actions may correspond to temporally extended behaviors, such as options or subgoal-conditioned policies, rather than single-step primitive actions.*

In an HMDP, the transition and reward dynamics are induced by the execution of higher-level decisions over multiple environment steps. This formulation preserves the Markov property at the level of abstract actions, enabling principled learning and analysis.

4.4.5 Advantages of Hierarchical Reinforcement Learning

Hierarchical reinforcement learning offers several key advantages over flat reinforcement learning:

- *Improved sample efficiency*, by reducing the effective planning horizon.
- *Better exploration*, through structured, goal-directed behavior.
- *Scalability*, by decomposing large decision spaces into smaller subproblems.
- *Enhanced credit assignment*, by providing intermediate learning signals through subgoal completion.

These properties make HRL particularly suitable for complex reasoning tasks, such as multi-hop decision-making and long-term planning in structured environments.

Chapter 5

Proposed Solutions

This chapter describes the proposed methodologies - a hierarchical reinforcement learning framework with abstract meaning representation injected - developed to address the problem considered in this thesis.

5.1 Problem Definition

A knowledge graph is $\mathscr{G} = (\mathscr{E}, \mathscr{R}, \mathscr{T})$, where $\mathscr{E}$ is the entity set, $\mathscr{R}$ is the relation set, and $\mathscr{T} \subseteq \mathscr{E} \times \mathscr{R} \times \mathscr{E}$ denotes factual triples. Given question q and knowledge graph $\mathscr{G}$, multi-hop QA identifies the answer entity $e^* \in \mathscr{E}$ by traversing a reasoning path $\pi = \{e_0, r_1, e_1, \ldots, r_T, e_T\}$ from source entity e_0, where $e_T = e^*$ and path length is bounded by $T_{\max}$.

5.2 NeuroPath: Neural Policy for Knowledge Graph Reasoning

5.2.1 Framework Overview

Our framework is designed to achieve strong multi-hop reasoning performance even under limited training data. Given a natural language question, the system follows a four-stage pipeline. First, Named Entity Recognition (NER) is applied to extract the entities mentioned in the question, which serve as the initial anchor points for path retrieval. Second, relational triplets are extracted from the provided context and organized into a temporary knowledge graph capturing the structured representation of the contextual information. Third, the knowledge graph is encoded using a Relational Graph Attention Network (Section 5.2.2), enabling each node to incorporate relation-aware information from its neighborhood. Fourth, multi-hop path retrieval is performed on the embedded graph using the neural policy described in Section 5.2.3. Lightweight language models are employed for the first two steps; the core contributions of this work lie in the graph encoding and reasoning components.

The overall retrieval mechanism is illustrated in Figure 5.1. The input question is encoded by a pretrained encoder to produce an initial query representation q^0. At each hop t, the embedded knowledge graph is processed together with the current query state q^t by the policy block, which outputs the next candidate node j^* and updates the query state to q^{t+1}. This iterative procedure is repeated until either no unvisited neighbors remain or the stopping module signals

termination.

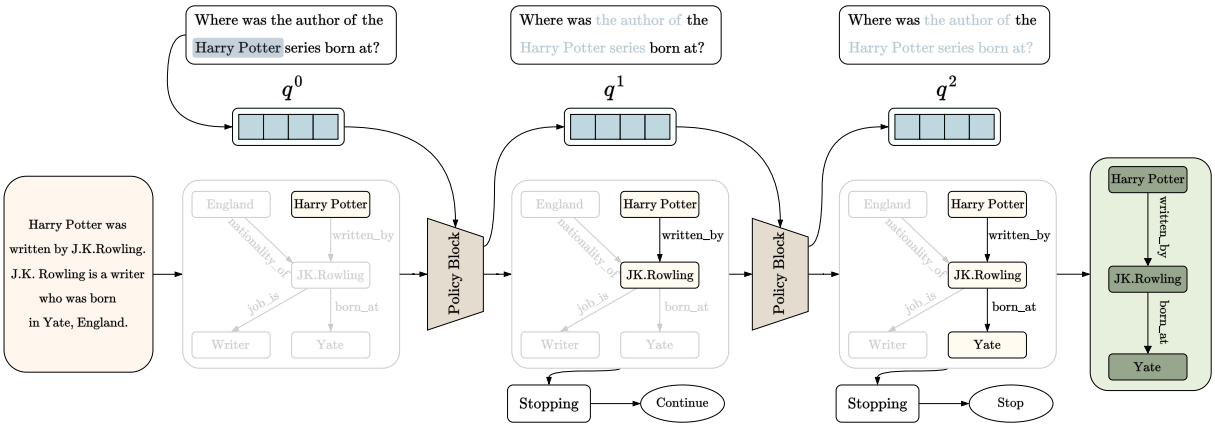


Figure 5.1: The retrieval mechanism of our model. The input question is encoded by a pre-trained encoder to produce the initial representation q^0. At each hop t, the embedded knowledge graph is processed together with the current query representation q^t by the policy block, which outputs the next candidate node j^* and updates the query state to q^{t+1}. This iterative procedure repeats until no unvisited neighbors remain or the stopping module signals termination.

5.2.2 Relational Graph Attention Network

We propose the use of a Relational Graph Attention Network (RGAT) as the encoding mechanism for the knowledge graph, extending the original Graph Attention Network (GAT) formulation [32] to handle graphs with labeled edges. Multi-head attention is incorporated to allow the model to attend to diverse aspects of neighboring nodes and relations simultaneously, improving the expressiveness of the resulting node representations.

Knowledge Graph. Given a set of entities $\mathscr{E}$, a set of relations $\mathscr{R}$, and a set of relational triplets $\mathscr{T} = \{(e_i, r_j, e_k)\} \subseteq \mathscr{E} \times \mathscr{R} \times \mathscr{E}$, we define a knowledge graph as $\mathscr{G} = (\mathscr{E}, \mathscr{R}, \mathscr{T})$.

RGAT Embedding. Each entity $e_i \in \mathscr{E}$ is associated with a node feature $h_i \in \mathbb{R}^D$, forming the feature set $\mathbf{h} = \{h_1, h_2, \ldots, h_{|\mathscr{E}|}\}$. Each triplet $(e_i, r_{ij}, e_j) \in \mathscr{T}$ is assigned a relation embedding $r_{ij} \in \mathbb{R}^D$, collected into $\mathbf{r} = \{r_{ij} \mid (e_i, r_{ij}, e_j) \in \mathscr{T}\}$. The objective of the RGAT layer is to derive updated node features $\mathbf{h}' = \{h'_1, \ldots, h'_{|\mathscr{E}|}\}$ that integrate both neighbor and relational context.

For each triplet (e_i, r_{ij}, e_j), we treat e_i as the *key*, e_j as the *value*, and r_{ij} as the *query*, following the attention formulation of [31]. These are projected into a shared hidden space of dimension F via learned matrices $W_k, W_q, W_v \in \mathbb{R}^{F \times D}$:

$$k_i = W_k h_i, \quad q_{ij} = W_q r_{ij}, \quad v_j = W_v h_j. \tag{5.1}$$

An unnormalized attention score for each neighboring entity $e_j \in \mathscr{N}_i$ is then computed by a single-layer feedforward network parameterized by $a \in \mathbb{R}^{3F}$, activated with LeakyReLU (slope 0.2) and scaled by $\sqrt{D}$:

$$\alpha_{ij} = \text{LeakyReLU}\left(\frac{a^\top \left[k_i \,\|\, q_{ij} \,\|\, v_j\right]}{\sqrt{D}}\right), \tag{5.2}$$

where $\|$ denotes concatenation. Masked attention ensures that α_{ij} is computed only for $e_j \in \mathcal{N}_i$, preserving the graph structure. The scores are normalized across neighbors via softmax:

$$\beta_{ij} = \text{softmax}_j(\alpha_{ij}) = \frac{\exp(\alpha_{ij})}{\sum_{e_k \in \mathcal{N}_i} \exp(\alpha_{ik})}, \tag{5.3}$$

and the updated node representation is computed as a weighted aggregation of neighbor features:

$$h'_i = \text{ReLU}\left(\sum_{e_j \in \mathcal{N}_i} \beta_{ij} h_j \right). \tag{5.4}$$

To improve stability, the mechanism is extended to $K = D/F$ attention heads. The outputs of each head are concatenated to form the final updated representation:

$$h'_i = \Big\Vert_{k=1}^{K} \text{ReLU}\left(\sum_{e_j \in \mathcal{N}_i} \beta_{ij}^k h_j \right), \tag{5.5}$$

where β_{ij}^k denotes the normalized attention coefficient from the k-th head. The architecture is illustrated in Figure 5.2.

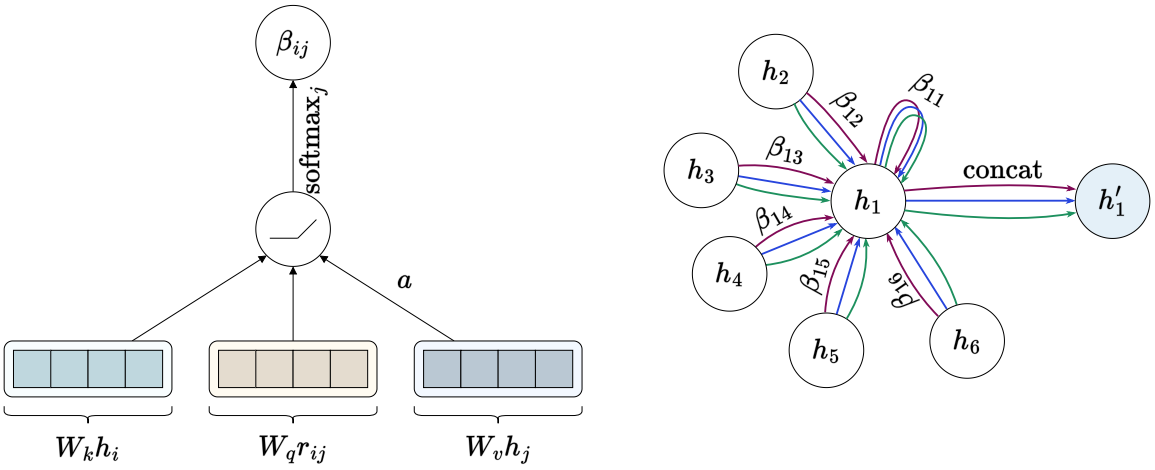


Figure 5.2: **Left.** The attention mechanism $a^\top [W_k h_i \| W_q r_{ij} \| W_v h_j]$ employed by our model, parameterized by $a \in \mathbb{R}^{3F}$ and followed by a LeakyReLU activation. **Right.** Multi-head attention with $K = 3$ heads applied by node 1 to its neighborhood. Different arrow styles denote independent attention computations; features from all heads are concatenated to obtain h'_1.

5.2.3 Path Retrieval on the Knowledge Graph

We introduce a neural method for multi-hop retrieval over the RGAT-embedded knowledge graph. At each hop t, the model is positioned at node e_i with a query representation $q^t \in \mathbb{R}^D$, initialized as the embedding of the input question. Let $\mathscr{C}_i^t \subseteq \mathcal{N}_i$ denote the set of unvisited neighbors of e_i at hop t. For each candidate $e_j \in \mathscr{C}_i^t$, a feature vector is constructed by concatenating the current node embedding, the query state, and the candidate embedding:

$$x_{ij} = \left[h_i \,\|\, q^t \,\|\, h_j \right] \in \mathbb{R}^{3D}. \tag{5.6}$$

The inclusion of q^t ensures that each navigation decision is conditioned not only on the local graph structure but also on the full traversal history, as encoded by the LSTM. The feature

vector is passed through a feedforward network with layer normalization to produce a scalar score, from which a categorical distribution over candidates is derived:

$$z_{ij} = \text{LayerNorm}\big(\text{ReLU}(W_1 x_{ij} + b_1)\big), \tag{5.7}$$

$$s_{ij} = W_2 z_{ij} + b_2, \tag{5.8}$$

$$\pi_\theta(a_t = j) = \frac{\exp(s_{ij})}{\sum_{j' \in \mathscr{C}_i^t} \exp(s_{ij'})}. \tag{5.9}$$

We interpret π_θ as a stochastic policy over the action space $\mathscr{C}_i^t$, analogous to a policy network in reinforcement learning. During training, the next node is selected by sampling from this distribution to encourage exploration:

$$j^* \sim \text{Categorical}\Big(\{\pi_\theta(a_t = j)\}_{j \in \mathscr{C}_i^t}\Big). \tag{5.10}$$

At inference, greedy selection ($\arg\max_j s_{ij}$) or beam search can be applied for more stable path generation.

Once the next node e_{j^*} is selected, the query embedding is updated via an LSTM to integrate new information while retaining relevant long-term context:

$$q^{t+1}, c^{t+1} = \text{LSTM}\big(h_{j^*}, (q^t, c^t)\big), \tag{5.11}$$

where c^t is the LSTM cell state, initialized to zero. This mechanism enables the model to progressively gather relevant information along the traversal path while suppressing redundant signals through the LSTM's gating structure. The retrieval procedure is illustrated in Figure 5.3.

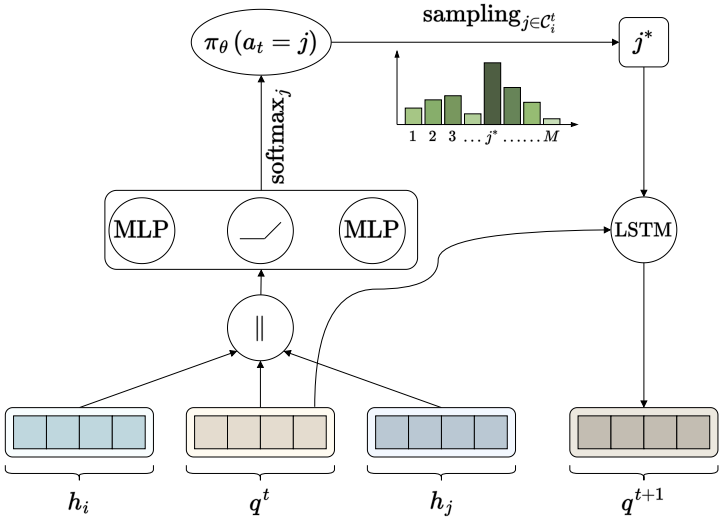


Figure 5.3: An overview of the retrieval model. At hop t, the model is at node e_i with query representation q^t. A probability distribution $\pi_\theta(a_t = j)$ is computed over unvisited neighbors; the selected node's embedding is integrated with q^t via an LSTM update to produce q^{t+1} for the next hop.

5.2.4 Stopping Module

A fundamental challenge in path-based exploration is determining when to terminate the search. We introduce a dedicated neural stopping module that predicts whether the accumulated evidence along the explored path is sufficient to answer the query, rather than relying on a fixed maximum hop count.

The core intuition is to assess whether the nodes visited up to hop t collectively provide adequate coverage of the query semantics. Given the set of visited nodes $F_t = \{e_0, \ldots, e_t\}$, we compute a query-conditioned summary of the explored subgraph via attention. The query and visited node embeddings are projected into a shared latent space:

$$q' = W_Q q, \qquad f_i = W_K h_i \quad (e_i \in F_t). \tag{5.12}$$

Unnormalized attention scores are computed via dot product and normalized using softmax:

$$s_i = f_i^\top q', \qquad \alpha_i = \frac{\exp(s_i)}{\sum_{j=0}^{t} \exp(s_j)}. \tag{5.13}$$

The coefficients α_i assign higher weights to nodes most relevant to the current query, yielding a query-conditioned context vector:

$$\tilde{h}_t = \sum_{i=0}^{t} \alpha_i h_i, \tag{5.14}$$

which encapsulates the accumulated evidence up to hop t. To jointly capture query intent and contextual evidence, $\tilde{h}_t$ and the query embedding q are concatenated and projected through a feedforward layer:

$$u_t = \left[\tilde{h}_t \parallel q\right], \qquad s_t = \mathrm{LayerNorm}(\mathrm{ReLU}(W_s u_t + b_s)). \tag{5.15}$$

From this shared representation, two scalar outputs are derived:

$$V_t = \sigma(w_V^\top s_t + b_V), \qquad c_t = \tanh(w_c^\top s_t + b_c). \tag{5.16}$$

The *value head* V_t estimates the expected long-term return, while the *commit head* c_t produces a termination score. This dual-head design mirrors the actor-critic framework, providing a stable training signal that balances exploration and termination. A learnable threshold parameter ε determines the stopping condition:

$$\ell_t = c_t - \varepsilon, \qquad p_t = \sigma(\ell_t), \qquad \mathsf{Stop} = \mathbb{I}(p_t \geq 0.5). \tag{5.17}$$

Since $\sigma(x) \geq 0.5$ if and only if $x \geq 0$, gradients propagate smoothly through this criterion, enabling end-to-end differentiable optimization of the stopping decision.

5.2.5 Training Objectives

The model is optimized via a joint loss that supervises both the path retrieval module and the stopping verifier, ensuring that the system retrieves accurate paths while making reliable termination decisions.

Query Module Loss.

The query module is trained to encourage step-wise accuracy, semantic separation between correct and incorrect candidates, and consistency of predicted path length with the ground truth. Let $\{h_t^{\text{pred}}\}_{t=1}^{T_{\text{pred}}}$ and $\{h_t^{\text{true}}\}_{t=1}^{T_{\text{true}}}$ denote the predicted and ground-truth path embeddings respectively, and let $\{h_i\}_{i=1}^{|\mathscr{E}|}$ be all node embeddings. The query loss is:

$$\mathscr{L}_{\text{query}} = \frac{1}{T} \sum_{t=1}^{T} \left[\alpha \left\| h_t^{\text{pred}} - h_t^{\text{true}} \right\|_2^2 + \beta \max\left(0,\, m + d_{\text{pos}} - d_{\text{neg}}^{\min}\right) \right] + \gamma |T_{\text{pred}} - T_{\text{true}}|, \tag{5.18}$$

where $T = \min(T_{\text{pred}}, T_{\text{true}})$, $d_{\text{pos}} = \|h_t^{\text{pred}} - h_t^{\text{true}}\|_2$, $d_{\text{neg}}^{\min} = \min_{j \neq t} \|h_j - h_t^{\text{pred}}\|_2$, and α, β, γ are balancing weights with m a margin hyperparameter. The first term is a mean squared error loss that directly penalizes deviations from the ground-truth trajectory. The second term is a contrastive loss that pushes predicted embeddings away from negative candidates while keeping them close to the positive ground-truth node, improving semantic discrimination. The third term penalizes discrepancies in path length, guiding the retrieval module to produce trajectories of the correct depth.

Verifier Loss.

The stopping verifier is trained with two complementary objectives: a *value loss* that regresses discounted returns from each step, and a *commit loss* that classifies the stopping step using focal binary cross-entropy.

The discounted return at step t is $G_t = \sum_{k=t}^{L-1} \gamma_v^{k-t} r_k$, where L is the maximum number of hops and γ_v is the discount factor. The value loss is:

$$\mathscr{L}_V = \frac{1}{N} \sum_t (V_t - G_t)^2 . \tag{5.19}$$

For the commit head, let $y_t \in \{0,1\}$ be the stopping label. Since each training trajectory with n hops yields $n-1$ negative samples ($y_t = 0$) and only one positive ($y_t = 1$), we apply focal binary cross-entropy to address this severe class imbalance:

$$\mathscr{L}_{\text{focal}} = \frac{1}{N} \sum_t \alpha (1 - p_t^*)^{\gamma_c} \left[-y_t \log p_t - (1 - y_t) \log(1 - p_t) \right], \tag{5.20}$$

where $p_t^* = y_t p_t + (1 - y_t)(1 - p_t)$. The class-balancing factor α is set based on the observed imbalance ratio in the dataset (we use $\alpha = 1/5$), and $\gamma_c \geq 0$ modulates the relative weight of hard versus easy examples. The total verifier loss is:

$$\mathscr{L}_{\text{verifier}} = \lambda_V \mathscr{L}_V + \lambda_c \mathscr{L}_{\text{focal}}. \tag{5.21}$$

The full model is trained end-to-end by jointly minimizing $\mathscr{L}_{\text{query}}$ and $\mathscr{L}_{\text{verifier}}$, ensuring that path retrieval accuracy and termination reliability are optimized in a mutually consistent manner.

5.3 SymR: Symbolic Reinforcement for Multi-Hop Knowledge Graph Reasoning

5.3.1 Framework Overview

We propose **SymR** (Symbolic Reinforcement), a hierarchical reinforcement learning framework for multi-hop reasoning over large-scale knowledge graphs. The central challenge motivating SymR is long-horizon relational reasoning: given a natural language question q, an agent must traverse a sequence of intermediate entities in a knowledge graph $\mathscr{G} = (\mathscr{E}, \mathscr{R}, \mathscr{T})$ to arrive at a final answer entity $e^* \in \mathscr{E}$. This problem is inherently difficult due to combinatorially large action spaces, sparse terminal rewards, and the need to maintain semantic coherence throughout the reasoning chain.

SymR addresses these challenges by decomposing the reasoning task into two interacting levels of abstraction, motivated by the Options Framework in hierarchical reinforcement learning. A high-level *manager* operates at a coarser temporal scale and sequences semantic subgoals derived from the symbolic structure of the input question. A low-level *worker* takes fine-grained navigation steps within the knowledge graph, conditioned on the active subgoal provided by the manager. A key innovation of SymR is grounding these subgoals in Abstract Meaning Representation (AMR) graphs parsed from the question, providing interpretable intermediate reasoning targets that exploit the semantic structure of the query as a symbolic inductive bias.

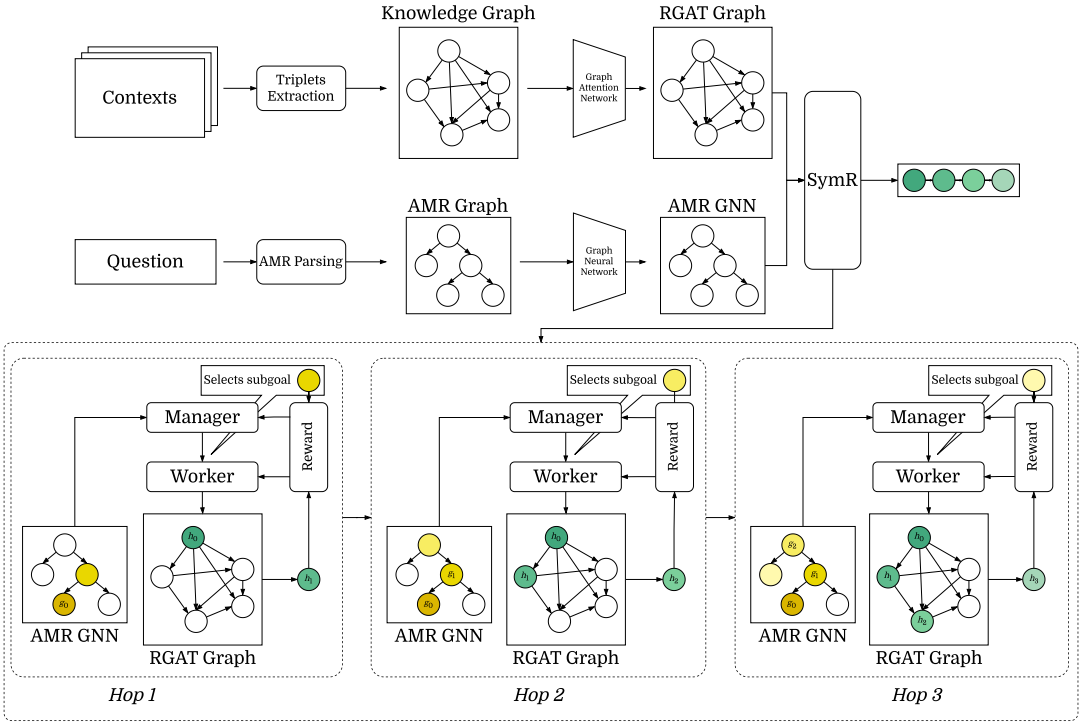


Figure 5.4: High-level architecture of SymR for multi-hop question answering on knowledge graphs. The question is parsed into an AMR graph and encoded by an AMR GCN to provide symbolic semantic guidance, while the knowledge graph is encoded using a relational graph attention network. A hierarchical reinforcement learning framework integrates both representations: the manager selects AMR-grounded symbolic subgoals, and the worker navigates the knowledge graph to execute them across multiple hops.

System Pipeline. Given a natural language question q, SymR operates as follows. First, q is parsed into an AMR graph $\mathscr{A} = (\mathscr{C}, \mathscr{E}_{\mathscr{A}}, \mathscr{L})$, which is subsequently encoded by a graph convolutional network into dense concept embeddings. Second, the background knowledge graph $\mathscr{G}$ is encoded by a Relational Graph Attention Network (RGAT), producing relation-aware entity representations. Third, the manager selects a semantic subgoal g_m from the AMR node set every K worker steps, where K is a fixed temporal abstraction parameter. The subgoal embedding $\mathbf{g}_m \in \mathbb{R}^d$ is communicated to the worker, which navigates $\mathscr{G}$ for the next K steps with the objective of satisfying that subgoal. This interleaving of coarse-grained semantic planning and fine-grained graph traversal constitutes the core of SymR's hierarchical reasoning mechanism, and is illustrated in Figure 5.5.

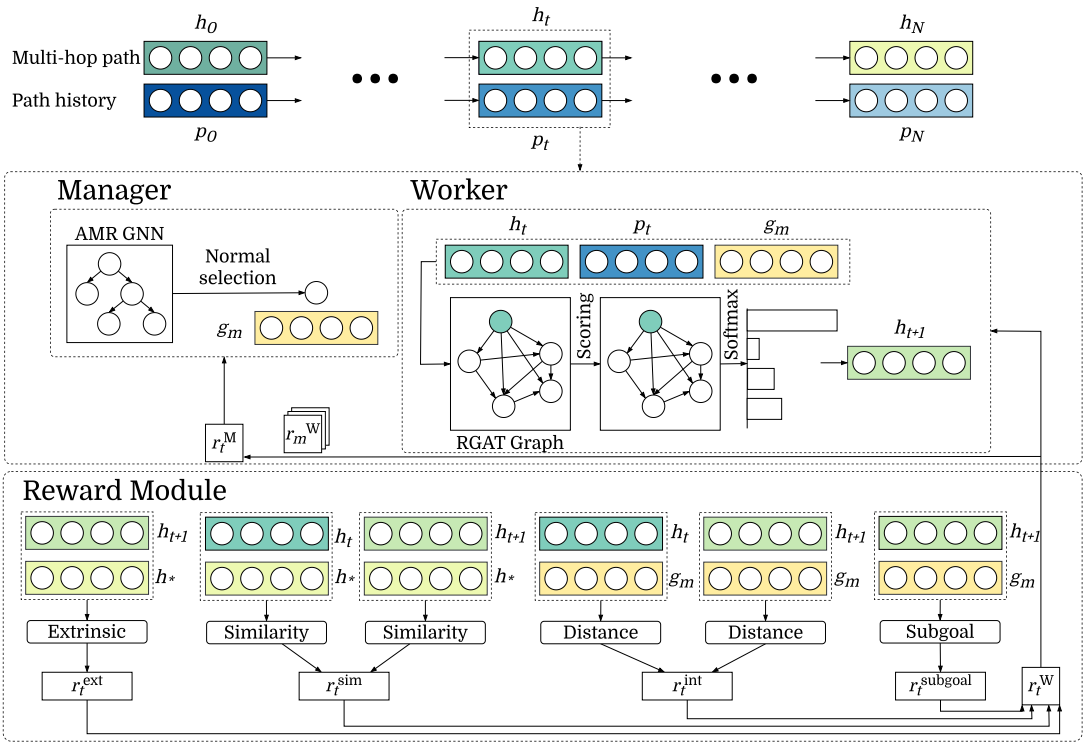


Figure 5.5: Architecture of SymR. The manager selects AMR-based semantic subgoals, while the worker navigates the knowledge graph using RGAT-based entity representations. A hierarchical reward combines extrinsic, intrinsic, and subgoal-oriented signals.

5.3.2 AMR Graph Encoding

Abstract Meaning Representation is a formalism for capturing the propositional content of a natural language sentence as a rooted, directed, labeled graph. Given an input question q, an AMR parser produces $\mathscr{A} = (\mathscr{C}, \mathscr{E}_{\mathscr{A}}, \mathscr{L})$, where $\mathscr{C} = \{c_1, \ldots, c_n\}$ is the set of concept nodes (named entities, predicate frames, semantic constants), $\mathscr{E}_{\mathscr{A}} \subseteq \mathscr{C} \times \mathscr{C}$ is the set of directed edges encoding semantic role relations (e.g., `:ARG0`, `:ARG1`, `:mod`), and $\mathscr{L}$ is the inventory of semantic role labels. Unlike syntactic parse trees, AMR abstracts away from surface word order and morphological variation, providing a canonicalized meaning representation that is well-suited for symbolic grounding.

Concept Initialization. Each concept $c_i \in \mathscr{C}$ is initialized to an embedding $\mathbf{x}_i^{(0)} \in \mathbb{R}^d$ by a pretrained BERT encoder $\mathscr{F}_{\text{enc}}$:

$$\mathbf{x}_i^{(0)} = \mathscr{F}_{\text{enc}}(c_i), \tag{5.22}$$

where c_i is treated as a short text string and $\mathscr{F}_{\text{enc}}$ extracts the `[CLS]` token representation. This endows each concept node with rich semantic content from large-scale pretraining before any graph-level aggregation is performed.

Graph Convolutional Encoding. To propagate relational information across $\mathscr{A}$, SymR applies a two-layer Graph Convolutional Network (GCN). Let $\tilde{\mathbf{A}} = \tilde{\mathbf{D}}^{-1/2}(\mathbf{A} + \mathbf{I}_n)\tilde{\mathbf{D}}^{-1/2}$ denote the symmetrically normalized adjacency matrix with added self-loops, where $\tilde{\mathbf{D}}_{ii} = \sum_j (\mathbf{A} + \mathbf{I}_n)_{ij}$. The two-layer update is:

$$\mathbf{H}^{(1)} = \text{ReLU}\left(\tilde{\mathbf{A}}\,\mathbf{X}^{(0)}\mathbf{W}_1\right), \qquad \mathbf{H}^{(2)} = \tilde{\mathbf{A}}\,\mathbf{H}^{(1)}\mathbf{W}_2, \tag{5.23}$$

where $\mathbf{X}^{(0)} \in \mathbb{R}^{n \times d}$ stacks all initial concept embeddings, and $\mathbf{W}_1, \mathbf{W}_2 \in \mathbb{R}^{d \times d}$ are learnable weight matrices. The first layer aggregates one-hop neighborhood information with a nonlinear activation; the second layer extends the receptive field to two hops with a linear output to preserve gradient magnitude. The final AMR concept representations $\mathbf{H}^{(2)} \in \mathbb{R}^{n \times d}$ serve as the subgoal embedding pool for the manager.

5.3.3 Hierarchical Policy Architecture

Path Encoding

Knowledge graph reasoning is inherently sequential: the relevance of a candidate next entity at step t depends not only on the current entity e_t but also on the full traversal history $\langle e_0, e_1, \ldots, e_t \rangle$. Without such memory, the agent cannot distinguish states that share the same current entity but differ in their traversal history, nor can it detect cycles. SymR addresses this by maintaining a compact sequential encoding of the worker's trajectory, termed the *path context* $\mathbf{p}_t \in \mathbb{R}^d$, produced by a GRU:

$$\mathbf{p}_t = \mathrm{GRU}(\mathbf{h}_t,\ \mathbf{p}_{t-1}), \tag{5.24}$$

with $\mathbf{p}_0 = \mathbf{0}$ at the start of each episode. Expanding the GRU recurrence, the update and reset gates govern how information is selectively retained and discarded:

$$\mathbf{z}_t = \sigma(\mathbf{W}_z \mathbf{h}_t + \mathbf{U}_z \mathbf{p}_{t-1} + \mathbf{b}_z)\,, \tag{5.25}$$

$$\mathbf{r}_t = \sigma(\mathbf{W}_r \mathbf{h}_t + \mathbf{U}_r \mathbf{p}_{t-1} + \mathbf{b}_r)\,, \tag{5.26}$$

$$\tilde{\mathbf{p}}_t = \tanh(\mathbf{W}_h \mathbf{h}_t + \mathbf{U}_h(\mathbf{r}_t \odot \mathbf{p}_{t-1}) + \mathbf{b}_h)\,, \tag{5.27}$$

$$\mathbf{p}_t = (1 - \mathbf{z}_t) \odot \mathbf{p}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{p}}_t\,, \tag{5.28}$$

where $\sigma(\cdot)$ is the sigmoid function and $\odot$ denotes element-wise multiplication. The update gate $\mathbf{z}_t$ controls how much of the previous context is carried forward; the reset gate $\mathbf{r}_t$ modulates how much prior history informs the candidate update $\tilde{\mathbf{p}}_t$. Crucially, the GRU's gating mechanism allows it to suppress the influence of uninformative early-hop entities - common in multi-hop reasoning, where initial hops frequently traverse generic hub nodes before reaching semantically specific intermediates - while retaining the most salient features of the traversal. The path context $\mathbf{p}_t$ is shared across both the worker and manager state representations, providing each level with a consistent view of the trajectory history.

Manager Policy

The manager is the high-level decision-making component of SymR, responsible for sequencing semantic subgoals from the AMR graph. It operates at a coarser temporal granularity than the worker, making a new decision every K worker steps. At worker step t, the manager step index is $m = \lfloor t/K \rfloor$, and a subgoal selected at step m remains active throughout worker steps $t \in [mK, (m+1)K)$.

Manager State. At manager step m (equivalently, worker step $t = mK$), the manager's state is a composite representation:

$$\mathbf{s}_m^M = \left[\mathbf{h}_t;\ \mathbf{p}_t;\ \mathbf{e}_{\text{target}}\right] \in \mathbb{R}^{3d}, \tag{5.29}$$

where $\mathbf{h}_t$ is the RGAT-encoded embedding of the entity currently occupied by the worker, $\mathbf{p}_t$ is the GRU path context summarizing the traversal history, and $\mathbf{e}_{\text{target}}$ is a fixed embedding of the expected answer type that anchors subgoal selection toward the final reasoning objective.

Subgoal Selection. A key design decision in SymR is that the manager does not learn an explicit stochastic policy over a latent subgoal space. Instead, subgoals are selected deterministically from the discrete node set $\mathscr{C}$ of the AMR graph, grounding manager decisions in the symbolic structure of the question and avoiding the instability associated with learning abstract latent subgoal representations.

At the first manager step ($m = 0$), the subgoal is initialized to the AMR concept most similar to the starting entity embedding:

$$g_0 = \underset{v_i \in \mathscr{C}}{\operatorname{argmax}} \ \operatorname{sim}\Big(\mathbf{h}_0,\ \mathbf{H}^{(2)}[i]\Big), \qquad \operatorname{sim}(\mathbf{u},\mathbf{v}) = \frac{\mathbf{u}^\top \mathbf{v}}{\|\mathbf{u}\| \cdot \|\mathbf{v}\|}. \tag{5.30}$$

For all subsequent steps ($m > 0$), the subgoal advances by a one-step random walk over the AMR graph from the previously selected node:

$$g_m \sim \operatorname{Uniform}(\mathscr{N}_{\mathscr{A}}(g_{m-1})), \tag{5.31}$$

where $\mathscr{N}_{\mathscr{A}}(g_{m-1})$ denotes the AMR neighbors of g_{m-1}. If g_{m-1} is a leaf node with no outgoing neighbors, the subgoal is retained: $g_m = g_{m-1}$. This walk-based progression ensures that consecutive subgoals are semantically related (since adjacent AMR nodes are connected by semantic role edges) while introducing stochasticity that encourages exploration of diverse reasoning paths during training. The active subgoal embedding communicated to the worker is retrieved directly from the GCN output:

$$\mathbf{g}_m = \mathbf{H}^{(2)}[g_m] \in \mathbb{R}^d. \tag{5.32}$$

Manager Value Function. Although subgoal selection is non-parametric, the manager maintains a value function $V_M : \mathbf{s}_m^M \mapsto \mathbb{R}$ to support advantage estimation during training. It is implemented as a two-layer MLP:

$$V_M(\mathbf{s}_m^M;\, \theta_M^v) = \mathbf{W}_2^v \operatorname{ReLU}\big(\mathbf{W}_1^v \mathbf{s}_m^M + \mathbf{b}_1^v\big) + \mathbf{b}_2^v. \tag{5.33}$$

Worker Policy

The worker is the low-level navigation agent. At each discrete time step t, it occupies a single entity $e_t \in \mathscr{E}$ and selects an adjacent entity to transition to, conditioned on the active subgoal $\mathbf{g}_m$ provided by the manager.

Worker State. The worker's state is:

$$\mathbf{s}_t^W = \big[\mathbf{h}_t;\, \mathbf{p}_t;\, \mathbf{g}_{\lfloor t/K \rfloor}\big] \in \mathbb{R}^{3d}, \tag{5.34}$$

comprising the current entity embedding $\mathbf{h}_t$, the path context $\mathbf{p}_t$, and the active subgoal embedding. The path context equips the worker with memory of previously visited entities, enabling cycle avoidance and coherent multi-step reasoning.

Action Space. At each step t, the worker's action space consists of all entities reachable via a single relational edge from e_t:

$$\mathscr{A}_t = \big\{ e_j \in \mathscr{E} : \exists\, r \in \mathscr{R},\ (e_t, r, e_j) \in \mathscr{T} \big\}. \tag{5.35}$$

To balance exploration with efficiency, SymR applies a soft masking strategy: previously visited entities receive a score penalty rather than being excluded entirely, allowing the worker to backtrack when necessary while discouraging redundant revisitation.

Policy Network. The worker's policy π_W is a parameterized distribution over $\mathscr{A}_t$. For each candidate entity $e_j \in \mathscr{A}_t$, a compatibility score $\phi(\mathbf{s}_t^W, \mathbf{h}_j)$ is computed by combining three signals: a linear projection of $\mathbf{s}_t^W$ encoding the traversal context; the RGAT embedding $\mathbf{h}_j$ of the candidate entity; and an element-wise interaction between the projected state and $\mathbf{h}_j$, analogous to a bilinear interaction term. These are combined through a linear output layer, and the policy is defined as:

$$\pi_W(a_t = e_j \mid \mathbf{s}_t^W; \theta_W) = \frac{\exp\big(\phi(\mathbf{s}_t^W, \mathbf{h}_j)\big)}{\sum_{e_k \in \mathscr{A}_t} \exp\big(\phi(\mathbf{s}_t^W, \mathbf{h}_k)\big)}. \tag{5.36}$$

The inclusion of $\mathbf{g}_m$ in $\mathbf{s}_t^W$ creates an implicit alignment mechanism: candidate entities whose embeddings are close to the active subgoal in the representation space receive higher compatibility scores, biasing the worker's navigation toward semantically relevant regions of the knowledge graph without requiring an explicit reward for each step.

Worker Value Function. The worker maintains a separate value function $V_W : \mathbf{s}_t^W \mapsto \mathbb{R}$, implemented as a two-layer MLP with the same architecture as the manager's value network:

$$V_W(\mathbf{s}_t^W; \theta_W^v) = \mathbf{W}_2^v \,\mathrm{ReLU}\big(\mathbf{W}_1^v \mathbf{s}_t^W + \mathbf{b}_1^v\big) + \mathbf{b}_2^v, \tag{5.37}$$

with parameters θ_W^v kept separate from the policy parameters θ_W. This actor-critic architecture allows the value function to serve as a variance-reducing baseline in the PPO optimization objective.

5.3.4 Hierarchical Reward Structure

Reward design is critical in multi-hop knowledge graph reasoning, as a naive sparse terminal reward creates a severe credit assignment problem: the agent must traverse several intermediate hops before receiving any feedback, making it extremely difficult to determine which early decisions were beneficial. SymR addresses this with a hierarchical reward structure combining five additive components at the worker level:

$$r_t^W = r_t^{\text{ext}} + r_t^{\text{sim}} + r_t^{\text{cycle}} + r_t^{\text{int}} + r_t^{\text{subgoal}}. \tag{5.38}$$

Extrinsic Reward. The sparse terminal reward reflects whether the worker successfully identifies the answer entity:

$$r_t^{\text{ext}} = \begin{cases} R_{\text{success}} = 4.0 & \text{if } e_T = e^* \text{ and } t = T, \\ R_{\text{fail}} = -0.5 & \text{if } e_T \neq e^* \text{ and } t = T, \\ -\beta_{\text{len}} = -0.01 & \text{otherwise.} \end{cases} \tag{5.39}$$

The per-step penalty $-\beta_{\text{len}}$ discourages unnecessarily long reasoning paths, and the mild failure penalty R_{fail} avoids destabilizing training through extremely negative returns.

Similarity-Based Shaping. To provide dense feedback at every step, a shaping reward proportional to the increase in cosine similarity toward the target is applied:

$$r_t^{\text{sim}} = \alpha_{\text{sim}}\big(\text{sim}(\mathbf{h}_{t+1}, \mathbf{e}_{\text{target}}) - \text{sim}(\mathbf{h}_t, \mathbf{e}_{\text{target}})\big), \tag{5.40}$$

where $\alpha_{\text{sim}} > 0$ is a scaling coefficient. This reward is positive when a transition brings the agent closer to the answer in embedding space, significantly mitigating the credit assignment problem inherent to sparse rewards.

Cycle Penalty. To discourage revisitation of previously visited entities, which consume hop budget without advancing the reasoning chain:

$$r_t^{\text{cycle}} = \begin{cases} -\beta_{\text{cycle}} & \text{if } e_{t+1} \in \{e_0, e_1, \ldots, e_t\}, \\ 0 & \text{otherwise.} \end{cases} \tag{5.41}$$

This complements the soft masking in the action space by providing a direct negative signal rather than merely reducing revisitation probability.

Intrinsic Reward. A subgoal-directed progress signal encourages the worker to move toward the currently active semantic subgoal $\mathbf{g}_m$, defined via cosine dissimilarity $d(\mathbf{u}, \mathbf{v}) = 1 - \text{sim}(\mathbf{u}, \mathbf{v})$:

$$r_t^{\text{int}} = \alpha_{\text{int}} \big(d(\mathbf{h}_t, \mathbf{g}_{\lfloor t/K \rfloor}) - d(\mathbf{h}_{t+1}, \mathbf{g}_{\lfloor t/K \rfloor}) \big), \tag{5.42}$$

where $\alpha_{\text{int}} > 0$. This potential-based shaping decomposes the global reasoning problem into a sequence of local navigation objectives, rewarding measurable progress toward each intermediate semantic waypoint.

Subgoal Bonus. A discrete bonus is awarded upon successfully reaching an entity sufficiently close to the active subgoal:

$$r_t^{\text{subgoal}} = \begin{cases} B_{\text{subgoal}} & \text{if } d(\mathbf{h}_{t+1}, \mathbf{g}_{\lfloor t/K \rfloor}) \leq \varepsilon_{\text{subgoal}}, \\ 0 & \text{otherwise,} \end{cases} \tag{5.43}$$

providing a clear, unambiguous positive signal upon subgoal attainment to reinforce successful completion of each semantic waypoint.

Manager Reward. Since the manager operates at a coarser timescale, its reward at step m is the sum of all worker rewards accumulated during the corresponding interval:

$$r_m^M = \sum_{t=mK}^{(m+1)K-1} r_t^W. \tag{5.44}$$

This ensures the manager is credited for the full consequence of its subgoal selection, creating a coherent credit assignment mechanism across both levels of the hierarchy.

5.3.5 Training and Optimization

SymR is trained using Proximal Policy Optimization (PPO) with Generalized Advantage Estimation (GAE), applied independently to the worker and manager components. Both levels share the same advantage estimation framework but differ in their optimization objectives due to the non-parametric nature of the manager's subgoal selection mechanism.

Generalized Advantage Estimation. Accurate advantage estimation is critical for stable policy gradient training. The one-step TD residual at time t is:

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t), \tag{5.45}$$

and the GAE advantage estimate is:

$$A_t = \sum_{l=0}^{T-t-1} (\gamma\lambda)^l \, \delta_{t+l}, \tag{5.46}$$

where $\lambda \in [0,1]$ is the bias-variance tradeoff parameter. When $\lambda = 0$, the estimate reduces to a single-step TD error (low variance, high bias); when $\lambda = 1$, it reduces to a discounted return minus the value baseline (low bias, high variance). In SymR, both levels use $\lambda = 0.95$. The worker uses discount factor $\gamma_W = 0.99$, while the manager uses $\gamma_M = 0.995$ to account for its longer effective horizon - the consequences of a subgoal selection may not fully materialize for K worker steps, necessitating greater weight on delayed rewards.

Worker Optimization. The worker's policy parameters θ_W and value parameters θ_W^v are jointly optimized via the PPO-Clip objective. Let $\rho_t(\theta_W) = \pi_W(a_t \mid s_t; \theta_W)/\pi_W^{\text{old}}(a_t \mid s_t; \theta_W^{\text{old}})$ denote the probability ratio between the updated and behavior policies. The clipped surrogate objective is:

$$\mathscr{L}_W^{\text{CLIP}}(\theta_W) = \mathbb{E}_t[\min(\rho_t(\theta_W)A_t,\ \text{clip}(\rho_t(\theta_W), 1-\varepsilon, 1+\varepsilon)A_t)], \tag{5.47}$$

where $\varepsilon = 0.2$ is the clipping hyperparameter. The full worker loss combines the policy loss, a value regression loss, and an entropy bonus:

$$\mathscr{L}_W(\theta_W) = -\mathscr{L}_W^{\text{CLIP}}(\theta_W) + c_1 \, \mathbb{E}_t\left[(V_W(s_t) - R_t)^2\right] - c_2 \, \mathscr{H}[\pi_W], \tag{5.48}$$

where $R_t = A_t + V_W(s_t)$ is the empirical return, $c_1 = 0.5$ weights the value loss, $\mathscr{H}[\pi_W] = -\sum_a \pi_W(a \mid s_t) \log \pi_W(a \mid s_t)$ is the policy entropy, and $c_2 = 0.01$ weights the entropy bonus. The entropy term discourages premature convergence to a deterministic policy, encouraging continued exploration throughout training.

Manager Optimization. Since the manager's subgoal selection is deterministic and derived from the AMR graph structure rather than a learned stochastic policy, the PPO-Clip objective is not applicable. Only the manager value function V_M is optimized, via a mean-squared error loss against the empirical return:

$$\mathscr{L}_M^V(\theta_M^v) = \mathbb{E}_m\left[\left(V_M(\mathbf{s}_m^M; \theta_M^v) - R_m^M\right)^2\right], \tag{5.49}$$

where $R_m^M = A_m^M + V_M(\mathbf{s}_m^M)$ is the manager-level empirical return and A_m^M is the GAE advantage computed using γ_M and the aggregated worker rewards r_m^M. Maintaining an accurate manager value function remains important even without an explicit policy: it enables GAE to produce lower-variance advantage estimates, improving the calibration of return targets through a self-consistent optimization loop.

Joint Training. The worker and manager are trained jointly in an alternating fashion. During each training iteration, a batch of complete reasoning episodes is collected using the current worker policy and the deterministic manager subgoal mechanism. Worker-level and manager-level experiences are extracted, advantages are computed via GAE at each respective level, and all learnable components - the worker's policy and value networks, the manager's value network, the GRU path encoder, the RGAT knowledge graph encoder, and the AMR GCN encoder - are updated end-to-end via gradient descent on the respective losses.

Chapter 6

Experiment Settings & Results

This chapter presents the experimental design, implementation, and evaluation of the proposed approaches, along with a comparative analysis of results against existing baselines.

6.1 Datasets & Preprocessing

We preprocess each dataset in two main steps to construct suitable knowledge graphs for reasoning:

1. **Triplet Extraction.** For each context, we apply `REBEL`[1] to extract explicit relational triplets. Since not all relevant relations are directly observable, we further employ `gemini-2.0-flash` to infer missing or implicit triplets, enriching the graph structure with additional relational evidence.

2. **Path Construction.** Given a question–answer pair, the model uses `Gemini` to select a sequence of triplets that forms a reasoning path:

   $$e_1 \xrightarrow{r_1} e_2 \cdots e_{n-1} \xrightarrow{r_{n-1}} e_n$$

   where e_n denotes the final answer entity. This reasoning path serves as the foundation for multi-hop inference within our framework.

We evaluate out approaches on four challenging multi-hop question answering benchmarks, namely HotpotQA, 2WikiMultihopQA, and MuSiQue, which require complex reasoning across multiple pieces of evidence. These datasets are widely used to assess the capability of models in multi-hop reasoning, explainability, and robustness.

HotpotQA [36]. HotpotQA is a multi-hop QA dataset with 113,000 questions requiring reasoning across multiple Wikipedia articles, each paired with annotated supporting facts to enable explainable answers. It introduces novel comparison questions that test ability to extract and contrast entity properties like dates or numbers. Designed to push beyond single-document QA, it challenges models to reason holistically in natural language.

[1] https://huggingface.co/Babelscape/rebel-large

2WikiMultihopQA [11]. The 2WikiMultiHopQA dataset is a large, high-quality multi-hop question answering dataset with 192,606 question-answer pairs, created by combining Wikipedia and Wikidata. It features four distinct question types, comparison, inference, compositional, and bridge-comparison, and provides crucial evidence information in the form of structured triples to explain reasoning paths. The dataset is designed to be challenging for multi-hop models, emphasizing the need for comprehensive reasoning and explanation capabilities.

MuSiQue [30]. The MuSiQue dataset is a multihop QA dataset comprising approximately 25,000 questions with 2-4 hops, created by systematically composing single-hop questions. Its key features include enforcing connected reasoning, minimizing train-test leakage, and incorporating challenging distractor contexts. This design results in a dataset that is significantly more difficult for models to solve via shortcuts, promoting the development of genuine multi-hop reasoning capabilities.

Table 6.1 summarizes the number of training and testing samples for each dataset.

Name	Train Samples	Test Samples
HotpotQA [36]	11,065	6,501
2WikiMultihopQA [11]	18,000	12,576
MuSiQue [30]	10,065	1,877

Table 6.1: Training and testing splits of the datasets used in our experiments.

6.2 Baselines

Besides our previous approaches, we compare against three major baseline categories commonly used in multi-hop QA: Large Language Models (LLM), retrieval-augmented generation (RAG), and hybrid methods combining LLMs with knowledge graphs (LLM+KG).

LLM Baselines. Standard LLMs rely solely on parametric knowledge without external retrieval, reflecting pure generative reasoning but often suffering from factual inconsistency and limited knowledge coverage.

RAG Baselines. RAG methods integrate an external retriever to fetch relevant passages as additional context, providing stronger factual grounding than standalone LLMs, though reasoning over multiple retrieved documents remains challenging.

LLM+KG Baselines. These hybrid methods combine language models with structured knowledge graphs, enabling explainable multi-hop reasoning over explicit entity-relation structures, though performance depends heavily on knowledge graph quality and coverage.

Furthermore, we employ **Qwen-2.5** with direct prompting ``Given the context (and KG), answer the question'' as an additional baseline, using the LLM solely at the final stage to convert the predicted reasoning path into a natural language answer.

Our primary objective is **reasoning chain generation** over KGs - KG construction is a necessary condition, not a contribution. We additionally compare against **Qwen-2.5** with full context and KG access to avoid LLM bias, confirming that even a capable small LLM **struggles with accurate multi-hop reasoning** under this setting.

6.3 Training Configurations

For training, we employ the AdamW optimizer with a learning rate of 2×10^{-5} and train both the retrieval and stopping modules for 20 epochs.

We adopt `google-bert/bert-base-uncased` as the backbone encoder to obtain contextual embeddings. Specifically, we extract the `[CLS]` token representation as the sentence-level embedding, which is subsequently fed into our architecture. For reasoning generation, we utilize `gemini-2.5-flash` to produce natural responses conditioned on the retrieved reasoning path.

6.4 Hyperparameter Implementation Settings

The hyperparameters values were selected empirically based on experimental results that yielded the best performance.

6.5 Question Answering Results

We begin by evaluating our framework on the HotpotQA dataset, a widely used benchmark for multi-hop reasoning.

Method	Explainable	Type	EM	F1
SG-Prompt [17]	✓	LLM	61.00	73.30
AnchorCoT [23]	✓	LLM	57.08	66.34
MSRAG [35]	✗	RAG	30.30	30.66
EfficientRAG [43]	✗	RAG	50.59	57.93
HippoRAG [14]	✓	LLM+KG	56.80	69.50
PER-QA [21]	✓	LLM+KG	64.40	67.90
Qwen2.5	✓	LLM	48.46	49.70
Qwen2.5 (KG)	✓	LLM	66.06	68.96
NeuroPath (Ours)	✓	Neural+KG	**70.20**	**74.46**
SymR (Ours)	✓	Neural+KG	**73.63**	**75.92**

Table 6.2: HotpotQA results.

Our method achieves 73.63 EM, outperforming all baselines. LLM-only baselines show limited ability to integrate evidence, while RAG methods struggle with aligning retrieved passages. LLM+KG approaches gain interpretability but remain constrained by graph coverage. Our framework combines neural reasoning with KG traversal, yielding more accurate and explainable predictions.

We next report results on 2WikiMultihopQA, a dataset designed to require deeper reasoning across multiple knowledge sources.

Method	Explainable	Type	EM	F1
SG-Prompt [17]	✓	LLM	49.00	57.90
AnchorCoT [23]	✓	LLM	54.42	63.86
MSRAG [35]	✗	RAG	50.80	56.46
EfficientRAG [43]	✗	RAG	44.18	51.64
HippoRAG [14]	✓	LLM+KG	47.30	52.10
PER-QA [21]	✓	LLM+KG	55.40	70.40
Qwen2.5	✓	LLM	52.13	64.11
Qwen2.5 (KG)	✓	LLM	68.00	73.12
NeuroPath (Ours)	✓	Neural+KG	**67.77**	**70.72**
SymR (Ours)	✓	Neural+KG	**80.91**	**82.43**

Table 6.3: 2WikiMultihopQA results.

Our model reaches 80.91 EM, again surpassing strong baselines. LLMs fail to bridge information across articles, RAG systems are sensitive to retrieval noise, and LLM+KG methods are bounded by coverage issues. By dynamically constructing reasoning paths and using a learned stopping mechanism, our approach generalizes well to complex multi-hop tasks.

Finally, we evaluate on MuSiQue, which is particularly challenging due to its compositional reasoning nature.

Method	Expl.	Type	EM	F1
SToC-ToT [2]	✓	LLM	42.00	55.30
AnchorCoT [23]	✓	LLM	41.83	53.31
SeQGraph [26]	✗	RAG	46.01	56.88
HPE [22]	✗	RAG	45.50	53.70
GeAR [28]	✓	LLM+KG	19.00	35.60
HOLMES [25]	✓	LLM+KG	29.00	38.00
Qwen2.5	✓	LLM	22.59	32.46
Qwen2.5 (KG)	✓	LLM	61.68	66.41
NeuroPath (Ours)	✓	Neural+KG	**58.42**	**62.46**
SymR (Ours)	✓	Neural+KG	**85.83**	**87.25**

Table 6.4: MuSiQue results.

The most compositional benchmark, our framework obtains 85.83 EM, setting a new state of the art. LLM prompting struggles with compositionality, RAG methods lose consistency over long reasoning chains, and LLM+KG systems suffer from noisy graph construction. Our purely neural KG-based reasoning recovers more complete paths and produces reliable answers.

Our Hierarchical RL-based approach-integrating explicit reasoning and a learned stopping mechanism-outperforms LLM-only, RAG, and LLM+KG baselines. By decoupling high-level decision-making from low-level execution, our framework ensures superior multi-hop accuracy, stability, and interpretability.

6.6 Reasoning Paths Analysis

This section analyzes the reasoning paths generated by the proposed approach, with a particular focus on their correctness, structural properties, and deviations from ground-truth paths.

Through representative examples, we examine both successful reasoning behaviors and notable special cases, followed by a discussion of their implications for multi-hop question answering.

6.6.1 Model Output Examples

To illustrate the reasoning capabilities of the proposed model across different levels of complexity, Table 6.5 presents representative examples of generated reasoning paths and final answers for entity-based, short commonsense, and long commonsense multi-hop questions.

Entity Target Question	**Commonsense (Short)**	**Commonsense (Long)**
Question: What is the place of birth of Clara Novello's father? **Reasoning Path:** • Clara Anastasia Novello $\xrightarrow{\text{father}}$ Vincent Novello • Vincent Novello $\xrightarrow{\text{place of birth}}$ London **Model Response:** The path indicates that Clara Anastasia Novello's father is Vincent Novello, and Vincent Novello's place of birth is London. Therefore, Clara Novello's father was born in London. **Answer:** London	**Question:** Which film came out first, *Repeated Absences* or *She Wore A Yellow Ribbon*? **Reasoning Path:** • *Repeated Absences* $\xrightarrow{\text{publication date}}$ 1972 • *She Wore A Yellow Ribbon* $\xrightarrow{\text{publication date}}$ 1950 **Model Response:** The question asks which film came out first. Path 0 indicates that *Repeated Absences* was released in 1972, and path 1 indicates that *She Wore A Yellow Ribbon* was released in 1950. Since 1950 is earlier than 1972, *She Wore A Yellow Ribbon* came out first. **Answer:** She Wore A Yellow Ribbon	**Question:** Which film has the director who died earlier, *Toen 'T Licht Verdween* or *Boum Sur Paris*? **Reasoning Path:** • *Toen 'T Licht Verdween* $\xrightarrow{\text{director}}$ Maurits Binger $\xrightarrow{\text{date of death}}$ April 9, 1923 • *Boum Sur Paris* $\xrightarrow{\text{director}}$ Maurice de Canonge $\xrightarrow{\text{date of death}}$ January 10, 1979 **Model Response:** The question asks which film has a director who died earlier. Path 0 and Path 1 tell us the directors of the two films: *Toen 'T Licht Verdween* was directed by Maurits Binger, and *Boum Sur Paris* was directed by Maurice de Canonge. Path 0 tells us Maurits Binger died on April 9, 1923. Path 1 tells us Maurice de Canonge died on January 10, 1979. Since 1923 is earlier than 1979, Maurits Binger died earlier than Maurice de Canonge. Therefore, the director of *Toen 'T Licht Verdween* died earlier. **Answer:** Toen 'T Licht Verdween

Table 6.5: Representative examples of model-generated reasoning paths and answers for entity-based, short commonsense, and long commonsense multi-hop questions.

These examples demonstrate that the model is capable of producing explicit, interpretable reasoning paths that correctly chain multiple relational hops. Importantly, the generated explanations remain faithful to the underlying paths and lead to correct final answers, even for longer commonsense queries requiring temporal comparison.

6.6.2 Special Cases of Reasoning Path Deviations

Beyond fully correct cases, we observe several characteristic deviations between predicted and ground-truth reasoning paths. These special cases are summarized below.

Type	Question and Reasoning Paths	Comments
Exact	**Question:** The American short horror *The Sub* stars which American actress? **Target Path:** *The Sub* $\xrightarrow{\text{cast member}}$ Heather Langenkamp **Predicted Path:** *The Sub* $\xrightarrow{\text{cast member}}$ Heather Langenkamp	The predicted reasoning path exactly matches the target path, demonstrating accurate relation selection and effective stopping behavior, resulting in a minimal and precise reasoning chain.
Longer	**Question:** What year was the father of Katherine of England crowned? **Target Path:** Katherine of England $\xrightarrow{\text{father}}$ Henry III $\xrightarrow{\text{crowned in}}$ Gloucester Cathedral $\xrightarrow{\text{inception}}$ 1216 **Predicted Path:** Katherine of England $\xrightarrow{\text{child of}}$ Eleanor of Provence $\xrightarrow{\text{child}}$ Catherine of England $\xrightarrow{\text{mother}}$ Eleanor of Provence $\xrightarrow{\text{spouse}}$ Henry II $\xrightarrow{\text{crowned in}}$ Gloucester Cathedral $\xrightarrow{\text{inception}}$ 1216	The predicted path is longer than necessary but remains semantically valid and leads to the correct answer. This indicates robustness to redundant relational hops, although with reduced path optimality.
Shorter	**Question:** *A Wind in the Door* is part of a series written by who? **Target Path:** *A Wind in the Door* $\xrightarrow{\text{part of series}}$ Time Quintet $\xrightarrow{\text{author}}$ Madeleine L'Engle **Predicted Path:** *A Wind in the Door* $\xrightarrow{\text{author}}$ Madeleine L'Engle	The model bypasses the intermediate series node and directly retrieves the correct author. Although shorter than the target path, the reasoning remains correct and demonstrates effective multi-hop compression.

Table 6.6: Comparison between target and predicted reasoning paths, illustrating exact matches, over-extended paths, and compressed paths produced by the model.

Overall, the examples in Table 6.6 demonstrate that the proposed model is capable of generating explicit and semantically meaningful reasoning paths that reliably lead to correct answers across diverse multi-hop question types. In cases where the predicted path exactly matches the target path, the model exhibits precise relation selection and effective stopping behavior. Even when deviations occur, such as over-extended paths with redundant hops or compressed paths that bypass intermediate entities, the model generally preserves semantic validity and answer correctness. These behaviors indicate a degree of robustness and flexibility in the learned reasoning policy, allowing the model to tolerate non-optimal path structures while maintaining factual accuracy. At the same time, the presence of longer-than-necessary paths highlights opportunities for further refinement in path optimality and efficiency, particularly in controlling unnecessary exploration during multi-hop traversal.

6.6.3 Qualitative Comparison Between NeuroPath and SymR

Table 6.7 presents a qualitative comparison between SymR and NeuroPath across three two-hop questions. Across all cases, SymR successfully traverses the full reasoning chain to reach the correct answer. NeuroPath, by contrast, exhibits two recurring failure modes: premature stopping, where the model halts at an intermediate entity rather than completing the chain, and incorrect relation selection, where a plausible but wrong relation leads the traversal astray. These examples complement the quantitative results and highlight the advantage of SymR's structured subgoal-driven reasoning.

This shows that our updated SymR from NeuroPath resolves the problem by using symbolic-based subgoal directions.

	QA Results	Comments
Question 1	Which company owns the manufacturer of Learjet 60?	
SymR	**Reasoning Path:** Learjet 60 $\xrightarrow{\text{manufacturer}}$ Bombardier Aerospace $\xrightarrow{\text{owned by}}$ Bombardier Inc. **Predicted Answer:** Bombardier Inc.	SymR correctly resolves the two-hop chain with a minimal and precise path, exactly matching the target answer.
NeuroPath	**Reasoning Path:** Learjet 60 $\xrightarrow{\text{manufacturer}}$ Bombardier Aerospace **Predicted Answer:** Bombardier Aerospace	NeuroPath stops prematurely after retrieving the manufacturer, conflating it with the parent company and failing to traverse the ownership relation.
Question 2	The developer of Mozilla Sunbird created which browser?	
SymR	**Reasoning Path:** Mozilla Sunbird $\xrightarrow{\text{owned by}}$ Mozilla $\xrightarrow{\text{developer}}$ Firefox **Predicted Answer:** Firefox	SymR correctly identifies the developer and retrieves the browser product in a concise two-hop path.
NeuroPath	**Reasoning Path:** Mozilla Sunbird $\xrightarrow{\text{developer}}$ Mozilla Foundation $\xrightarrow{\text{product}}$ Thunderbird **Predicted Answer:** Thunderbird	NeuroPath selects an incorrect relation at the second hop, retrieving a sibling product rather than the target browser, resulting in a wrong answer despite a plausible-looking path.
Question 3	Who was married to the creator of *Ruby Loftus Screwing a Breech Ring*?	
SymR	**Reasoning Path:** *Ruby Loftus Screwing a Breech Ring* $\xrightarrow{\text{creator}}$ Laura Knight $\xrightarrow{\text{spouse}}$ Harold Knight **Predicted Answer:** Harold Knight	SymR correctly identifies the creator and traverses the spouse relation to reach the target answer in a minimal two-hop chain.
NeuroPath	**Reasoning Path:** *Ruby Loftus Screwing a Breech Ring* $\xrightarrow{\text{creator}}$ Laura Knight **Predicted Answer:** Laura Knight	NeuroPath stops after identifying the creator, failing to traverse the spouse relation and returning the wrong entity as the final answer.

Table 6.7: Comparison between SymR and NeuroPath across three multi-hop questions. SymR consistently produces correct answers; NeuroPath fails due to premature stopping or incorrect relation selection.

6.6.4 Success Rate and Convergence Analysis

We analyze the learning dynamics of various reinforcement learning methods by examining their success rates and convergence behaviors over training epochs.

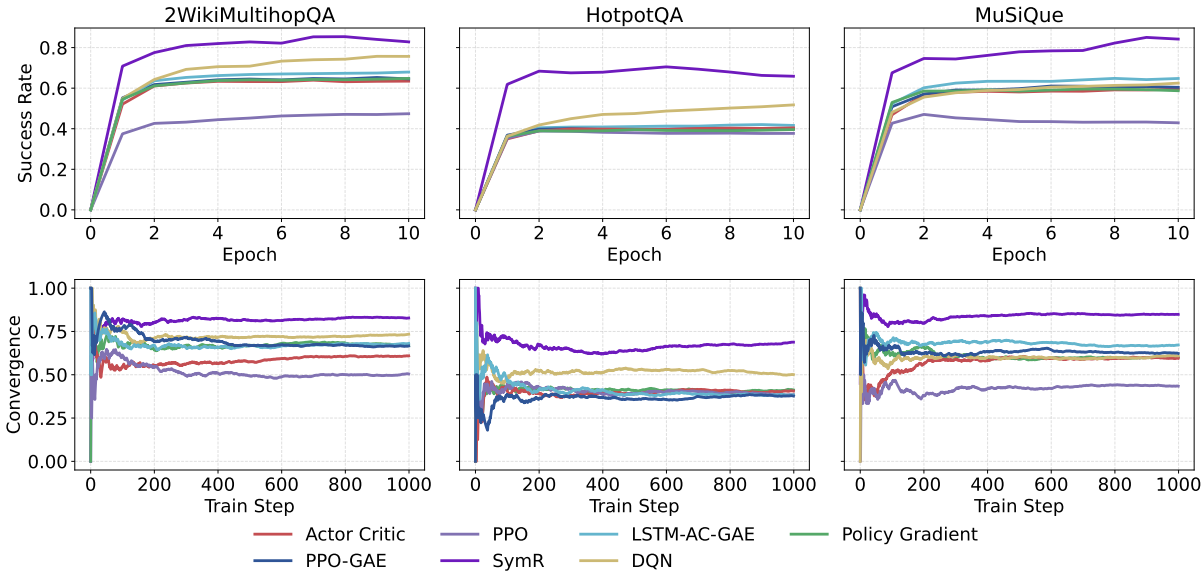


Figure 6.1: Success rate over training epochs (top) and convergence rate (down) across three datasets.

As illustrated in Figure 6.1, flat RL baselines exhibit slow and often unstable improvements. This phenomenon can be attributed to two fundamental challenges: sparse reward signals and long decision horizons. In multi-hop reasoning tasks, rewards are typically only observed upon reaching a correct terminal node, resulting in delayed feedback that hinders effective credit assignment.

Let $G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k}$ denote the return at time step t. In sparse reward settings, $R_t = 0$ for most time steps, which leads to high variance in gradient estimates when optimizing policy-based objectives:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta} \left[\nabla_\theta \log \pi_\theta(a_t \mid s_t) \cdot G_t \right]. \tag{6.1}$$

Such high-variance updates contribute to instability and slow convergence. Moreover, the exponential growth of possible action sequences exacerbates the exploration problem.

In contrast, the proposed SymR framework addresses these challenges through hierarchical decomposition. Specifically, trajectories are segmented into semantically coherent subgoals, each associated with intermediate intrinsic rewards. These rewards provide denser supervision signals, effectively reshaping the reward landscape.

As a result, SymR demonstrates significantly faster convergence and achieves higher success rates across all evaluated datasets. The structured exploration induced by semantic subgoals allows the agent to focus on meaningful transitions, reducing the effective search space and improving learning stability.

6.6.5 Chain Generation Results

We evaluate the effectiveness of different reinforcement learning approaches in generating accurate reasoning chains across multiple benchmark datasets. The results are summarized in Table 6.8.

Navigation accuracy is defined as the percentage of queries for which the agent successfully identifies a valid multi-hop reasoning path leading to the correct answer. Formally, let $\mathscr{D}$ denote the set of evaluation queries and let $\hat{y}_i$ and y_i denote the predicted and ground-truth reasoning outcomes for query i, respectively. The accuracy is computed as:

$$\text{Accuracy} = \frac{1}{|\mathscr{D}|} \sum_{i \in \mathscr{D}} \mathbf{1}(\hat{y}_i = y_i), \tag{6.2}$$

where $\mathbf{1}(\cdot)$ is the indicator function.

The empirical results demonstrate that SymR consistently outperforms all baseline methods across the HotpotQA, 2WikiQA, and MuSiQue datasets. Notably, while value-based methods such as DQN achieve relatively strong performance due to improved sample efficiency, they still fall short of SymR's results.

The superior performance of SymR can be attributed to its hierarchical control mechanism and the incorporation of semantic guidance. By structuring the reasoning process into intermediate subgoals, the model effectively reduces the complexity of long-horizon decision-making and mitigates the impact of sparse rewards.

Furthermore, the symbolic component enables better generalization by enforcing structured reasoning patterns, which are particularly beneficial in multi-hop question answering tasks where logical consistency is critical.

Overall, these findings highlight the importance of hierarchical and semantically guided reinforcement learning for scalable and robust multi-hop reasoning over complex knowledge graphs.

Table 6.8: Navigation accuracy (%) across three datasets.

Method	**Type**	**HotpotQA**	**2WikiQA**	**MuSiQue**
Policy Gradient	Neural	46.20	56.83	61.18
Actor–Critic (AC)	Value	41.61	54.32	60.59
LSTM–AC	Memory	45.79	64.65	74.10
PPO	Value	47.60	45.91	66.57
PPO–GAE	Deep	46.73	63.31	66.13
DQN	Deep	60.71	79.30	76.09
SymR	Symbolic	**75.79**	**83.29**	**87.45**

6.6.6 Analysis and Discussion

The above cases reveal several important characteristics of the proposed reasoning framework. First, correctness of the final answer does not strictly depend on exact alignment with the annotated ground-truth path; alternative paths may exist that are shorter or longer yet remain logically valid. Second, longer predicted paths often arise from exploratory behavior in the graph, indicating that the model prioritizes relational connectivity over strict path optimality. Conversely, shorter paths suggest that the model can exploit implicit relational shortcuts when such information is directly available.

These observations highlight a key advantage of explicit reasoning path generation: it enables fine-grained inspection of model behavior beyond answer accuracy alone. At the same time, they expose challenges related to path optimality and stopping decisions, motivating the incorporation of learned termination mechanisms and hierarchical control strategies. Overall, the analysis confirms that the proposed approach produces interpretable and flexible reasoning paths, while also revealing avenues for further refinement in controlling reasoning depth and efficiency.

Chapter 7

Conclusion

This chapter presents the conclusions of the thesis, highlighting key findings and suggesting directions for future research.

7.1 Summary

In this thesis, we investigated the problem of multi-hop question answering on knowledge graphs through two complementary approaches: a Neural reasoning model (**NeuroPath**) and a Symbolic Hierarchical Reinforcement Learning framework. The Neural approach provides a foundation for relational reasoning using deep learning architectures, while the HRL framework - realized as **SymR** - leverages a multi-level decision-making structure to better reflect the hierarchical and compositional nature of multi-hop inference.

SymR models reasoning chain generation as a hierarchical decision process in which a high-level manager selects symbolic subgoals derived from the Abstract Meaning Representation (AMR) of the input question, and a low-level worker navigates the knowledge graph to satisfy these subgoals. This design integrates structured semantic guidance directly into the reinforcement learning loop, reducing spurious reasoning paths and improving alignment between natural language questions and graph traversal. A relational graph attention network (RGAT) encodes knowledge graph structure, while a dedicated GRU-based path encoder maintains traversal history across hops. A hierarchical reward structure combining extrinsic terminal rewards, similarity-based shaping, cycle penalties, and subgoal-completion bonuses provides dense feedback at both policy levels, enabling effective credit assignment in long-horizon settings.

In addition to proposing these models, we presented a comprehensive theoretical background on deep neural networks and reinforcement learning, including mathematical formulations and supporting analyses. Experimental evaluations on three standard multi-hop question answering benchmarks - HotpotQA, 2WikiMultihopQA, and MuSiQue - demonstrate that SymR consistently outperforms both flat reinforcement learning baselines and competitive LLM, RAG, and LLM+KG methods, achieving particular gains on datasets requiring longer reasoning chains and higher compositionality. Beyond accuracy, SymR produces explicit and interpretable reasoning paths, offering faithful symbolic grounding for downstream language model-based answer generation. These findings highlight the benefit of decomposing the multi-hop reasoning process into structured subtasks guided by AMR-grounded hierarchical policies.

7.2 Possible Improvements

Despite its advantages, the SymR framework relies on an off-the-shelf AMR parser to extract symbolic subgoals. The quality of high-level guidance is therefore directly dependent on the robustness of the upstream semantic parsing component, and parsing errors may propagate into suboptimal subgoal sequences. Joint or end-to-end learning of semantic representations and reasoning policies could mitigate this coupling.

Furthermore, the current manager policy follows a deterministic AMR-traversal strategy rather than a fully learned stochastic policy. While this design improves interpretability and reduces the search space, it may limit the framework's ability to adapt its planning strategy to the specific demands of individual questions. Introducing a learned manager policy while retaining AMR-based initialization could offer a better balance between semantic grounding and adaptive control.

Another area for improvement lies in the reward structure. A more carefully shaped reward function - potentially incorporating logical consistency checks or entity-type constraints - could allow the worker to detect incorrect reasoning trajectories and encourage re-exploration of more promising relational paths. Integrating confidence estimation or uncertainty-aware decision-making may also strengthen the framework's ability to avoid or recover from erroneous reasoning chains. Finally, the current framework assumes access to a static, pre-constructed knowledge graph; extending SymR to operate over dynamic or incomplete knowledge graphs would broaden its applicability to real-world settings.

7.3 Future Plans

Future work will focus on the following research directions:

- **Vietnamese-specific text embedding models.** We aim to develop specialized text embedding models for the Vietnamese language that can effectively handle language-specific tokenization challenges, such as word segmentation ambiguity and compound expressions. This includes exploring advanced Transformer-based architectures and domain-adaptive pretraining strategies to improve semantic representation quality for knowledge graph reasoning tasks.
- **Large-scale Vietnamese knowledge graph reasoning dataset.** We plan to construct a large-scale dataset for multi-hop knowledge graph reasoning in the Vietnamese language, with a particular focus on the education domain. The dataset will consist of structured knowledge graphs and corresponding reasoning queries, addressing the current lack of high-quality Vietnamese resources for complex reasoning and question answering.
- **Application and evaluation of the proposed SymR framework.** The proposed hierarchical reinforcement learning framework will be applied and evaluated using the developed Vietnamese embeddings and dataset. This direction aims to assess the robustness, generalization, and effectiveness of SymR in a low-resource linguistic setting and to validate its applicability beyond English-centric benchmarks.

Overall, these future research directions aim to extend the proposed framework toward a more comprehensive and practically applicable reasoning system for the Vietnamese language. By jointly advancing language-specific representation learning, dataset construction, and hierarchical reinforcement learning-based reasoning, this line of work seeks to bridge the gap between

theoretical modeling and real-world deployment, while contributing foundational resources for Vietnamese knowledge graph reasoning and multi-hop question answering.

References

[1] Laura Banarescu et al. “Abstract Meaning Representation for Sembanking”. In: *Proceedings of the 7th Linguistic Annotation Workshop and Interoperability with Discourse*. Sofia, Bulgaria: Association for Computational Linguistics, Aug. 2013, pp. 178–186.

[2] Zhenyu Bi et al. “StoC-TOT: Stochastic Tree-of-Thought with Constrained Decoding for Complex Reasoning in Multi-Hop Question Answering”. In: *Proceedings of the 4th International Workshop on Knowledge-Augmented Methods for Natural Language Processing*. 2025.

[3] Yan Chen, Shuai Sun, and Xiaochun Hu. “DRKG: Faithful and Interpretable Multi-Hop Knowledge Graph Question Answering via LLM-Guided Reasoning Plans”. In: *Applied Sciences* 15.12 (2025), p. 6722.

[4] Zheng Chu et al. “BeamAggR: Beam Aggregation Reasoning over Multi-source Knowledge for Multi-hop Question Answering”. In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Bangkok, Thailand: Association for Computational Linguistics, Aug. 2024, pp. 1229–1248.

[5] Hyung Won Chung et al. “Scaling instruction-finetuned language models”. In: *J. Mach. Learn. Res.* 25.1 (Jan. 2024). ISSN: 1532-4435.

[6] Rajarshi Das et al. “Go for a Walk and Arrive at the Answer: Reasoning Over Paths in Knowledge Bases using Reinforcement Learning”. In: *International Conference on Learning Representations*. 2018.

[7] Zhenyun Deng et al. “Interpretable AMR-Based Question Decomposition for Multi-hop Question Answering”. In: *Proceedings of the Thirty-First International Joint Conference on Artificial Intelligence, IJCAI-22*. Main Track. International Joint Conferences on Artificial Intelligence Organization, July 2022, pp. 4093–4099.

[8] Andrew Drozdov et al. “Inducing and Using Alignments for Transition-based AMR Parsing”. In: *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Ed. by Marine Carpuat, Marie-Catherine de Marneffe, and Ivan Vladimir Meza Ruiz. Seattle, United States: Association for Computational Linguistics, July 2022, pp. 1086–1098.

[9] Jeffrey Flanigan et al. “A Discriminative Graph-Based Parser for the Abstract Meaning Representation”. In: *Proceedings of the 52nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Ed. by Kristina Toutanova and Hua Wu. Baltimore, Maryland: Association for Computational Linguistics, June 2014, pp. 1426–1436.

[10] Simeng Han et al. “FOLIO: Natural Language Reasoning with First-Order Logic”. In: *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. Miami, Florida, USA: Association for Computational Linguistics, Nov. 2024, pp. 22017–22031.

[11] Xanh Ho et al. “Constructing A Multi-hop QA Dataset for Comprehensive Evaluation of Reasoning Steps”. In: *Proceedings of the 28th International Conference on Computational Linguistics*. Barcelona, Spain (Online): International Committee on Computational Linguistics, Dec. 2020.

[12] Sepp Hochreiter and Jürgen Schmidhuber. “Long short-term memory”. In: *Neural computation* 9.8 (1997), pp. 1735–1780.

[13] Guangming Huang et al. “Prompting Explicit and Implicit Knowledge for Multi-hop Question Answering Based on Human Reading Process”. In: *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*. Torino, Italia: ELRA and ICCL, May 2024, pp. 13179–13189.

[14] Bernal Jimenez Gutierrez et al. “Hipporag: Neurobiologically inspired long-term memory for large language models”. In: *Advances in Neural Information Processing Systems* 37 (2024).

[15] Tushar Khot et al. “Decomposed Prompting: A Modular Approach for Solving Complex Tasks.” In: *ICLR*. OpenReview.net, 2023.

[16] Ioannis Konstas et al. “Neural AMR: Sequence-to-Sequence Models for Parsing and Generation”. In: *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Ed. by Regina Barzilay and Min-Yen Kan. Vancouver, Canada: Association for Computational Linguistics, July 2017, pp. 146–157.

[17] Ruosen Li and Xinya Du. “Leveraging Structured Information for Explainable Multi-hop Question Answering and Reasoning”. In: *Findings of the Association for Computational Linguistics: EMNLP 2023*. 2023.

[18] Xiang Li et al. “Teaching Small Language Models to Reason for Knowledge-Intensive Multi-Hop Question Answering”. In: *Findings of the Association for Computational Linguistics: ACL 2024*. Bangkok, Thailand: Association for Computational Linguistics, Aug. 2024, pp. 7804–7816.

[19] Jinzhi Liao et al. “To hop or not, that is the question: Towards effective multi-hop reasoning over knowledge graphs”. In: *World Wide Web* 24.5 (2021), pp. 1837–1856.

[20] Xi Victoria Lin, Richard Socher, and Caiming Xiong. “Multi-Hop Knowledge Graph Reasoning with Reward Shaping”. In: *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*. 2018.

[21] Qichuan Liu et al. “Beyond the Answer: Advancing Multi-Hop QA with Fine-Grained Graph Reasoning and Evaluation”. In: *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 2025.

[22] Ye Liu et al. “HPE: Answering Complex Questions over Text by Hybrid Question Parsing and Execution”. In: *Findings of the Association for Computational Linguistics: EMNLP 2023*. 2023.

[23] Tianshi Ming et al. “AnchorCoT: Anchors Pave the Way for Multi-hop Reasoning”. In: *Findings of the Association for Computational Linguistics: ACL 2025*. 2025.

[24] Martha Palmer, Daniel Gildea, and Paul Kingsbury. “The Proposition Bank: An Annotated Corpus of Semantic Roles”. In: *Computational Linguistics* 31.1 (2005), pp. 71–106.

[25] Pranoy Panda et al. “HOLMES: Hyper-Relational Knowledge Graphs for Multi-hop Question Answering using LLMs”. In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 2024.

[26] Gowtham Ramesh, Makesh Narsimhan Sreedhar, and Junjie Hu. “Single Sequence Prediction over Reasoning Graphs for Multi-hop QA”. In: *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 2023.

[27] Yelong Shen et al. “M-walk: Learning to walk over graphs using monte carlo tree search”. In: *Advances in Neural Information Processing Systems* 31 (2018).

[28] Zhili Shen et al. “GeAR: Graph-enhanced Agent for Retrieval-augmented Generation”. In: *CoRR* (2024).

[29] Harsh Trivedi et al. “Interleaving Retrieval with Chain-of-Thought Reasoning for Knowledge-Intensive Multi-Step Questions”. In: *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Toronto, Canada: Association for Computational Linguistics, July 2023, pp. 10014–10037.

[30] Harsh Trivedi et al. “MuSiQue: Multihop Questions via Single-hop Question Composition”. In: *Transactions of the Association for Computational Linguistics* 10 (2022).

[31] Ashish Vaswani et al. “Attention is all you need”. In: *Advances in neural information processing systems* 30 (2017).

[32] Petar Veličković et al. “Graph Attention Networks”. In: *International Conference on Learning Representations*. 2018.

[33] Nan Wang et al. “KG-o1: Enhancing Multi-hop Question Answering in Large Language Models via Knowledge Graph Integration”. In: (2025).

[34] Jian Wu et al. “GenDec: A robust generative Question-decomposition method for Multi-hop reasoning”. In: (2024).

[35] Ridong Wu et al. “A multi-source retrieval question answering framework based on RAG”. In: *2024 5th International Conference on Information Science, Parallel and Distributed Systems (ISPDS)*. IEEE. 2024.

[36] Zhilin Yang et al. “HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering”. In: *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*. 2018.

[37] Semih Yavuz et al. “Modeling Multi-hop Question Answering as Single Sequence Prediction”. In: *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Dublin, Ireland: Association for Computational Linguistics, May 2022, pp. 974–990.

[38] Eric Zelikman et al. “STaR: self-taught reasoner bootstrapping reasoning with reasoning”. In: *Proceedings of the 36th International Conference on Neural Information Processing Systems*. NIPS ’22. New Orleans, LA, USA: Curran Associates Inc., 2022. ISBN: 9781713871088.

[39] Xiangrui Zhang et al. “TreeQA: Enhanced LLM-RAG with logic tree reasoning for reliable and interpretable multi-hop question answering”. In: *Knowledge-Based Systems* 330 (2025), p. 114526. ISSN: 0950-7051.

[40] Ben Zhou et al. “Learning to Decompose: Hypothetical Question Decomposition Based on Comparable Texts”. In: *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*. Abu Dhabi, United Arab Emirates: Association for Computational Linguistics, Dec. 2022, pp. 2223–2235.

[41] Denny Zhou et al. “Least-to-Most Prompting Enables Complex Reasoning in Large Language Models”. In: *Proceedings of the Eleventh International Conference on Learning Representations (ICLR)*. 2023.

[42] Anjie Zhu et al. “Step by step: A hierarchical framework for multi-hop knowledge graph reasoning with reinforcement learning”. In: *Knowledge-Based Systems* 248 (2022), p. 108843.

[43] Ziyuan Zhuang et al. “EfficientRAG: Efficient Retriever for Multi-Hop Question Answering”. In: *EMNLP*. 2024.

Appendix A

Workload Description

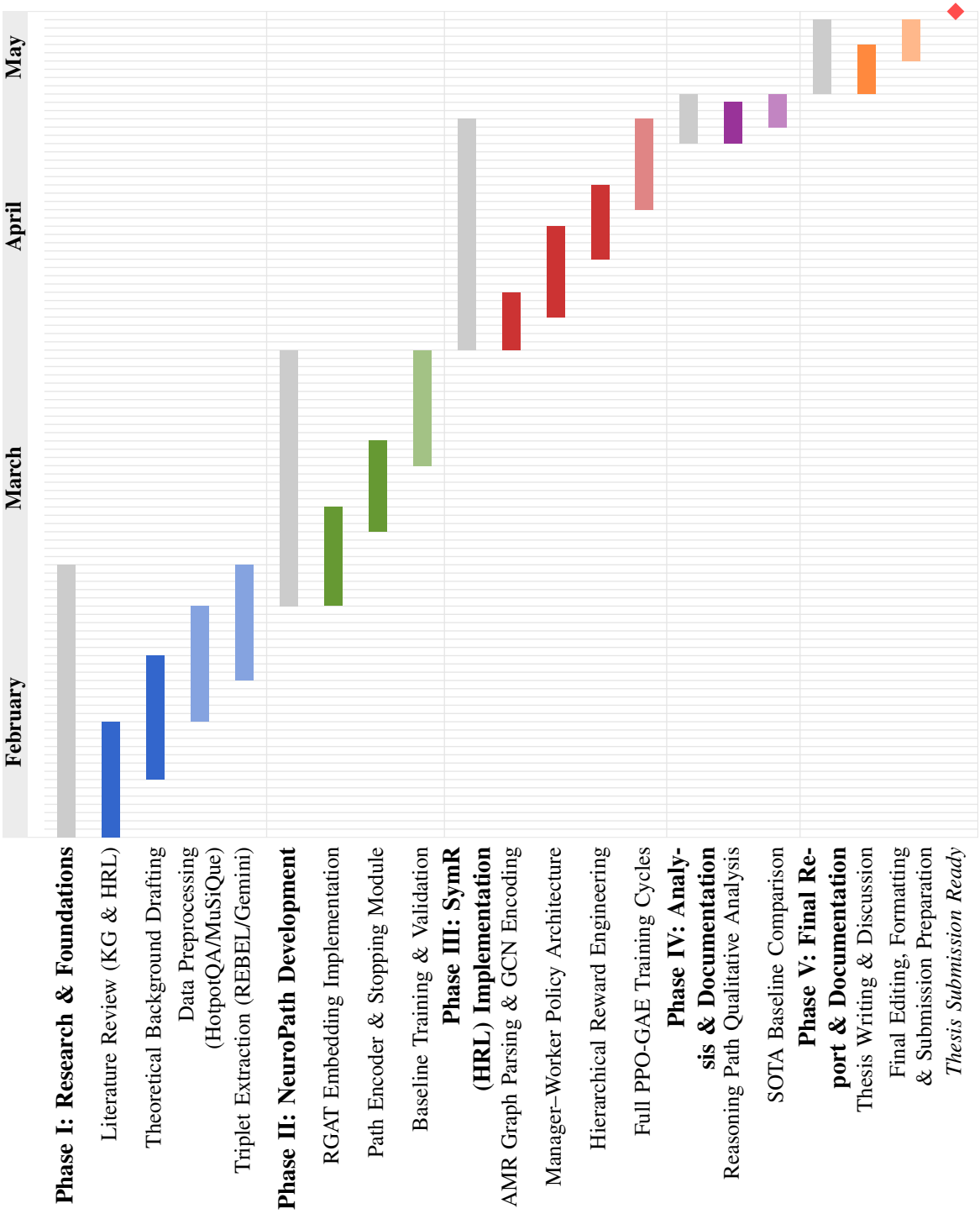
As the sole researcher responsible for this thesis, the author independently carried out all technical, experimental, and analytical components of the study. The major contributions and tasks completed include:

- **Research Formulation:** Investigating the fundamental challenges of multi-hop reasoning over knowledge graphs and defining the research objectives.
- **Literature Review:** Conducting an extensive review of knowledge graph reasoning, neural-symbolic methods, and hierarchical reinforcement learning.
- **Theoretical Derivation:** Presenting formal definitions and mathematical formulations for deep learning (GNNs, Transformers) and reinforcement learning (MDPs, PPO, GAE).
- **Graph Construction:** Implementing a preprocessing pipeline using REBEL and Gemini-2.0-flash for triplet extraction and reasoning path construction.
- **NeuroPath Implementation:** Developing a Neural reasoning framework including a Relational Graph Attention Network (RGAT) and an automated stopping module.
- **SymR Framework Design:** Designing and implementing the Symbolic Reinforcement (SymR) model with a two-level Hierarchical RL architecture.
- **Symbolic Integration:** Encoding Abstract Meaning Representation (AMR) graphs to provide high-level semantic guidance for the RL agent.
- **Reward Engineering:** Deriving a hierarchical reward structure combining extrinsic signals with similarity-based shaping and subgoal-completion bonuses.
- **Experimental Evaluation:** Executing end-to-end training (20 epochs) and testing across HotpotQA, 2WikiMultihopQA, and MuSiQue benchmarks.
- **Analytical Assessment:** Performing detailed reasoning path analysis and success rate/-convergence studies to validate model robustness.
- **Documentation:** Preparing all quantitative visualizations, tables, and the final thesis document.

All conceptual design, implementation, experimentation, and documentation stages of the research were completed solely by the author.

Appendix B

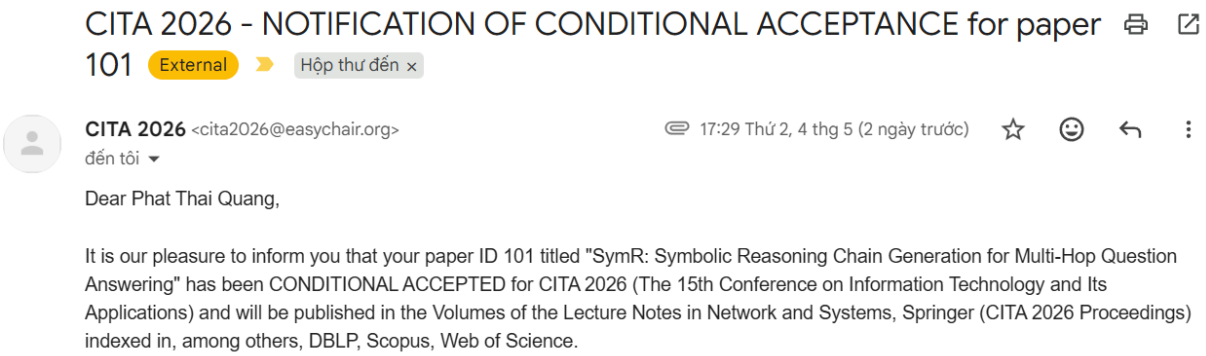
Timeline Of Workload



Appendix C

Publications

1. **Phat Thai**, Trong Le, Thang Bui, Tuan Bui, Tho Quan. *SymR: Symbolic Reasoning Chain Generation for Multi-Hop Question Answering*. In: *Proceedings of the 15th Conference on Information Technology and Its Applications (CITA 2026)*. Lecture Notes in Networks and Systems (Springer), conditional acceptance, 2026.


CITA 2026 - NOTIFICATION OF CONDITIONAL ACCEPTANCE for paper 101 External Hộp thư đến ×

CITA 2026 <cita2026@easychair.org> 17:29 Thứ 2, 4 thg 5 (2 ngày trước)
đến tôi

Dear Phat Thai Quang,

It is our pleasure to inform you that your paper ID 101 titled "SymR: Symbolic Reasoning Chain Generation for Multi-Hop Question Answering" has been CONDITIONAL ACCEPTED for CITA 2026 (The 15th Conference on Information Technology and Its Applications) and will be published in the Volumes of the Lecture Notes in Network and Systems, Springer (CITA 2026 Proceedings) indexed in, among others, DBLP, Scopus, Web of Science.

2. **Phat Thai**, Trong Le, Thang Bui, Tho Quan, Tuan Bui. *NeuroPath: Stepwise Knowledge Graph Reasoning with Neural Intelligence*. In: *Computational Intelligence in Engineering Science*. Springer Nature Switzerland, 2026, pp. 30–44. DOI: `https://doi.org/10.1007/978-3-032-21625-0_3`


Home > Computational Intelligence in Engineering Science > Conference paper

NeuroPath: Stepwise Knowledge Graph Reasoning with Neural Intelligence

Conference paper | First Online: 05 April 2026

pp 30–44 | Cite this conference paper

Save conference paper

Computational Intelligence in Engineering Science

(ICCIES 2026)

Phat Thai, Trong Le, Thang Bui, Tho Quan & Tuan Bui

Access this chapter